



Manual do Utilizador

Tipo de Documento Manual

Número do Documento D2.A

Título Manual do Utilizador

Autor Principal Igor Coelho | Instituto de Telecomunicações

Versão / Estado 1 | Final

Data 30/07/2026

Nível de Distribuição PU (public)



arte



**Financiado pela
União Europeia**
NextGenerationEU

INFORMAÇÃO GERAL

DETALHES DO PROJETO

Acrónimo	DT4MOB
Título	Digital Twins for Mobility
Website	https://dt4mob.av.it.pt/
Coordenador	Instituto de Telecomunicações
Referência	2025.00153.DT4ST

CONTRIBUIDORES

Nome	Organização
Bernardo Figueiredo	Instituto de Telecomunicações
Cecília Santos	Instituto de Telecomunicações
Igor Coelho	Instituto de Telecomunicações
João Capucho	Instituto de Telecomunicações
Tomás Victal	Instituto de Telecomunicações
Zakhar Kruptsala	Instituto de Telecomunicações

Financiado pela União Europeia - NextGenerationEU. No entanto, os pontos de vista e opiniões expressos são exclusivamente do(s) autor(es) e não refletem necessariamente os pontos de vista e opiniões da União Europeia ou da Comissão Europeia. Nem a União Europeia nem a Comissão Europeia são responsáveis por eles.

ÍNDICE

1	INTRODUÇÃO	14
1.1	Visão Geral e Arquitetura do projeto	14
2	GARBAGE COLLECTOR	16
2.1	Visão Geral	16
2.2	Pré-requisitos	16
2.3	Configuração	16
2.3.1	Geral	16
2.3.2	Ligação Ditto	16
2.3.3	Autenticação (OIDC)	17
2.4	Implantação	17
2.4.1	Opção 1: Local com Docker	17
2.4.2	Opção 2: Local com Python + uv	18
2.4.3	Opção 3: Kubernetes via Helm	18
2.5	Como Funciona	19
2.6	Monitorização e Logs	19
2.7	Resolução de Problemas	20
3	FIRE-SIMULATOR	20
3.1	Visão Geral	20
3.1.1	Máquina de Estados	20
3.2	Pré-requisitos	21
3.3	Configuração	21
3.3.1	Configurações Comuns	21
3.3.2	Ligação Ditto	21
3.3.3	Autenticação (Keycloak / OIDC)	22
3.3.4	S3 / SeaweedFS	22
3.3.5	Fontes de Dados Raster (Preparação Única)	23
3.4	Preparação de Dados (Única)	23
3.5	Implantação	23
3.5.1	Opção A: Docker Compose	23
3.5.2	Opção B: Helm (Kubernetes)	24
3.5.3	Utilização (Fluxo de Eventos)	24
3.5.4	Monitorização	25
3.6	Resultados	25
4	DITTO EVENTS API	25
4.1	Visão Geral	25
4.2	Configuração	25

4.2.1	Geral	26
4.2.2	Autenticação (Keycloak OIDC)	26
4.2.3	TimescaleDB	26
4.2.4	Delimitador de Configuração Aninhada	26
4.3	Execução Local	27
4.3.1	Pré-requisitos	27
4.3.2	Configuração	27
4.3.3	Execução	27
4.4	Execução com Docker	27
4.4.1	Construção	27
4.4.2	Execução	27
4.5	Endpoints da API	28
4.5.1	GET /historic/things	28
4.5.2	GET /historic/events/{thing_id}/paths	29
4.5.3	GET /historic/events	29
4.5.4	GET /historic/events/{thing_id}	29
4.5.5	POST /historic/events	30
4.5.6	DELETE /historic/events/{thing_id}	30
4.5.7	GET /historic/events/projection/{thing_id}	30
4.5.8	GET /historic/events/time-buckets/{thing_id}	30
4.6	Autenticação	31
4.7	Implantação	31
4.7.1	Kubernetes (Helm)	31
4.7.2	Variáveis de Ambiente para Implantação	32
5	HISTORICAL-WRITER	32
5.1	Visão Geral	32
5.2	Requisitos	32
5.3	Configuração	32
5.3.1	Kafka	33
5.3.2	TimescaleDB	33
5.3.3	Geral	33
5.3.4	Exemplo de .env	33
5.4	Execução Local	34
5.5	Execução com Docker	34
5.5.1	Desenvolvimento	34
5.5.2	Produção (sem dependências de desenvolvimento)	34
5.6	Execução em Kubernetes (Helm)	34
5.6.1	Instalação	34
5.7	Formato de Entrada Esperado	35
5.8	Esquema de Base de Dados	35
5.9	Monitorização e Resolução de Problemas	36
5.9.1	Logs	36

6	LEVELAMS LOADER	36
6.1	Visão Geral	36
6.2	Pré-requisitos	36
6.2.1	Instalação	36
6.3	Guia de Utilização	37
6.3.1	Configuração	38
6.4	Versões Atuais	42
6.5	Execução do Serviço	43
6.6	CSV de Coordenadas de Instrumento	43
6.7	Resolução de Problemas	44
7	GANTRY SERVICE	44
7.1	Visão Geral	44
7.2	Pré-requisitos	44
7.3	Instalação	45
7.4	Guia de Utilização	45
7.5	Configuração	47
7.6	Informações de Versão	51
7.7	Resolução de Problemas	51
8	SENSOR DATA CONVERSION SERVICE	52
8.1	Visão Geral	52
8.2	Pré-requisitos	52
8.3	Instalação	53
8.4	Guia de Utilização	53
8.5	Configuração	55
8.6	Versões Atuais	57
8.7	Resolução de Problemas	57
9	PLATAFORMA DE VISUALIZAÇÃO DE DADOS	58
9.1	Visão Geral	58
9.2	Início de Sessão	58
9.3	Navegação no Globo	59
9.3.1	Modo RTS	59
9.3.2	Modo FreeFly	59
9.3.3	Mudança de Modos de Câmara	60
9.4	Compreender a Interface	60
9.4.1	Barra de Ferramentas	60
9.4.2	Painel de Outline da Cena	60
9.4.3	Marcadores de Entidade	61
9.5	Interação com Entidades	61
9.5.1	Pairar e Selecionar	61
9.5.2	Janela de Inspeção	61
9.5.3	Separador de Associações	62
9.5.4	Separador de Modelos	62
9.6	Painel de Outline da Cena	62

9.7	Colocação de Entidades	62
9.8	Atualizações em Tempo Real	63
10	DATA BRIDGE	63
10.1	Visão Geral	63
10.2	Pré-requisitos	63
10.3	Configuração	64
10.3.1	Objeto Hono	64
10.3.2	Array de Dispositivos	64
10.3.3	Estratégias	65
10.4	Nota sobre geotiles	67
10.5	Guia de Deployment	67
10.5.1	Instanciação Direta	68
10.5.2	Dockerfile	68
10.5.3	Chart Helm	68
11	METEO POPULATOR	69
11.1	Visão Geral	69
11.2	Pré-requisitos	69
11.3	Configuração	69
11.3.1	Objeto Populator	70
11.3.2	Objeto Logging	70
11.3.3	Objeto Station	70
11.3.4	Objeto Filter	70
11.3.5	Objeto OIDC	71
11.4	Nota sobre geotiles	72
11.5	Guia de Deployment	72
11.5.1	Instanciação Direta	72
11.5.2	Dockerfile	72
11.5.3	Chart Helm	73
12	NAMESPACE REMOVER	73
12.1	Visão Geral	73
12.2	Pré-requisitos	73
12.3	Configuração	73
12.3.1	Objeto OIDC	74
12.3.2	Exemplo de config.toml	74
12.4	Argumentos de CLI	74
12.5	Fluxo de Execução	75
12.6	Utilização	75
12.6.1	Utilização Básica	75
12.7	Configuração de Variáveis de Ambiente	75
12.8	Notas de Deployment	76
12.9	Visão Geral	76
12.10	Pré-requisitos	76
12.11	Configuração	77

12.11.1	API Object	77
12.11.2	AUTH Object	78
12.11.3	S3 Object	78
12.11.4	MESH Object	78
12.11.5	Substituição por Variáveis de Ambiente	79
12.12	Como Funciona	79
12.12.1	Autenticação	79
12.12.2	Obtenção da Malha	79
12.12.3	Verificação de Cache	80
12.12.4	Obtenção de Dados	80
12.12.5	Cálculo de Cores	80
12.12.6	Interpolação	81
12.12.7	Exportação	81
12.13	Formato de Saída	81
12.14	Guia de Implantação	81
12.14.1	Instanciação Direta	81
12.14.2	Helm Chart	82
13	SIMULAÇÃO DA FAIXA RESERVADA A5	82
13.1	Visão Geral	82
13.2	Aceder à aplicação	83
13.3	Configuração da simulação	83
13.3.1	Percentagem de Veículos	83
13.3.2	Preço da Portagem (€)	83
13.3.3	Fator de Escala do Tráfego	83
13.4	Execução da Simulação	83
13.5	Interpretação dos resultados	84
14	SIMULAÇÃO DE TRÁFEGO VCI	84
14.1	Visão Geral	84
14.2	Estrutura da Simulação	84
14.3	Execução	85
14.4	Resultados	85
14.5	Atualização do modelo	86
15	MODELOS ML PARA PREVISÃO DO FLUXO DE TRÁFEGO	86
15.1	Visão Geral	86
15.2	Execução	86
15.3	Monitorização	86
15.4	Resultados	87
15.5	Modelos Treinados	87
16	MODELO AGÊNTICO	87
16.1	Visão Geral	87
16.2	Execução	87
16.3	Fluxo de Execução	88
16.4	Resultados e Análise	88

17	DT4MOB PLATFORM	88
17.1	Introdução	88
17.1.1	Visão geral	88
17.1.2	Implementação de gémeos digitais	89
17.2	Instalação	89
17.2.1	Pré-requisitos	89
17.2.2	Instalação do release	90
17.2.3	Abertura da plataforma ao exterior	90
17.2.6	Configuração de backups	92
17.2.7	Configuração do bucket S3	93
17.2.8	Outras configurações	94
17.3	Operação	94
17.3.1	Gestão de utilizadores	94
17.3.6	Ditto	101

LISTA DE FIGURAS

Figura 1 - Interação entre os diferentes componentes	15
Figura 2 - Fluxo interno da camada de visualização	15
Figura 3 - Helm dependency graph.	88
Figura 4 - Digital twins components architecture.	89
Figura 5 - Consola de administração do Keycloak	95
Figura 6 - Seleção de realms no Keycloak	96
Figura 7 - Listagem de utilizadores do realm	96
Figura 8 - User details page.....	97
Figura 9 - Setting the user's password	97
Figura 10 - Atribuição de cargos	98
Figura 11 - Seleção de cargos da plataforma	99
Figura 12 - Listagem de clientes no realm	100
Figura 13 - Listagem de cargos do cliente	101
Figura 14 - Painel de políticas	102

LISTA DE TABELAS

Tabela 1 - Configurações Gerais	16
Tabela 2 - Ligação Ditto	17
Tabela 3 - Autenticação (OIDC).....	17
Tabela 4 - Valores de Referência Helm	19
Tabela 5 - Resolução de Problemas.....	20
Tabela 6 - Configurações Comuns	21
Tabela 7 - Ligação ao Ditto	22
Tabela 8 - Autenticação (Keycloak / OIDC)	22
Tabela 9 - S3 / SeaweedFS	23
Tabela 10 - Fontes de Dados Raster	23
Tabela 11 - Recursos do Chart	24
Tabela 12 - Ficheiros de Resultados	25
Tabela 13 - Configurações Gerais	26
Tabela 14 - Autenticação (Keycloak OIDC)	26
Tabela 15 - TimescaleDB	26
Tabela 16 - Endpoints da API	28
Tabela 17 - Parâmetros de /historic/things	29
Tabela 18 - Parâmetros de /historic/events/{thing_id}/paths	29
Tabela 19 - Parâmetros de /historic/events	29
Tabela 20 - Parâmetros de /historic/events/{thing_id}	30
Tabela 21 - Parâmetros de DELETE	30
Tabela 22 - Parâmetros de Projeção	30
Tabela 23 - Parâmetros de Time-Buckets	31
Tabela 24 - Recursos do Chart	32
Tabela 25 - Configuração do Kafka	33
Tabela 26 - Configuração do TimescaleDB	33
Tabela 27 - Configurações Gerais	33

Tabela 28 - Esquema da Base de Dados	35
Tabela 29 - Requisitos do LevelAMS Loader	36
Tabela 30 - Definições Gerais do LevelAMS Loader	38
Tabela 31 - Definições do componente Loader	39
Tabela 32 - Configuração das credenciais do Loader	39
Tabela 33 - Configuração das definições para Eclipse Hono	40
Tabela 34 - Configuração para a API Ditto e criação de Things.....	40
Tabela 35 - Configuração do componente Dashboard	41
Tabela 36 - Configuração do componente Historic.....	41
Tabela 37 - Definições de OIDC	41
Tabela 38 - Referência de Configuração de Limites.....	42
Tabela 39 - Tipos de limite.....	42
Tabela 40 - Estados de medição	42
Tabela 41 - Dependências e versões atuais.....	43
Tabela 42 - Problemas comuns	44
Tabela 43 - Formatos de sensor suportados pelo Gantry Service	46
Tabela 44 - Formatos de Ditto Envelope Suportados pelo Gantry Service	46
Tabela 45 - Configurações globais	47
Tabela 46 - Definições de Identidade Ditto	47
Tabela 47 – Configuração dos conversores Ativos	47
Tabela 48 - Configuração dos formatadores ativos	47
Tabela 49 - Opções do Formatador Ditto Thing	48
Tabela 50 - Carregamento de Metadados de Portagens.....	48
Tabela 51 – Configuração do Sender de Saída.....	48
Tabela 52 - Configuração do Sender MQTT ([sender].backend = "mqtt")	49
Tabela 53 - Configuração do Configuração do Sender HTTP ([sender].backend = "http")	49
Tabela 54 - Consumidores de entrada	49
Tabela 55 - Definições de Mensagem do Consumer.....	50
Tabela 56 - Informações de Versão.....	51

Tabela 57 - Pré-requisitos do Sensor Data Conversion Service	52
Tabela 58 - Definições Globais	55
Tabela 59 - Consumidor de entrada	55
Tabela 60 - Consumidor MQTT ([consumer].backend = "mqtt")	55
Tabela 61 - Consumidor Webhook ([consumer].backend = "webhook")	56
Tabela 62 - Sender de Saída	56
Tabela 63 - Plano de Referência OutSight	56
Tabela 64 - Plano de Referência AToBe	56
Tabela 65 - Versões Atuais	57
Tabela 66 - Problemas Comuns	58
Tabela 67 - Campos de Início de Sessão	59
Tabela 68 - Controlos do Modo RTS	59
Tabela 69 - Controlos do Modo FreeFly	60
Tabela 70 - Pré-requisitos do Data Bridge	63
Tabela 71 - Estrutura de Configuração	64
Tabela 72 - Campos do Objeto Hono	64
Tabela 73 - Campos do Array de Dispositivos	65
Tabela 74 - Estrutura de Estratégia	65
Tabela 75 - Definição da estratégia meteorológica	66
Tabela 76 - Definição da estratégia de avisos meteorológicos	66
Tabela 77 - Definição da estratégia de tráfego	67
Tabela 78 - Definição da estratégia GeoJSON	67
Tabela 79 - Pré-requisitos do Meteo Populator	69
Tabela 80 - Estrutura da configuração do Meteo Populator	70
Tabela 81 - Campos do Objeto Populator	70
Tabela 82 - Campos do Objeto Logging	70
Tabela 83 - Campos do Objeto Station	70
Tabela 84 - Campos do Objeto Filter	71
Tabela 85 - Campos do Objeto OIDC	72

Tabela 86 - Pré-requisitos do Namespace Remover	73
Tabela 87 - Configuração do Namespace Remover	74
Tabela 88 - Campos do Objeto OIDC	74
Tabela 89 - Argumentos de CLI do Namespace Remover	75
Tabela 90 - Variáveis de ambiente para configuração do Namespace Remover	76
Tabela 91 - Pré-requisitos do Texture Creations	77
Tabela 92 - Prioridades de Configuração	77
Tabela 93 - Configurações do Objeto API	78
Tabela 94 - Configurações do Objeto AUTH	78
Tabela 95 - Configurações do Objeto S3	78
Tabela 96 - Configurações do Objeto MESH	79
Tabela 97 - Substituição por Variáveis de Ambiente	79
Tabela 98 - Cores da Textura Alarmista	81
Tabela 99 - Cores da Textura de Deslocamento	81

1 INTRODUÇÃO

Este é o manual de utilizador do projeto DT4MOB. Descreve como configurar, dar deploy e utilizar cada um dos serviços e componentes da plataforma de digital twin do DT4MOB, incluindo os trabalhadores de ingestão, serviços de API, ferramentas de visualização e componentes de simulação

O manual está organizado por serviço, apresentando para cada um os pré-requisitos necessários, as opções de configuração (variáveis de ambiente ou ficheiros de configuração), os métodos de implantação (Docker, Python direto ou Kubernetes/Helm) e instruções de utilização e resolução de problemas.

Destina-se a operadores, administradores de sistema e utilizadores técnicos responsáveis pela operação e manutenção da plataforma DT4MOB.

1.1 Visão Geral e Arquitetura do projeto

Para facilitar a compreensão global do sistema antes de se detalharem os serviços individuais, esta secção apresenta três diagramas de arquitetura em diferentes níveis de abstração: uma visão macro em camadas funcionais (Figura 1), e o fluxo interno da camada de visualização (Figura 2). Estes diagramas devem ser lidos como um todo, isto é a Figura 1 fornece o contexto conceptual e a Figura 2 aprofunda o comportamento da plataforma de visualização em Unreal Engine.

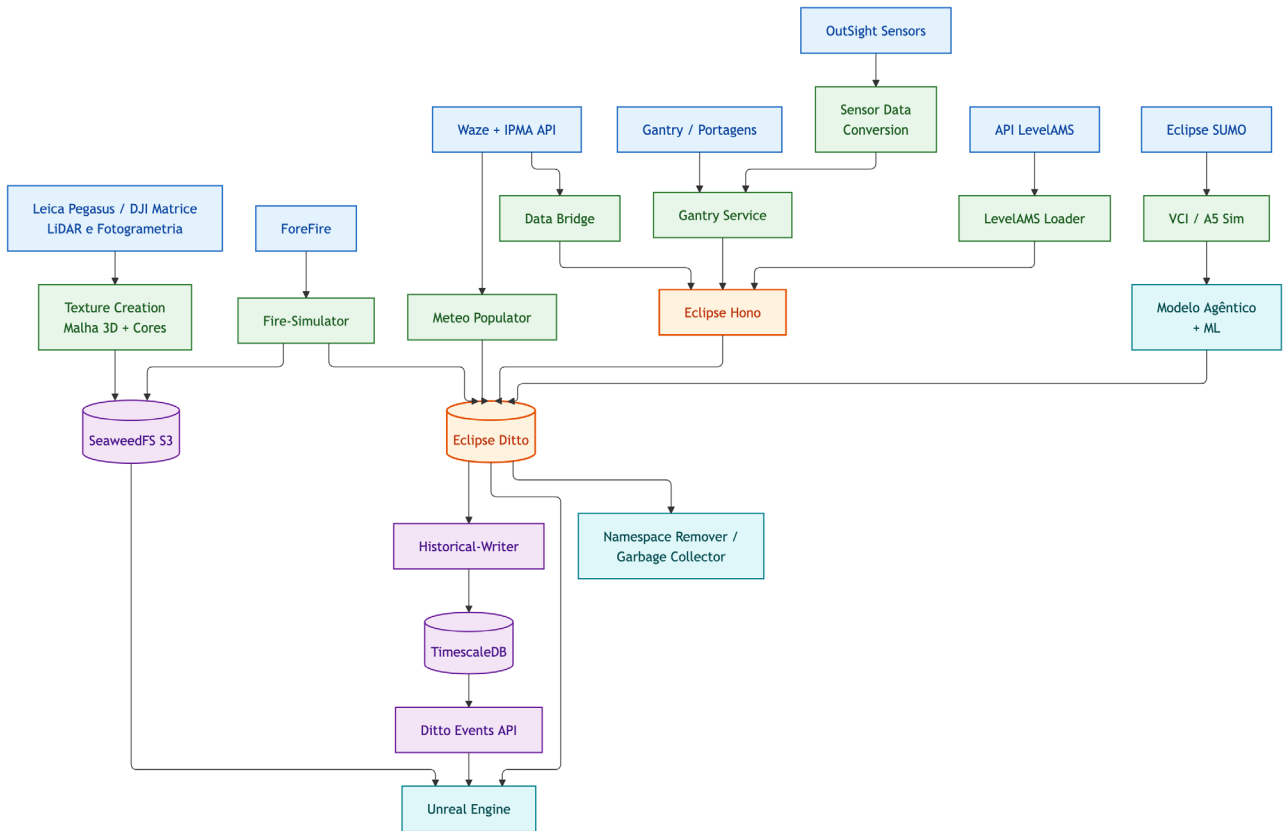


Figura 1 - Interação entre os diferentes componentes

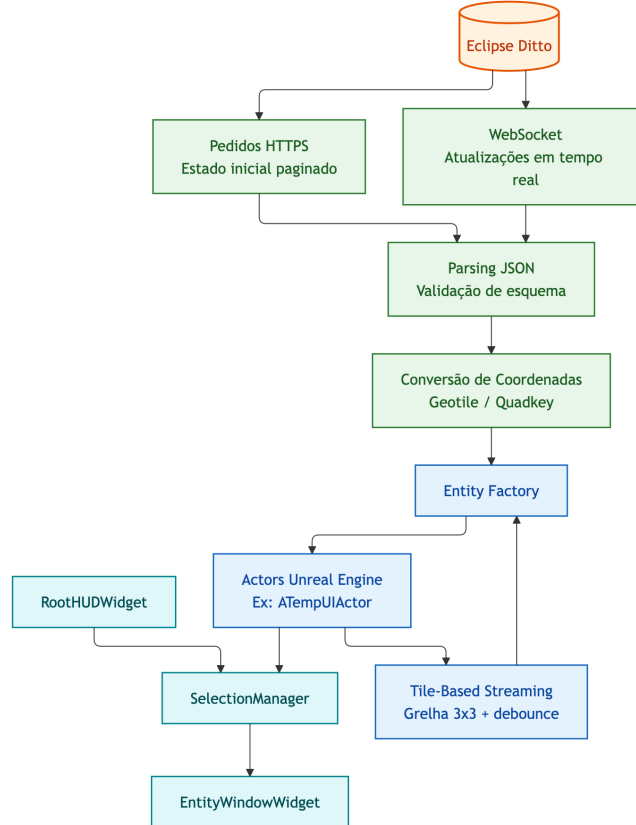


Figura 2 - Fluxo interno da camada de visualização

2 GARBAGE COLLECTOR

2.1 Visão Geral

dt4mob-garbage-collector é um job batch que limpa periodicamente gêmeos digitais expirados do Eclipse Ditto. Para tal, este liga-se ao Ditto, pesquisa Things cujo timestamp `attributes/expiry_ts` está no passado e elimina-os.

Foi desenhado para ser executado como um **Kubernetes CronJob** (por omissão: a cada 5 minutos), mas também pode ser executado localmente com Docker ou Python.

2.2 Pré-requisitos

- Uma instância do **Eclipse Ditto** (com Thing Search ativado)
- Um **servidor de autenticação OIDC** (ex.: Keycloak) com um cliente capaz de emitir tokens via fluxo *password grant*
- Para execução local: Python 3.13 + [uv](#) ou Docker
- Para Kubernetes: um cluster Kubernetes + Helm 3

2.3 Configuração

Toda a configuração é feita através de variáveis de ambiente. As chaves aninhadas usam `__` como delimitador (ex.: `DITTO__API_URL`).

2.3.1 Geral

Variável	Padrão	Descrição
LOG_LEVEL	INFO	Nível de log: CRITICAL, ERROR, WARNING, INFO, DEBUG

Tabela 1 - Configurações Gerais

2.3.2 Ligação Ditto

Variável	Padrão	Descrição
DITTO__API_URL	<code>http://localhost:8080</code>	URL base da API Ditto
DITTO__BASE_API_PATH	<code>/api/2</code>	Prefixo do caminho da API REST
DITTO__BASE_WS_PATH	<code>/ws/2</code>	Prefixo do caminho WebSocket
DITTO__SEARCH_SIZE	200	Tamanho da página para resultados de pesquisa

Tabela 2 - Ligação Ditto

2.3.3 Autenticação (OIDC)

Variável	Padrão D	Descrição
AUTH__TOKEN_ENDPOINT	https://localhost:8443/auth/realms/dt4mob/protocol/openid-connect/token	Endpoint de token OIDC
AUTH__USERNAME	—	Nome de utilizador (mutuamente exclusivo com AUTH__USERNAME_FILE)
AUTH__USERNAME_FILE	—	Caminho para o ficheiro com o nome de utilizador
AUTH__PASSWORD	—	Palavra-passe (mutuamente exclusiva com AUTH__PASSWORD_FILE)
AUTH__PASSWORD_FILE	—	Caminho para o ficheiro com a palavra-passe
AUTH__CLIENT_ID	ditto	ID do cliente OIDC
AUTH__CA_CERT_FILE	—	Caminho para o ficheiro de certificado CA para TLS do servidor OIDC

Tabela 3 - Autenticação (OIDC)

É obrigatório definir um de AUTH__USERNAME ou AUTH__USERNAME_FILE (o mesmo para a palavra-passe). São mutuamente exclusivos.

2.4 Implantação

2.4.1 Opção 1: Local com Docker

```
# Build
docker build -t dt4mob-garbage-collector:latest .
```

```
# Executar com ficheiro .env
docker run --env-file .env dt4mob-garbage-collector:latest
```

2.4.2 Opção 2: Local com Python + uv

```
# Instalar dependências
uv sync --locked

# Executar
uv run main.py
```

2.4.3 Opção 3: Kubernetes via Helm

O projeto inclui um Helm chart em `charts/dt4mob-garbage-collector/`.

```
# Instalar (a partir do diretório charts)
helm install garbage-collector ./charts/dt4mob-garbage-collector --values values.yaml
```

Referência de valores Helm:

Valor	Padrão	Descrição
<code>image.repository</code>	<code>atnog-harbor.av.it.pt/dt4mob/garbage-collector</code>	Imagem do contentor
<code>image.tag</code>	<code>latest</code>	Tag da imagem
<code>image.imagePullPolicy</code>	<code>Always</code>	Política de pull
<code>config.schedule</code>	<code>* / 5 * * * *</code>	Agendamento Cron
<code>config.logLevel</code>	<code>INFO</code>	Nível de log
<code>config.ditto.apiUrl</code>	<code>ditto-gateway:8080</code>	URL da API Ditto
<code>config.ditto.baseApiPath</code>	<code>/api/2</code>	Caminho REST do Ditto
<code>config.ditto.baseWsPath</code>	<code>/ws/2</code>	Caminho WebSocket do Ditto
<code>config.auth.tokenEndpoint</code>	<code>`https://dt4mob-keycloak:8443/auth/realms/dt4mob/protocol/openid-connect/token`</code>	Endpoint de token OIDC

Valor	Padrão	Descrição
<code>config.auth.existingSecret</code>	""	Nome de um Kubernetes Secret existente (se vazio, o chart cria um)
<code>config.auth.caSecret</code>	""	Nome do Secret que contém o certificado CA (chave: ca.crt)

Tabela 4 - Valores de Referência Helm

Credenciais no Kubernetes:

Se `config.auth.existingSecret` estiver vazio e uma palavra-passe for fornecida, o chart cria um Secret com o nome de utilizador e palavra-passe. Também pode pré-criar um Secret com as chaves `username` e `password` e referenciá-lo via `config.auth.existingSecret`.

Durante o provisionamento da infraestrutura, este serviço é automaticamente implementado (*deployed*), uma vez que faz parte das dependências dos Helm Charts da infraestrutura. Este componente é essencial para o correto funcionamento da mesma.

2.5 Como Funciona

- O script abre uma ligação **WebSocket** ao Ditto e autentica-se com um token JWT Bearer.
- Inicia uma tarefa em background que **renova o token** antes de expirar, enviando o novo token para o Ditto via mensagem de controlo.
- Consulta a REST search API do Ditto por Things onde `lt(attributes/expiry_ts, '<current_time')'` — i.e., Things cujo timestamp de expiração está no passado.
- Para cada Thing expirada, constrói um envelope de comando delete do Protocolo Ditto.
- Envia cada comando delete através da ligação **WebSocket** e aguarda uma resposta de sucesso (status 200 ou 204).
- Após todas as eliminações serem confirmadas, o script **termina**.
- O **CronJob** (ou outro scheduler) desencadeia a próxima execução de acordo com o schedule configurado.

2.6 Monitorização e Logs

Docker:

```
docker logs <container_id>
```

Kubernetes:

```
kubectl -n {namespace} get cj,garbage-collector
kubectl -n {namespace} get jobs -l app.kubernetes.io/name=garbage-collector
```

```
kubectl -n {namespace} logs job/<job_name>
```

O output dos logs inclui informação sobre: - Número de Things expiradas encontradas - Cada comando delete enviado e o seu resultado - Eventos de renovação de token - Ciclo de vida da ligação.

2.7 Resolução de Problemas

Problema	Causa Provável	Verificação
O script não consegue ligar-se	URL do Ditto incorreto ou inacessível	Verificar DITTO__API_URL e conectividade de rede
Autenticação falha	Credenciais OIDC incorretas ou expiradas	Verificar AUTH__USERNAME/AUTH__PASSWORD e AUTH__TOKEN_ENDPOINT
Nenhuma Thing eliminada	Sem Things expiradas, ou atributo expiry_ts em falta nas Things	Verificar Things no Ditto e o caminho do atributo
Erros de renovação de token	Endpoint OIDC inacessível ou problema de TLS	Verificar AUTH__CA_CERT_FILE se utilizar CA personalizada
Instalação Helm falha	Valores obrigatórios em falta	Confirmar que username/password ou existingSecret foram fornecidos

Tabela 5 - Resolução de Problemas

3 FIRE-SIMULATOR

3.1 Visão Geral

O dt4mob-fire-simulator é um serviço de ingestão de simulações de incêndios florestais. Escuta eventos de ignição de incêndio do Eclipse Ditto (uma plataforma de gémeos digitais), gera um cone de propagação do vento, executa opcionalmente uma simulação de incêndio florestal ForeFire e carrega os resultados (perímetros GeoJSON, modelos 3D GLB) para o SeaweedFS (armazenamento compatível com S3). O gémeo digital do incidente de incêndio no Ditto é atualizado ao longo de todo o ciclo de vida.

3.1.1 Máquina de Estados

```
NEW_IGNITION → SIMULATING → SIMULATED (se o cone intersectar a zona de risco)
                                     → FAILED (se a simulação falhar)
                                     → NO_RISK (se o cone não intersectar a zona de risc
```

o)

3.2 Pré-requisitos

- **Docker + Docker Compose** (para deployment local),

OU - Cluster Kubernetes com: - **Operador SeaweedFS** instalado (para CRDs de políticas/identidades S3) - **Eclipse Ditto** implementado e acessível - **Keycloak** implementado e acessível (autenticação Ditto) - **Helm 3**.

3.3 Configuração

Toda a configuração é feita através de variáveis de ambiente (.env para Docker Compose) ou values.yaml (para Helm).

3.3.1 Configurações Comuns

Variável / key value	Padrão	Descrição
LOG_LEVEL	INFO	Nível de verbosidade do log
SIM_BASE_DIR	/data/simulations	Diretório de saída das simulações
RISKY_AREAS_PATH	/data/raw/brisa_zones.geojson	GeoJSON das zonas de risco da autoestrada Brisa
FOREFIRE_BIN	/usr/local/bin/forefire	Caminho do binário ForeFire

Tabela 6 - Configurações Comuns

3.3.2 Ligação Ditto

Variável / key value	Padrão D	Descrição
DITTO__API_URL / service.config.ditto.apiUrl	http://dt4mob-ditto-gateway:8080 U D	RL base da API REST do Ditto
DITTO__BASE_API_PATH	/api/2 C D	aminho da versão da API Ditto
DITTO__WS_PATH	/ws/2 C D	Caminho WebSocket do Ditto

Tabela 7 - Ligação ao Ditto

3.3.3 Autenticação (Keycloak / OIDC)

Variável / key value	Padrão	Descrição
AUTH__TOKEN_ENDPOINT / service.config.auth.url	URL do token do realm Keycloak	Endpoint de token OAuth2
AUTH__CLIENT_ID	ditto	ID do cliente OIDC
AUTH__USERNAME	—	Nome de utilizador Keycloak
AUTH__PASSWORD	—	Palavra-passe Keycloak
AUTH__CA_CERT_FILE	—	Ficheiro de certificado CA TLS para Keycloak (K8s: definido via auth.caSecret)

Tabela 8 - Autenticação (Keycloak / OIDC)

As credenciais também podem ser carregadas a partir de ficheiros (AUTH__USERNAME_FILE / AUTH__PASSWORD_FILE) — usado no Helm chart onde são montadas a partir de um Secret.

3.3.4 S3 / SeaweedFS

Variável / key value	Padrão	Descrição
S3__URL_INTERNAL / service.config.s3.UrlInternal	http://seaweed-s3:8333	Endpoint S3 interno (dentro do cluster)
S3__URL_EXTERNAL / service.config.s3.UrlExternal	http://localhost:8333	Endpoint S3 externo (para URLs armazenados no Ditto)
S3__BUCKET / seaweedfs.bucketName	dt4mob-public	Bucket S3 para resultados das simulações
S3__ACCESS_KEY	—	Chave de acesso S3 (gerada automaticamente no K8s via operador SeaweedFS)
S3__SECRET_KEY	—	Chave secreta S3

Tabela 9 - S3 / SeaweedFS

3.3.5 Fontes de Dados Raster (Preparação Única)

Variável / key value	Padrão	Descrição
COS_URL / rasterJob.fileUrls. cosUrl	https://geo2.dgterritorio.gov.pt/cosc /COSc2025.zip	URL do raster de ocupação do solo COSc
DEM_URL / rasterJob.fileUrls. demUrl	http://gis.ciimar.up.pt/lidar- dgt/MDT-10m-PT.tif	URL do raster DTM LiDAR

Tabela 10 - Fontes de Dados Raster

3.4 Preparação de Dados (Única)

O sistema requer três conjuntos de dados raster para executar simulações:

- **Modelo Digital de Elevação (DEM)** — DTM LiDAR, reprojectado para EPSG:4326
- **Mapa de Tipo de Combustível** — Ocupação do solo COSc, reamostrado para corresponder à grelha do DEM, códigos não combustíveis remapeados para 0
- **GeoJSON de Zonas de Risco** — Zonas de risco das autoestradas Brisa descarregadas do SeaweedFS

Estes são produzidos pela imagem `dt4mob-raster-prepare-fire-simulator`. Este processo:

- Descarrega os rasters DEM e COSc a partir dos URLs configurados
- Constrói um VRT e reprojecta para EPSG:4326
- Extrai e ajusta o raster COSc para corresponder ao DEM
- Remapeia códigos de ocupação do solo não combustíveis para 0
- Descarrega o GeoJSON das zonas de risco do S3
- Escreve um ficheiro sentinela `.done` em `/data/processed/.done`

Em **Docker Compose**, execute o job manualmente (descomente `prepare_rasters_job` no `docker-compose.yml`, depois `docker-compose up prepare_rasters_job`).

Em **Kubernetes**, o Prepare Job é executado automaticamente como um Helm hook no `helm install / helm upgrade`.

3.5 Implantação

3.5.1 Opção A: Docker Compose

```
cd app_deploy
cp .env.example .env          # edite com os seus valores
docker-compose up -d
```

O serviço `ingestion` inicia, executa `python main.py`, conecta-se ao Ditto via WebSocket e aguarda eventos de ignição.

3.5.2 Opção B: Helm (Kubernetes)

Primeiro, verifique o ficheiro `values.yaml` para configurar todas as variáveis.

```
helm upgrade --install dt4mob-fire chart/ \
  --namespace dt4mob \
  --set service.config.secret.userName=<ditto-user> \
  --set service.config.secret.password=<ditto-pass>
```

O chart cria:

Recurso K	ind P	Propósito
dt4mob-fire-simulator-raster-processor	Job (Helm hook)	Prepara os rasters antes de a API iniciar
dt4mob-fire-simulator-api	Deployment	Executa o worker de ingestão (com contentor init wait-for-rasters)
dt4mob-fire-simulator	Service	Expõe o worker na porta 8080
dt4mob-fire-simulator-pvc	PVC (20Gi)	Volume partilhado para dados raster + simulações
dt4mob-fire-simulator-secret	Secret (condicional)	Credenciais Ditto
dt4mob-fire-simulator-s3-secret	Secret	Chaves S3 (preenchidas automaticamente pelo operador SeaweedFS)
dt4mob-fire-simulator-seaweed-user	S3Identity	Utilizador S3 SeaweedFS
dt4mob-fire-simulator-s3-policy / *-job-s3-policy	S3Policy S3PolicyBinding	+ Controlo de acesso S3 para os buckets da aplicação e do job

Tabela 11 - Recursos do Chart

O contentor principal aguarda que o job de preparação de rasters termine através de um contentor init que verifica `/data/processed/.done`.

3.5.3 Utilização (Fluxo de Eventos)

O sistema é reativo — não faz nada até que um evento Ditto chegue.

- **Crie um incidente de incêndio no Ditto** com `attributes/state = "new_ignition"` e `attributes/ignition` contendo `lat` e `lon`.
- O worker de ingestão recebe o evento via WebSocket e inicia o processamento.
- **Fase do cone:** Um cone de propagação do vento com 4 horizontes é gerado usando dados da estação meteorológica mais próxima e carregado no S3. O gémeo digital Ditto é atualizado para o estado `SIMULATING`.

- **Verificação de risco:** Se o cone intersectar uma zona de risco das autoestradas Brisa, a simulação ForeFire completa é executada (horizonte de 2 horas, passos de 15 minutos). Caso contrário, o estado passa a NO_RISK.
- **Fase de simulação:** O ForeFire é executado e produz 8 ficheiros GeoJSON de perímetros, que são convertidos para um modelo 3D GLB e carregados no S3. O gémeo digital é atualizado com URLs dos perímetros e estado SIMULATED (ou FAILED em caso de erro).
- **Limpeza:** Os ficheiros de simulação locais são eliminados.

3.5.4 Monitorização

- Verifique o gémeo digital Ditto (fire:<uuid>) para transições de estado e URLs de ficheiros carregados em features.
- Os registos são impressos no stdout (defina LOG_LEVEL para DEBUG para saída verbosa).
- Os buckets S3 podem ser navegados na interface do SeaweedFS ou através de qualquer cliente S3.

3.6 Resultados

Após o processamento, cada incidente de incêndio produz:

Ficheiro	Formato	Descrição
cone_horizon.geojson	GeoJSON FeatureCollection	4 horizontes do cone (30, 60, 90, 120 min)
fire_cone.glb	GLB (glTF Binary)	Modelo 3D do cone
step_0015.geojson – step_0120.geojson	GeoJSON Feature	8 perímetros de incêndio (a cada 15 min)
fire_simulation.glb	GLB (glTF Binary)	Modelo 3D de todos os perímetros combinados

Tabela 12 - Ficheiros de Resultados

Estes são carregados no bucket S3 no caminho do thing_id. Os URLs correspondentes são armazenados nas features do gémeo digital Ditto.

4 DITTO EVENTS API

4.1 Visão Geral

A Ditto Events API (dt4mob-historical-api) é um serviço FastAPI que expõe uma interface REST para consultar, inserir e eliminar eventos históricos produzidos pelo Eclipse Ditto. Os eventos são armazenados numa hipertabela TimescaleDB, permitindo consultas eficientes de séries temporais, projeções JSONPath e agregações temporais.

Este serviço faz parte da plataforma **DT4Mob** e é implantado em conjunto com o dt4mob-historical-writer, que ingere eventos do Kafka para a mesma base de dados.

4.2 Configuração

Toda a configuração é feita através de variáveis de ambiente. Um ficheiro .env também é suportado

no diretório de trabalho.

4.2.1 Geral

Variável	Obrigatório	Padrão	Descrição
LOG_LEVEL	Não	INFO	Nível de log (CRITICAL, ERROR, WARNING, INFO, DEBUG)

Tabela 13 - Configurações Gerais

4.2.2 Autenticação (Keycloak OIDC)

Variável	Obrigatório	Padrão D	Descrição
AUTH__CLIENT_ID	Sim	—	ID do cliente Keycloak
AUTH__AUDIENCE	Não	account	Audiência OIDC
AUTH__SERVER_URI	Sim	—	URI do realm Keycloak (ex.: https://keycloak.example.com/realms/myrealm)
AUTH__ISSUER	Sim	—	URL do emissor OIDC (pode ser uma lista: ["ur11", "ur12"])
AUTH__SIGNATURE_CACHE_TTL	Não	3600	TTL da cache de assinatura JWT em segundos
AUTH__READ_ROLE	Não	["historical-read"]	Roles Keycloak necessárias para endpoints de leitura
AUTH__WRITE_ROLE	Não	["historical-write"]	Roles Keycloak necessárias para endpoints de escrita

Tabela 14 - Autenticação (Keycloak OIDC)

4.2.3 TimescaleDB

Variável	Obrigatório	Padrão	Descrição
TIMESCALE__CONNECTION	Sim	—	URI de conexão (ex.: postgresql://user:pass@host:5432/db)
TIMESCALE__DATABASE_TIMEZONE	Não	Europe/Lisbon	Fuso horário da base de dados

Tabela 15 - TimescaleDB

4.2.4 Delimitador de Configuração Aninhada

O separador __ (duplo underscore) é usado pelo pydantic-settings para aninhar variáveis. Por exemplo: AUTH__CLIENT_ID mapeia para settings.auth.client_id.

4.3 Execução Local

4.3.1 Pré-requisitos

- Python ≥ 3.13
- [uv](#) gestor de pacotes
- Uma instância TimescaleDB em execução

4.3.2 Configuração

```
# Clonar o repositório e entrar no diretório do serviço
cd services/dt4mob-historical-api

# Instalar dependências
uv sync

# Criar um ficheiro .env com as variáveis necessárias (ver §2)
# Exemplo:
# LOG_LEVEL=DEBUG
# AUTH__CLIENT_ID=my-client
# AUTH__SERVER_URI=https://keycloak.example.com/realms/myrealm
# AUTH__ISSUER=https://keycloak.example.com/realms/myrealm
# TIMESCALE__CONNECTION=postgresql://user:pass@localhost:5432/historical
```

4.3.3 Execução

```
# Desenvolvimento (auto-reload)
uv run fastapi dev app/main.py --port 8080

# Produção
uv run fastapi run app/main.py --port 8080 --host 0.0.0.0
```

A API estará disponível em `http://localhost:8080/historic/`. O Swagger UI é servido no path raiz.

4.4 Execução com Docker

4.4.1 Construção

```
docker build -t data-lake-api -f dockerfile .
```

4.4.2 Execução

```
docker run -p 8080:8080 \
  -e LOG_LEVEL=INFO \
  -e AUTH__CLIENT_ID=my-client \
  -e AUTH__SERVER_URI=https://keycloak.example.com/realms/myrealm \
  -e AUTH__ISSUER=https://keycloak.example.com/realms/myrealm \
  -e TIMESCALE__CONNECTION=postgresql://user:pass@host:5432/db \
  data-lake-api
```

4.5 Endpoints da API

Todos os endpoints são montados sob o path base `/historic`. Endpoints de leitura requerem a role `historical-read`; endpoints de escrita requerem `historical-write`.

Method	Path	Descrição	Role
GET	<code>/historic/</code>	Swagger UI (HTML personalizado)	None
GET	<code>/historic/things</code>	Listar IDs de Things distintas	<code>historical-read</code>
GET	<code>/historic/events/{thing_id}/paths</code>	Listar pares (path, action) distintos	<code>historical-read</code>
GET	<code>/historic/events</code>	Consultar eventos com filtros	<code>historical-read</code>
GET	<code>/historic/events/{thing_id}</code>	Consultar eventos por Thing ID	<code>historical-read</code>
POST	<code>/historic/events</code>	Inserir eventos	<code>historical-write</code>
DELETE	<code>/historic/events/{thing_id}</code>	Eliminar eventos num intervalo temporal	<code>historical-write</code>
GET	<code>/historic/events/projection/{thing_id}</code>	Extrair campos via JSONPath	<code>historical-read</code>
GET	<code>/historic/events/time-buckets/{thing_id}</code>	Agregações temporais	<code>historical-read</code>

Tabela 16 - Endpoints da API

Os endpoints estão disponíveis numa interface Swagger UI para consulta:

1 Swagger UI para a API de Dados Históricos.

4.5.1 GET `/historic/things`

Devolve todos os `thing_id` distintos, opcionalmente filtrados por prefixo.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
<code>thing_id_prefix</code>	<code>str</code>	Não	Filtrar Things cujo ID começa com este prefixo

Tabela 17 - Parâmetros de /historic/things

4.5.2 GET /historic/events/{thing_id}/paths

Devolve pares (path, action) distintos para uma Thing num intervalo temporal.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
since	datetime (ISO 8601)	Sim	Início do intervalo
until	datetime (ISO 8601)	Não	Fim do intervalo (padrão: agora)

Tabela 18 - Parâmetros de /historic/events/{thing_id}/paths

4.5.3 GET /historic/events

Consulta eventos em todas as Things com múltiplos filtros.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
since	datetime	Sim	Início do intervalo
until	datetime	Não	Fim do intervalo (padrão: agora)
thing_id_prefix	str	Não	Filtrar por prefixo do Thing ID
path_prefix	str	Não	Filtrar por prefixo de path
action	enum	Não	Filtrar por ação (created, modified, deleted, merged)

Tabela 19 - Parâmetros de /historic/events

4.5.4 GET /historic/events/{thing_id}

Consulta eventos para uma Thing específica.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
since	datetime	Sim	Início do intervalo
until	datetime	Não	Fim do intervalo (padrão: agora)
path_prefix	str	Não	Filtrar por prefixo de path
action	enum	Não	Filtrar por ação

Tabela 20 - Parâmetros de /historic/events/{thing_id}

4.5.5 POST /historic/events

Insere um ou mais eventos diretamente (alternativa à ingestão por Kafka).

Corpo do pedido: Objeto JSON com um array events:

```
{
  "events": [
    {
      "time": "2024-01-01T00:00:00Z",
      "thing_id": "org.eclipse.ditto:my-thing",
      "action": "modified",
      "path": "/features/temperature/properties/value",
      "revision": 1,
      "value": 25.5
    }
  ]
}
```

4.5.6 DELETE /historic/events/{thing_id}

Apaga eventos para uma Thing num intervalo temporal.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
since	datetime	Sim	Início do intervalo
until	datetime	Sim	Fim do intervalo

Tabela 21 - Parâmetros de DELETE

4.5.7 GET /historic/events/projection/{thing_id}

Extraí campos específicos do value JSON dos eventos usando uma expressão JSONPath.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
json_path	str	Sim	Expressão JSONPath (ex.: \$.temperature)
since	datetime	Sim	Início do intervalo
until	datetime	Não	Fim do intervalo (padrão: agora)
path_prefix	str	Não	Filtrar por prefixo de path

Tabela 22 - Parâmetros de Projeção

4.5.8 GET /historic/events/time-buckets/{thing_id}

Agrupa eventos em buckets temporais e executa agregações sobre um valor JSONPath.

Parâmetros de query:

Parâmetro	Tipo	Obrigatório	Descrição
json_path	str	Sim	JSONPath para o valor a agregar
since	datetime	Sim	Início do intervalo
until	datetime	Sim	Fim do intervalo
bucket_minutes	int	Não	Largura do bucket em minutos (omitir ou <=0 para um único bucket)
agg_type	enum	Não	Função de agregação: count, sum, avg, min, max (padrão: count)
path_prefix	str	Não	Prefixos de path separados por vírgula para segmentar resultados

Tabela 23 - Parâmetros de Time-Buckets

Nota: Quando path_prefix contém múltiplos valores separados por vírgula, o endpoint usa get_events_custom_time_buckets_with_path que agrupa resultados também por path.

4.6 Autenticação

A API usa **Keycloak OIDC** (OpenID Connect) para autenticação. O middleware fastapi-oidc valida tokens JWT em todos os pedidos.

- Endpoints de leitura requerem a role historical-read (configurável via AUTH__READ_ROLE).
- Endpoints de escrita requerem a role historical-write (configurável via AUTH__WRITE_ROLE).

Para aceder à API, inclua um token Bearer no cabeçalho Authorization:

Authorization: Bearer <seu-token-jwt-do-keycloak>

4.7 Implantação

4.7.1 Kubernetes (Helm)

Os serviços API e writer são implantados em conjunto através de um único Helm chart localizado em charts/dt4mob-historical/.

Recursos principais criados pelo chart:

Recurso K	ind D	escrição
api-deployment.yaml	Deployment	Executa o contentor da API na porta 8080
api-service.yaml	Service	Expõe a API na porta 8080

Recurso K	ind D	escrição
writer-deployment.yaml	Deployment	Executa o writer (consumidor Kafka)
writer-config.yaml	ConfigMap	Configuração Kafka e TimescaleDB para o writer
kafka-topic.yaml	KafkaTopic (Strimzi)	Topic para eventos Ditto
timescale-cluster.yaml	Cluster (CNPG)	Cluster PostgreSQL + TimescaleDB

Tabela 24 - Recursos do Chart

Imagens dos contentores (publicadas em atnog-harbor.av.it.pt/dt4mob/): - API: data-lake-api - Writer: data-gateway.

A API é exposta através de um ingress que encaminha /historic para o serviço da API.

4.7.2 Variáveis de Ambiente para Implantação

No Helm chart, as variáveis de autenticação são definidas diretamente no deployment, enquanto TIMESCALE__CONNECTION é injetada através de um Kubernetes Secret por segurança.

Como o chart está presente como dependência do chart principal da infraestrutura, este será configurado automaticamente quando o Helm chart da infraestrutura for instalado.

5 HISTORICAL-WRITER

5.1 Visão Geral

O historical-writer é um serviço que consome mensagens de um tópico Kafka (seguindo o protocolo Eclipse Ditto) e persiste eventos numa hipertabela TimescaleDB. Faz parte da extensão de dados históricos da plataforma DT4Mob.

5.2 Requisitos

- **Python** >= 3.12
- **uv** (gestor de pacotes: <https://docs.astral.sh/uv/>)
- **Kafka** (com certificados SSL)
- **TimescaleDB** (PostgreSQL com extensão Timescale)

5.3 Configuração

Toda a configuração é feita através de variáveis de ambiente ou de um ficheiro .env no diretório raiz do serviço.

5.3.1 Kafka

Variável	Obrigatório	Descrição
KAFKA_BOOTSTRAP_SERVERS	Sim	Endereço do broker Kafka (ex.: kafka-cluster-kafka-bootstrap:9092)
KAFKA_SECURITY_PROTOCOL	Sim	Protocolo de segurança (ex.: SSL)
KAFKA_SSL_CA_LOCATION	Sim	Caminho para o ficheiro de certificado CA
KAFKA_SSL_CERTIFICATE_LOCATION	Sim	Caminho para o certificado do cliente
KAFKA_SSL_KEY_LOCATION	Sim	Caminho para a chave privada do cliente
KAFKA_CONSUMER_GROUP	Sim	ID do grupo de consumidores
KAFKA_TOPIC	Sim	Tópico a consumir
KAFKA_AUTO_OFFSET_RESET	Sim	Estratégia de reinício de offset (latest, earliest)

Tabela 25 - Configuração do Kafka

5.3.2 TimescaleDB

Variável	Obrigatório	Padrão	Descrição
TIMESCALE_CONNECTION	Sim	—	String de conexão (ex.: postgresql://user:pass@host:5432/db)
TIMESCALE_DATABASE_TIMEZONE	Não	Europe/Lisbon	Fuso horário para colunas de timestamp

Tabela 26 - Configuração do TimescaleDB

5.3.3 Geral

Variável	Obrigatório	Padrão	Descrição
LOG_LEVEL	Não	INFO	Nível de log (DEBUG, INFO, WARNING, ERROR, CRITICAL)

Tabela 27 - Configurações Gerais

5.3.4 Exemplo de .env

```
KAFKA_BOOTSTRAP_SERVERS=kafka-cluster-kafka-bootstrap:9092
KAFKA_SECURITY_PROTOCOL=SSL
KAFKA_SSL_CA_LOCATION=/certs/ca.crt
KAFKA_SSL_CERTIFICATE_LOCATION=/certs/user.crt
KAFKA_SSL_KEY_LOCATION=/certs/user.key
```

```
KAFKA_CONSUMER_GROUP=historical-writer-group
KAFKA_TOPIC=exports-test-tenant
KAFKA_AUTO_OFFSET_RESET=latest
TIMESCALE_CONNECTION=postgres://user:password@timescale-host:5432/historical
TIMESCALE_DATABASE_TIMEZONE=Europe/Lisbon
LOG_LEVEL=INFO
```

5.4 Execução Local

```
# Instalar dependências
uv sync --locked

# Executar
uv run main.py
```

5.5 Execução com Docker

5.5.1 Desenvolvimento

```
docker build -f dockerfile -t historical-writer:dev .
docker run --env-file .env historical-writer:dev
```

5.5.2 Produção (sem dependências de desenvolvimento)

```
docker build -f dockerfile.prod -t historical-writer:prod .
docker run --env-file .env historical-writer:prod
```

O contentor espera que os certificados SSL estejam montados em `/certs/` (ou nos caminhos definidos nas variáveis de ambiente).

5.6 Execução em Kubernetes (Helm)

A implantação no Kubernetes é gerida através do Helm chart `dt4mob-historical`, localizado na infraestrutura do projeto em `charts/dt4mob-historical/`. Este chart não faz parte do código fonte deste serviço — reside neste repositório na pasta `charts`.

O Helm chart provê:

- **Writer Deployment** — executa este serviço
- **Writer ConfigMap** — mapeia valores do chart para variáveis de ambiente
- **TimescaleDB Cluster** — cluster CloudNativePG com extensão TimescaleDB
- **KafkaTopic** — tópico Strimzi (opcional)
- **API Deployment + Service** — API de leitura complementar (código separado)

5.6.1 Instalação

```
helm upgrade --install dt4mob-historical charts/dt4mob-historical -f my-values.yaml
```

Este serviço já é instalado automaticamente junto com a infraestrutura principal, pois esta depende

do mesmo para servir o histórico da infraestrutura.

5.7 Formato de Entrada Esperado

O serviço consome mensagens no formato do **protocolo Eclipse Ditto**, publicadas no tópico Kafka configurado. Exemplo de mensagem:

```
{
  "topic": "org.dt4mob:vehicle-123/things/live/modified",
  "headers": {
    "dt4mob-historic-timestamp-override": "2025-01-15T10:30:00Z"
  },
  "path": "/features/speed/properties/value",
  "value": 42.5,
  "revision": 3,
  "timestamp": "2025-01-15T10:30:05Z"
}
```

- topic deve seguir o padrão {namespace}:{name}/things/{channel}/{action}
- dt4mob-historic-timestamp-override (cabeçalho opcional) substitui o timestamp guardado na base de dados caso este esteja presente nos headers do evento proveniente do Ditto.

5.8 Esquema de Base de Dados

O serviço cria a tabela dittoevent com as seguintes colunas:

Coluna	Tipo	Notas
time	timestampz	Chave primária (1.ª parte)
index	integer	Chave primária (2.ª parte, auto-incremento)
thing_id	text	Indexado
action	enum	modified, created, deleted, merged
path	text	Caminho JSON do Ditto
revision	integer	Pode ser nulo
value	jsonb	Pode ser nulo

Tabela 28 - Esquema da Base de Dados

Configuração da hipertabela TimescaleDB:

- **Intervalo de chunk:** 1 dia
- **Compressão:** ativada
- **Segmento de compressão:** thing_id
- **Ordenação de compressão:** time DESC
- **Intervalo de compressão:** dados com mais de 7 dias são comprimidos

5.9 Monitorização e Resolução de Problemas

5.9.1 Logs

Defina `LOG_LEVEL=DEBUG` para obter saída detalhada mostrando cada mensagem consumida e escrita na base de dados.

6 LEVELAMS LOADER

6.1 Visão Geral

O LevelAMS Loader é um serviço de sincronização de dados para sistemas de monitorização geotécnica. Extrai dados da API disponibilizada pela LevelAMS e carrega-os no Eclipse Ditto via Eclipse Hono, permitindo capacidades de gémeos digitais para ativos geotécnicos e os seus instrumentos.

Problema que resolve: A monitorização geotécnica gera dados contínuos de instrumentos como inclinómetros e células de carga. Este serviço mantém a plataforma de gémeos digitais (Eclipse Ditto) sincronizada com as configurações mais recentes dos ativos e valores de medição, enquanto mantém opcionalmente registos históricos através de uma API de histórico dedicada.

Caso de uso principal: Este script foi desenvolvido para ser executado como um CronJob Kubernetes para sincronização periódica automatizada. Suporta também execução manual para carregamentos iniciais de dados ou recuperação de dados históricos.

6.2 Pré-requisitos

Antes de utilizar o LevelAMS Loader, assegure-se de que tem:

<i>Requirement</i>	<i>Version/Details</i>
<i>Python</i>	<i>3.13 ou superior</i>
<i>uv package manager</i>	<i>Instalar uv</i>
<i>Acesso à API LevelAMS</i>	<i>Credenciais válidas ou token JWT</i>
<i>Eclipse Hono</i>	<i>Instância com um broker MQTT</i>
<i>Eclipse Ditto</i>	<i>Instância com acesso API</i>
<i>Instrument coordinates</i>	<i>Ficheiro CSV com coordenadas ETRS89 TM06</i>

Tabela 29 - Requisitos do LevelAMS Loader

Requisitos opcionais:

- API de Histórico com autenticação OIDC/Keycloak (para recuperação de dados históricos)
- Dashboards Grafana (para URLs de visualização de instrumentos)

6.2.1 Instalação

1. Clonar o repositório

```
git clone https://github.com/ATNoG/dt4mob-level-ams-loader.git
cd dt4mob-level-ams-loader
```

2. Instalar dependências

```
uv sync
```

Isto instala todos os packages Python necessários num ambiente isolado.

3. Criar ficheiro de configuração.

```
cp config.example.toml config.toml
```

4. Configurar o serviço

Edite `config.toml` com as suas definições específicas de ambiente. Consulte a secção [Configuração](#) para todas as opções disponíveis.

6.3 Guia de Utilização

Execução Padrão (Carregar um ativo geotécnico).

O loader requer um ID de ativo geotécnico para sincronizar. Este é o modo de operação principal.

```
uv run main.py --geo-asset-id <ASSET_ID>
```

Exemplo:

```
uv run main.py --geo-asset-id "geo-asset-123"
```

O que acontece:

1. Obtém detalhes do ativo e lista de instrumentos da API LevelAMS
2. Cria o ativo geotécnico como uma Thing Ditto
3. Cada instrumento como uma Thing Ditto com coordenadas e URLs de dashboard
4. Obtém valores de medição mais recentes para cada parâmetro e calcula estado de monitorização
5. Publica todos os dados no Eclipse Ditto através do adaptador MQTT do Eclipse Hono

Filtrar Instrumentos Específicos.

Para carregar apenas instrumentos específicos de um ativo:

```
uv run main.py --geo-asset-id <ASSET_ID> --instrument-ids <ID1> [<ID2> ...]
```

Exemplo:

```
uv run main.py --geo-asset-id "geo-asset-123" --instrument-ids "A25.1" "A25.2" "I1C"
```

Nota: `--instrument-ids` requer que `--geo-asset-id` esteja definido.

Modo de Carregamento de Histórico.

O carregamento de histórico é reservado para operações manuais de recuperação de dados (ex., após perda de dados). Este modo obtém todos os valores históricos de parâmetros que an API disponibiliza e envia-os para an API de histórico.

Para ativar o carregamento de histórico:

1. Defina `history.enabled = true` em `config.toml`
2. Configure a secção `[oidc]` com credenciais válidas

3. Execute o loader:

```
uv run main.py --geo-asset-id <ASSET_ID>
```

Importante: O carregamento de histórico deve ser executado apenas quando explicitamente necessário. Em operações normais, o carregamento de histórico deve permanecer desativado.

Execução Docker.

```
#### Construir a imagem
docker build -t level-ams-loader .
```

```
#### Executar o container
docker run level-ams-loader --geo-asset-id <ASSET_ID>
```

6.3.1 Configuração

A configuração é carregada de config.toml na raiz do projeto. As variáveis de ambiente podem substituir qualquer definição usando o prefixo LEVEL_AMS_LOADER_ com __ como delimitador aninhado.

Formato da variável de ambiente:

```
LEVEL_AMS_LOADER_<SEÇÃO>__<CAMPO>=valor
```

Exemplo:

```
export LEVEL_AMS_LOADER_LOG_LEVEL=DEBUG
export LEVEL_AMS_LOADER_LOADER__JWT=my-jwt-token
export LEVEL_AMS_LOADER_HONO__HOST=mqtt.example.com
```

Definições Gerais

Definition	Type	Default	Description
log_level	String	"INFO"	Verbosidade do log da aplicação. Opções: TRACE, DEBUG, INFO, SUCCESS, WARNING, ERROR, CRITICAL

Tabela 30 - Definições Gerais do LevelAMS Loader

Definições do Loader [loader].

Definition	Type	Default	Description
base_url	URL	"http://localhost"	URL base da API LevelAMS
jwt	String	null	Token JWT para API autenticação. Forneça este ou credenciais
http_timeout	Float	30.0	Timeout de pedido HTTP em segundos. Defina como null para disable
start_page_idx	Integer	1	Índice de page inicial para paginação (deve ser >= 1)

Definition	Type	Default	Description
instrument_coords_file	Path	"instrument_coords.csv"	Caminho para ficheiro CSV com coordenadas dos instrumentos
parameters_chunk_size	Integer	10	Número de parâmetros processados em paralelo por instrumento
instruments_chunk_size	Integer	10	Número de instrumentos processados em paralelo

Tabela 31 - Definições do componente Loader

Credenciais do Loader [loader.credentials].

Utilizado quando jwt não é fornecido. O loader irá executar um pedido de login para obter um token JWT.

Definition	Type	Default	Description
username	String	-	Username da API
password	String	-	Password da API

Tabela 32 - Configuração das credenciais do Loader

Definições do Eclipse Hono [hono].

Ligação MQTT ao Eclipse Hono para publicar mensagens de protocolo Ditto.

Definition	Type	Default	Description
host	String	"127.0.0.1"	Hostname do broker MQTT
port	Integer	null	Port do broker MQTT. Defina como null para padrão
device	String	null	Identificador do dispositivo MQTT. Combinado com tenant para formar name de user (device@tenant)
tenant	String	null	Identificador do tenant MQTT. Combinado com device para formar name de user (device@tenant)
password	String	null	Password de autenticação MQTT opcional
cafile	Path	null	Caminho para o ficheiro de certificado CA para TLS
certfile	Path	null	Caminho para o ficheiro de certificado do cliente para TLS
keyfile	Path	null	Caminho para o ficheiro de chave privada do cliente para TLS

Definition	Type	Default	Description
topic	String	"telemetry"	Topic MQTT para publicar mensagens

Tabela 33 - Configuração das definições para Eclipse Hono

Definições do Eclipse Ditto [ditto].

Configuração para an API Ditto e criação de Things.

Definition	Type	Default	Description
base_url	URL	"http://localhost/api/2"	URL base da REST API Ditto
namespace	String	"namespace"	Prefixo namespace para IDs de Things (ex., "namespace:subject:assetId")
default_policy	String	"dt4mob:default"	ID de política atribuído a todos os Things criados
subject	String	"test"	Prefixo subject para nomes de Things
username	String	null	Username de authentication da API Ditto optional
password	String	null	Password de authentication da API Ditto optional
cafile	Path	null	File de certificado CA optional para TLS da API Ditto
cadata	String	null	Dados de certificado CA em PEM codificados optional

Tabela 34 - Configuração para a API Ditto e criação de Things

Definições do Dashboard [dashboard].

Configuração para URLs de dashboard dos instrumentos. Estas URLs são armazenados como atributos em cada instrumento nos Things Ditto.

Definition	Type	Default	Description
inclinometer_base_url	URL	"http://127.0.0.1"	URL base para dashboard de inclinómetro
inclinometer_query_param	String	"var-instrument_id"	Parâmetro de queries para ID do instrumento no URL do inclinómetro
load_cell_base_url	URL	"http://127.0.0.1"	URL base para dashboard de célula de carga
load_cell_query_param	String	"var-instrument_id"	Parâmetro de queries para ID do instrumento no URL da célula de

Definition	Type	Default	Description
			carga

Tabela 35 - Configuração do componente Dashboard

Definições de Carregamento de Histórico [history].

Controla o comportamento de carregamento de dados históricos. Reservado para operações manuais de recuperação de dados.

Definition	Type	Default	Description
enabled	Boolean	false	Ativar carregamento de dados históricos
base_url	URL	"http://127.0.0.1/historic"	URL base da API de Histórico

Tabela 36 - Configuração do componente Historic

Definições OIDC [oidc].

Necessário quando history.enabled = true. Configura autenticação com an API de histórico via OIDC/Keycloak.

Definition	Type	Default	Description
username	String	"admin"	Username OIDC/Keycloak
password	String	"admin"	Password OIDC/Keycloak
client_id	String	"client"	ID de client OAuth2
realm	String	"realm"	Name do realm Keycloak
base_url	String	"http://localhost"	URL base do fornecedor OIDC

Tabela 37 - Definições de OIDC

Limites dos Instrumentos [instrument_thresholds].

Limites de alerta/alarme opcionais por instrumento. Indexados por matrícula do instrumento.

Prioridade: Limites específicos do instrumento substituem limites por tipo de instrumento.

```
[instrument_thresholds."A25.1"]
parameter_types = ["INCL01"]
alert = { threshold_type = "GT", value = 5.0 }
alarm = { threshold_type = "GT", value = 10.0 }
```

Limites por Tipo de Instrumento [instrument_type_thresholds].

Limites de alerta/alarme opcionais por tipo de instrumento. Aplicados quando não existe limite específico do instrumento.

Tipos de instrumento suportados:

- InstrumentoTipoInstrumento_Inclinometro
- InstrumentoTipoInstrumento_CelulaCarga

```
[instrument_type_thresholds."InstrumentoTipoInstrumento_Inclinometro"]
parameter_types = ["INCL01", "INCL02"]
```

```

alert = { threshold_type = "GT", value = 5.0 }
alarm = { threshold_type = "GT", value = 10.0 }

```

Referência de Configuração de Limites

Field	Type	Description
parameter_types	Conjunto de String	Corresponde a keys de parâmetro without números finais, pois são considerados do mesmo type de parâmetro
threshold_type	String	Type de comparação. Ver abaixo
value	Float	Value limite

Tabela 38 - Referência de Configuração de Limites

Tipos de limite:

Type	Behavior	Example
GT	Maior que	Dispara quando <code>value > threshold</code>
LT	Menor que	Dispara quando <code>value < threshold</code>
DELTA	Delta Absoluto	Dispara quando <code>abs(value - earliestKnownValue) > threshold</code>

Tabela 39 - Tipos de limite

Estados de medição:

State	Description
OK	Value dentro da faixa normal
ALERT	Value excede limite de alerta
ALARM	Value excede limite de alarme

Tabela 40 - Estados de medição

6.4 Versões Atuais

Dependency	Version	Purpose
Python	3.13	Ambiente de execução
uv	Último	Gestor de packages
amqtt	==0.11.3	Client MQTT assíncrono
httpx	==0.28.1	Client HTTP assíncrono
loguru	==0.7.3	Log de logs
pydantic	==2.12.5	Validação de dados
pydantic-settings	==2.12.0	Gestão de definições

Dependency	Version	Purpose
<i>pydantic-argparse</i>	<i>==0.10.0</i>	<i>Análise de argumentos CLI</i>
<i>pyproj</i>	<i>==3.7.2</i>	<i>Transformações de coordenadas</i>

Tabela 41 - Dependências e versões atuais

6.5 Execução do Serviço

Execução Local

```
#### Assegurar que config.toml está configurado
uv run main.py --geo-asset-id <ASSET_ID>
```

Execução Docker

```
#### Construir
docker build -t level-ams-loader .

#### Executar
docker run level-ams-loader --geo-asset-id <ASSET_ID>

#### Executar com ficheiros e variáveis de ambiente corretos
docker run \
  -v ./config.toml:/app/config.toml:ro \
  -e LEVEL_AMS_LOADER_LOG_LEVEL="DEBUG" \
  level-ams-loader --geo-asset-id <ASSET_ID>
```

Alternativamente pode utilizar a imagem docker existente assim:

```
docker run atnog-harbor.av.it.pt/dt4mob/level-ams-loader --geo-asset-id <ASSET_ID>
```

Deploy Helm

Existe um Helm Chart no [repositório dt4mob-platform](#), que é recomendado para deploy do script como um CronJob Kubernetes para sincronizar periodicamente a plataforma de gémeos digitais com os valores LevelAMS mais recentes.

6.6 CSV de Coordenadas de Instrumento

O loader requer um ficheiro CSV contendo coordenadas dos instrumentos no formato ETRS89 TM06 (EPSG:3763). As coordenadas são automaticamente convertidas para WGS84 (latitude/longitude) para atributos dos Things Ditto.

Formato CSV:

```
matricula,x,y,z
A1,-77300.1234,-85100.6543,82.1234
A2,-77300.2345,-85100.5432,83.2345
I1,-77330.5678,-85130.2109,72.5678
I2,-77340.6789,-85140.1098,71.6789
```

Consulte `instrument_coords.example.csv` na raiz do projeto para um exemplo.

6.7 Resolução de Problemas

Problemas Comuns

Problem	Solution
Provide at least one of jwt or credentials	<i>Defina jwt ou [loader.credentials] em config.toml</i>
Login failed with status code: 401	<i>Verifique credenciais e base_url em [loader]`</i>
Connection refused ao MQTT	<i>Verifique host/port e caminhos de certificados TLS em [hono], bem como device, tenant e password`</i>
Failed getting details for id	<i>Verifique se o ID do ativo geotécnico existe na API LevelAMS</i>
Respostas estranhas da API	<i>Reduza o número de chunks de instrumentos e parâmetros, pois a API pode estar a limitar a taxa de requests</i>

Tabela 42 - Problemas comuns

Registo de Logs.

Aumente a verbosidade definindo `log_level = "DEBUG"` ou `log_level = "TRACE"` em `config.toml`.

7 GANTRY SERVICE

7.1 Visão Geral

O Gantry Service é uma aplicação de ponte que recolhe dados de deteção de veículos de sensores rodoviários (câmaras, LiDARs) e os encaminha para o Eclipse Ditto via Eclipse Hono, permitindo representações de gémeos digitais do tráfego real de portagens. Consome formatos de mensagem específicos dos sensores, normaliza-os, envolve-os no protocolo Eclipse Ditto e entrega-os ao backend IoT.

7.2 Pré-requisitos

Antes de utilizar o Gantry Service, assegure-se de que tem:

- Python 3.13+ instalado (verificar com `python --version`)
- Gestor de packages uv instalado (Visite <https://docs.astral.sh/uv/getting-started/installation/> para instalar)
- Docker e Docker Compose (se executar via containers)
- Um broker MQTT acessível (ex., Mosquitto) se usar consumers MQTT ou sender
- Instância Eclipse Hono acessível se enviar via MQTT
- Um ficheiro de certificado CA (ca.crt) para ligações TLS ao seu broker Hono/MQTT, colocado no diretório pai do projeto`

7.3 Instalação

Opção A: Executar Diretamente com uv.

```
#### Clonar o repositório
git clone https://github.com/ATNoG/dt4mob-gantry-service.git
cd dt4mob-gantry-service

#### Instalar dependências
uv sync

#### Copiar e editar a configuração de exemplo
cp config.example.toml config.toml
#### Editar config.toml com as suas definições (ver secção Configuração)

#### Copiar dados de portagens (se usar carregador de portagens)
cp tolls.example.json tolls.json

#### Executar o serviço
uv run main.py
```

Opção B: Executar com Docker Compose.

```
#### Clonar o repositório
git clone https://github.com/ATNoG/dt4mob-gantry-service.git
cd dt4mob-gantry-service

#### Copiar e editar a configuração de exemplo
cp config.example.toml config.toml
#### Editar config.toml com as suas definições (ver secção Configuração)

#### Copiar dados de portagens (se usar carregador de portagens)
cp tolls.example.json tolls.json

#### Editar o ficheiro compose.yml conforme necessário e executar para construir e iniciar
docker compose up --build
```

A configuração disponível do Docker Compose automaticamente:

- Monta config.toml como volume read-only em /app/config.toml
- Monta ca.crt como volume read-only em /app/ca.crt (para TLS)
- Monta tolls.json como volume read-only em /app/tolls.json
- Expõe a porta 8000 (usada pelo consumer webhook)
- Mapeia host.docker.internal para o gateway do anfitrião para acesso local ao broker

7.4 Guia de Utilização

Iniciar o Serviço

Uma vez iniciado, o serviço irá:

1. Carregar metadados de portagens de tolls.json (se configurado) e registar Things de portagens no Ditto
2. Ligar a todos os consumers ativos (brokers MQTT ou iniciar endpoints webhook)

3. Começar a aceitar mensagens de sensores
4. Converter mensagens recebidas, envolvê-las em envelopes Ditto, e enviá-las ao backend configurado

Pipeline de Processamento.

O serviço processa mensagens através de quatro etapas:

Sensores → Consumer → Conversor de Mensagem → Formatador de Envelope → Sender

1. **Consumer** recebe dados originais de sensores via subscrição MQTT ou HTTP webhook POST
2. **Conversor de Mensagem** normaliza formatos específicos de fornecedor num formato interno padrão
3. **Formatador de Envelope** envolve os dados normalizados em envelopes de protocolo Eclipse Ditto
4. **Sender** entrega os envelopes ao Eclipse Hono via MQTT ou um endpoint HTTP

Formatos de Sensor Suportados

Format	Description	Message Types
a-to-be	Formato de dados ORT Tracking de câmaras e LiDARs	realtime (deteções ao vivo), history (dados históricos de path)
outsight	Dados de sensor OutSight com deteção baseada em alerta	Events de entrada/saída de alerta com coordenadas delimitadoras

Tabela 43 - Formatos de sensor suportados pelo Gantry Service

Formatos de Ditto Envelope Suportados.

Format	Description
toll-feature	Cria um único Ditto Thing por toll com features por sensor. Os dados do veículo são armazenados em <code>/features/{source_name}/properties/{vehicle_id}</code> . Includes limpeza circular (máx. 50 veículos por sensor) e limpeza periódica completa. Também inclui um path especial <code>/features/{source_name}/properties/historic</code> , na qual é colocada a última mensagem histórica recebida.````
ditto-Thing	Cria um Ditto Thing separado por veículo detetado com creation diferida. Cada veículo recebe a sua própria Thing com features State. Inclui gestão automática de expiração.

Tabela 44 - Formatos de Ditto Envelope Suportados pelo Gantry Service

Ciclo de Limpeza.

O serviço envia periodicamente comandos delete para limpar todos os dados de sensor das Ditto Things, prevenindo a acumulação de entradas obsoletas e a ultrapassagem do tamanho máximo do Thing. Isto é executado a cada 150 segundos por padrão (configurável com o `clear_loop_interval_s`).

7.5 Configuração

A configuração é gerida através de `config.toml`. As variáveis de ambiente podem substituir qualquer definição usando o prefixo `GS_` com `__` como delimitador aninhado (ex., `GS_SENDER__HOST=10.0.0.1`).

Definições Globais.

Field	Type	Default	Description
<code>log_level</code>	<i>string</i>	"INFO"	Verbosidade de log. Opções: <i>CRITICAL, ERROR, WARNING, INFO, DEBUG, TRACE</i>
<code>clear_loop_intervals</code>	<i>float positivo</i>	150	Segundos entre varreduras periódicas de limpeza Ditto

Tabela 45 - Configurações globais

[ditto] — Definições de Identidade Ditto.

Field	Type	Default	Description
<code>toll_id</code>	<i>int não-negativo</i>	0	Identificador numérico da toll no Ditto
<code>namespace</code>	<i>string</i>	"default"	Namespace Ditto para o Thing
<code>subject</code>	<i>string</i>	"test"	Subject Ditto usado no thingId para controle de acesso
<code>policy_id</code>	<i>string</i>	"default:policy"	ID de política Ditto para Things criados

Tabela 46 - Definições de Identidade Ditto

[message_converters] — Conversores Ativos.

Field	Type	Default	Description
<code>a-to-be</code>	<i>bool</i>	false	Ativar o conversor de mensagens A-to-Be ORT
<code>outsight</code>	<i>bool</i>	false	Ativar o conversor de mensagens OutSight

Tabela 47 – Configuração dos conversores Ativos

[envelope_formatters] — Formatadores Ativos.

Field	Type	Default	Description
<code>toll-feature</code>	<i>bool</i>	false	Ativar o formatador de envelope toll-feature (Thing único por toll)
<code>ditto-Thing</code>	<i>bool</i>	false	Ativar o formatador de envelope ditto-Thing (Thing separado por veículo)

Tabela 48 - Configuração dos formatadores ativos

[ditto_thing_settings] — Opções do Formatador Ditto Thing.

Field	Type	Default	Description
deferred_creation_offset	int	10	Offset adicionado aos IDs de veículo para creation diferida de Things no formatador ditto-Thing

Tabela 49 - Opções do Formatador Ditto Thing

[toll_loader] — Carregamento de Metadados de Portagens.

Field	Type	Default	Description
backend	string	"disabled"	Backend do carregador de tolls. Opções: "disabled", "json-file"
filepath	path	"tolls.json"	Caminho para o ficheiro JSON de tolls (usado apenas quando backend é "json-file")

Tabela 50 - Carregamento de Metadados de Portagens

Formato JSON de portagens:

```
{
  "tolls": [
    {
      "id": 1,
      "coords": { "latitude": 38.0, "longitude": -9.0 },
      "name": "Ocean Gantry",
      "sensors": ["camera", "lidar"],
      "dashboardUrl": "https://example.com/grafana/gantry"
    }
  ]
}
```

[sender] — Sender de Saída

Field	Type	Default	Description
backend	string	"disabled"	Backend do sender. Opções: "disabled", "mqtt", "http"

Tabela 51 – Configuração do Sender de Saída

Sender MQTT ([sender].backend = "mqtt").

Field	Type	Default	Description
host	string	"localhost"	Hostname do broker MQTT
port	int	1883	Port do broker MQTT
username	string	—	Username MQTT
password	string	—	Password MQTT
tls	bool	true	Ativar TLS para a ligação
cafile	string	—	Caminho para o ficheiro de certificado CA para verification TLS
certfile	string	—	Path para certificado de client para mTLS

Field	Type	Default	Description
keyfile	string	—	Path para key privada de client para mTLS
publish_topic	string	"telemetry"	Topic MQTT para publicar envelopes Ditto

Tabela 52 - Configuração do Sender MQTT ([sender].backend = "mqtt")

Sender HTTP ([sender].backend = "http").

Field	Type	Default	Description
host	string	—	Hostname do endpoint HTTP alvo
port	int	443	Port do endpoint HTTP alvo
username	string	—	Username de autenticação básica (opcional)
password	string	—	Password de autenticação básica (opcional)
tls	bool	false	Ativar TLS para a ligação
cafile	string	—	Caminho para o ficheiro de certificado CA para verification TLS
client_key	string	—	Path para key privada de client para mTLS
client.crt	string	—	Path para certificado de client para mTLS
base_path	string	—	Path HTTP para enviar payloads JSON via POST

Tabela 53 - Configuração do Configuração do Sender HTTP ([sender].backend = "http")

[[consumers]] — Consumers de Entrada.

Cada bloco [[consumers]] define uma fonte de dados de sensor. Pode definir múltiplos consumers.

Field	Type	Default	Description
backend	string	—	Backend do consumer. Opções: "mqtt", "webhook"
enabled	bool	true	Se este consumer está ativo
host	string	—	Hostname do broker MQTT ou de escuta do webhook
port	int	—	Port do broker MQTT ou de escuta do webhook
secure	bool	false	Usar TLS para ligação MQTT
username	string	—	Username MQTT
password	string	—	Password MQTT

Tabela 54 - Consumidores de entrada

Definições de Mensagem do Consumer ([[consumers.message_settings]]).

Cada consumer pode ter múltiplos blocos de definições de mensagem definindo os tópicos que gere.

Field	Type	Default	Description
expected_message_type	string	—	Formato do sensor: "a-to-be" ou "outsight"
topic	string não-vazia	—	Topic MQTT para subscrever, ou segmento de path URL do webhook
source_name	string não-vazia	—	Identificador para esta fonte de sensor (usado nos caminhos de features Ditto)
allowed_ditto_formats	conjunto de strings	[]	Quais formatadores de envelope use. Opções: "toll-feature", "ditto-Thing"
atobe_type	string	—	Sub-tipo A-to-Be: "realtime" ou "history" (apenas quando expected_message_type é "a-to-be")

Tabela 55 - Definições de Mensagem do Consumer

Configuração de Exemplo.

```

log_level = "INFO"

[ditto]
namespace = "tolls"
policy_id = "dt4mob:default"
toll_id = 1

[[consumers]]
backend = "mqtt"
host = "localhost"
username = "sensor_user"
password = "secret"

[[consumers.message_settings]]
topic = "vehicle/realtime-tracking"
source_name = "camera-ORT"
expected_message_type = "a-to-be"
atobe_type = "realtime"
allowed_ditto_formats = ["toll-feature", "ditto-thing"]

[toll_loader]
backend = "json-file"
filepath = "tolls.json"

[message_converters]
a-to-be = true

[envelope_formatters]
toll-feature = true

```

```
[sender]
backend = "mqtt"
host = "hono.example.com"
port = 31918
tls = true
cafile = "../ca.crt"
username = "tolls@hono"
password = "secret"
```

7.6 Informações de Versão

Component	Version
<i>Python</i>	3.13
<i>Gestor de packages</i>	<i>uv (Astral)</i>
<i>amqtt</i>	≥0.11.3
<i>FastAPI</i>	≥0.128.1
<i>httpx</i>	≥0.28.1
<i>loguru</i>	≥0.7.3
<i>Pydantic</i>	≥2.11.7
<i>pydantic-settings</i>	≥2.10.1

Tabela 56 - Informações de Versão

7.7 Resolução de Problemas

Serviço não inicia

- Verifique se `config.toml` existe na raiz do projeto e é TOML válido
- Altere o `log_level` para `DEBUG` ou `TRACE` para saída detalhada
- Assegure-se de que todos os ficheiros de certificado e chave (ex., `ca.crt`, `crt.pem`, `key.pem`) estão presentes no caminho especificado na sua configuração

Mensagens não aparecem no Ditto.

- Confirme que pelo menos um de cada componente está ativo e configurado
- Verifique se os detalhes de ligação do sender MQTT/HTTP (host, porta, credenciais) estão corretos
- Verifique se o caminho do certificado CA está correto e o ficheiro é legível
- Assegure-se de que o carregador de portagens está configurado se o seu formatador de envelope requer metadados de portagens

Consumer webhook não recebe dados.

- Confirme que a porta do consumer webhook (padrão 8000) não está bloqueada por uma firewall
- Verifique se o URL do pedido POST corresponde ao campo `topic` configurado

- Verifique se `expected_message_type` e `source_name` estão definidos nos `message_settings` correspondentes

Dados obsoletos no Ditto.

- O ciclo de limpeza é executado a cada 150 segundos por padrão; ajuste `clear_loop_interval_s` se precisar de limpeza mais rápida
- O formatador toll-feature mantém um buffer circular de 50 IDs de veículo por sensor; entradas mais antigas são eliminadas automaticamente

Ajuda Adicional

- Verifique `config.toml` contra `config.example.toml` para referência de campos
- Ative o registo DEBUG ou TRACE para ver saída de processamento detalhada
- For architecture details, see the DT4Mob Programmer's Manual (Deliverable D2.A).

8 SENSOR DATA CONVERSION SERVICE

8.1 Visão Geral

O Sensor Data Conversion Service é um serviço de ponte que converte dados de deteção de veículos do formato OutSight para o formato ORT Tracking. Consome mensagens no formato OutSight contendo deteções de caixas delimitadoras com coordenadas locais, transforma-as em mensagens ORT Tracking com dimensões devidamente calculadas, coordenadas geográficas e posições locais, e publica a saída convertida num broker MQTT.

Problema que resolve: Os sensores OutSight e os sistemas ORT Tracking utilizam formatos de dados e sistemas de referência de coordenadas diferentes. O OutSight fornece cantos de caixas delimitadoras originais num sistema de coordenadas local, enquanto o ORT Tracking espera dimensões (comprimento, largura, altura), coordenadas locais relativas a uma câmara de referência, e coordenadas geográficas absolutas (latitude/longitude). Este serviço trata automaticamente das transformações de coordenadas e derivações de campos, permitindo uma integração transparente entre os dois sistemas.

Caso de uso principal: Executa-se como um serviço de longa duração num deploy de portagem ou monitorização rodoviária, convertendo continuamente mensagens de deteção de veículos em tempo real e históricas de produtores utilizando o formato OutSight para o formato ORT Tracking.

8.2 Pré-requisitos

Antes de utilizar o Sensor Data Conversion Service, assegure-se de que tem:

<i>Requirement</i>	<i>Version/Details</i>
<i>Python</i>	<i>3.13 ou superior</i>
<i>uv package manager</i>	<i>Instalar uv</i>
<i>Broker MQTT</i>	<i>Instância acessível (ex., Mosquitto) para o sender e opcionalmente o consumer</i>

Tabela 57 - Pré-requisitos do Sensor Data Conversion Service

Requisitos opcionais:

- Docker e Docker Compose (se executar via containers)
- Um sistema de sensores OutSight a publicar mensagens

- Um consumer compatível com ORT Tracking a subscrever os tópicos de saída convertidos

8.3 Instalação

Opção A: Executar Diretamente com uv.

```
#### Clonar o repositório
git clone https://github.com/ATNoG/dt4mob-sensor-data-conversion.git
cd dt4mob-sensor-data-conversion

#### Instalar dependências
uv sync

#### Copiar e editar a configuração de exemplo
cp config.example.toml config.toml
#### Editar config.toml com as suas definições (ver secção Configuração)

#### Executar o serviço
uv run main.py
```

Opção B: Executar com Docker Compose.

```
#### Clonar o repositório
git clone https://github.com/ATNoG/dt4mob-sensor-data-conversion.git
cd dt4mob-sensor-data-conversion

#### Copiar e editar a configuração de exemplo
cp config.example.toml config.toml
#### Editar config.toml com as suas definições (ver secção Configuração)

#### Construir e iniciar
docker compose up --build
```

A configuração do Docker Compose automaticamente:

- Monta config.toml como volume read-only em /app/config.toml
- Expõe a porta 8000 (usada pelo consumer webhook)
- Mapeia host.docker.internal para o gateway do anfitrião para acesso local ao broker

8.4 Guia de Utilização

Iniciar o Serviço

Uma vez iniciado, o serviço irá:

1. Ligar ao consumer configurado (broker MQTT ou endpoint webhook)
2. Começar a ler mensagens OutSight
3. Converter cada mensagem recebida para o formato ORT Tracking (em tempo real e opcionalmente histórico)
4. Publicar as mensagens convertidas no broker MQTT configurado

Pipeline de Processamento.

O serviço processa mensagens através de três etapas:

1. **Consumer** recebe mensagens JSON no formato OutSight através subscrições MQTT ou

HTTP webhook POST

2. **Conversor** transforma dados OutSight no formato ORT Tracking:
3. Deriva **dimensões** do objeto (comprimento, largura, altura) a partir dos cantos da caixa delimitadora
4. Converte **coordenadas locais** via intermediário ENU (East-North-Up) usando os planos de referência configurados
5. Calcula **coordenadas absolutas** (latitude/longitude) a partir da posição ENU
6. Extrai **ID do objeto**, **timestamp**, e **velocidade** diretamente
7. **Sender** publica as mensagens convertidas nos tópicos MQTT configurados

Formato de Entrada OutSight.

As mensagens OutSight contêm detecções de veículos com coordenadas de caixa delimitadora:

```
{
  "start_timestamp": "2024-01-15T10:30:00Z",
  "end_timestamp": "2024-01-15T10:30:01Z",
  "data": {
    "in_alert": {
      "object_id": 42,
      "speed": 85.5,
      "coordinate": [[x0, y0], [x1, y1], [x2, y2], [x3, y3]],
      "height": 2.1
    },
    "out_alert": {
      "object_id": 42,
      "speed": 82.3,
      "coordinate": [[x0, y0], [x1, y1], [x2, y2], [x3, y3]],
      "height": 2.1
    }
  }
}
```

Formato de Saída ORT Tracking.

Mensagens em tempo real convertidas são publicadas no atb_realtime_topic:

```
{
  "objectID": 42,
  "timeOfMeasurement": "2024-01-15T10:30:01Z",
  "speedKmh": 82.3,
  "absoluteCoordinates": { "latitude": 38.7113, "longitude": -9.3092 },
  "dimensions": { "width": 1.85, "length": 4.52, "height": 2.1 },
  "localCoordinates": { "x": 1.23, "y": 4.56 }
}
```

Se um out_alert estiver presente (indicando que o veículo saiu da zona de detecção), uma mensagem de histórico é também publicada no atb_history_topic contendo um caminho com pontos de detecção in e out:

```
{
  "objectID": 42,
  "path": [
    { "timeOfMeasurement": "...", "speedKmh": 85.5, "localCoordinates": {...}
  , "absoluteCoordinates": {...}, "dimensions": {...} },

```

```

    { "timeOfMeasurement": "...", "speedKmh": 82.3, "localCoordinates": {...}
  , "absoluteCoordinates": {...}, "dimensions": {...} }
]
}

```

Parar o Serviço

Prima Ctrl+C ou envie um sinal SIGINT/SIGTERM. O serviço irá registrar uma mensagem de terminação e desconectar-se corretamente.

8.5 Configuração

A configuração é gerida através de `config.toml`. As variáveis de ambiente podem substituir qualquer definição usando o prefixo `SENSOR_DATA_CONVERSION_` com `__` como delimitador aninhado (ex., `SENSOR_DATA_CONVERSION_SENDER__URL=mqtt://broker:1883`).

Prioridade: Variáveis de ambiente substituem valores em `config.toml`.

Definições Globais.

Field	Type	Default	Description
log_level	string	"INFO"	Verbosidade de log. Opções: CRITICAL, ERROR, WARNING, INFO, DEBUG
atb_realtime_topic	string	""	Topic MQTT para publicar mensagens ORT Tracking de tempo real convertidas
atb_history_topic	string	""	Topic MQTT para publicar mensagens ORT Tracking de histórico convertidas

Tabela 58 - Definições Globais

[consumer] – Consumer de Entrada.

O consumer define como o serviço ingere mensagens OutSight. Apenas um backend pode estar ativo de cada vez.

Field	Type	Default	Description
backend	string	"disabled"	Backend do consumer. Opções: "disabled", "mqtt", "webhook"

Tabela 59 - Consumidor de entrada

Consumer MQTT ([consumer].backend = "mqtt").

Field	Type	Default	Description
url	URL	"mqtt://localhost"	URL de ligação ao broker MQTT
topics	list	[]	Topics MQTT para subscrever

Tabela 60 - Consumidor MQTT ([consumer].backend = "mqtt")

Consumer Webhook ([consumer].backend = "webhook").

Field	Type	Default	Description
host	string	"localhost"	Hostname de escuta do webhook

Field	Type	Default	Description
port	int	8000	Port de escuta do webhook
base_path	string	"/"	Path HTTP para receber pedidos POST

Tabela 61 - Consumidor Webhook ([consumer].backend = "webhook")

[sender] – Sender de Saída.

O sender define onde as mensagens convertidas são publicadas.

Field	Type	Default	Description
url	URL	"mqtt://localhost"	URL de ligação ao broker MQTT

Tabela 62 - Sender de Saída

[converter] – Definições do Conversor.

O conversor define os planos de referência utilizados para transformações de coordenadas entre os sistemas de coordenadas OutSight e ORT Tracking.

[converter.outsight_reference_plane] – Plano de Referência OutSight.

Define o sistema de coordenadas geográficas dos dados OutSight recebidos.

Field	Type	Default	Description
origin	[lat, lon]	[0.0, 0.0]	Coordenadas geográficas [latitude, longitude] da origem do plano de referência
point_on_positive_horizontal_axis	[lat, lon]	[0.0, 1.0]	Coordenadas geográficas [latitude, longitude] que definem a direção do eixo horizontal positivo

Tabela 63 - Plano de Referência OutSight

[converter.atobe_reference_plane] – Plano de Referência AToBe.

Define o sistema de coordenadas geográficas alvo para a saída ORT Tracking convertida.

Field	Type	Default	Description
origin	[lat, lon]	[0.0, 0.0]	Coordenadas geográficas [latitude, longitude] da origem do plano de referência
point_on_positive_horizontal_axis	[lat, lon]	[0.0, 1.0]	Coordenadas geográficas [latitude, longitude] que definem a direção do eixo horizontal positivo

Tabela 64 - Plano de Referência AToBe

Substituições por Variáveis de Ambiente.

Todas as definições podem ser substituídas por variáveis de ambiente com o prefixo SENSOR_DATA_CONVERSION_ e __ como delimitador aninhado:

```
export SENSOR_DATA_CONVERSION_LOG_LEVEL="DEBUG"
export SENSOR_DATA_CONVERSION_SENDER__URL="mqtt://broker:1883"
export SENSOR_DATA_CONVERSION_CONSUMER__BACKEND="mqtt"
```

```

export SENSOR_DATA_CONVERSION_CONSUMER__URL="mqtt://sensor-broker:1883"
export SENSOR_DATA_CONVERSION_CONSUMER__TOPICS='["detections"]'
export SENSOR_DATA_CONVERSION_ATB_REALTIME_TOPIC="vehicle/realtime"
export SENSOR_DATA_CONVERSION_ATB_HISTORY_TOPIC="vehicle/history"

```

Configuração de Exemplo.

```

log_level = "INFO"

atb_realtime_topic = "vehicle/realtime"
atb_history_topic = "vehicle/history"

[consumer]
backend = "mqtt"
url = "mqtt://sensor-broker:1883"
topics = ["outsight/detections"]

[sender]
url = "mqtt://hono-broker:1883"

[converter.outsight_reference_plane]
origin = [38.711289, -9.309308]
point_on_positive_horizontal_axis = [38.711341, -9.309187]

[converter.atobe_reference_plane]
origin = [38.711308, -9.309153]
point_on_positive_horizontal_axis = [38.711164, -9.309436]

```

8.6 Versões Atuais

Dependency	Version	Purpose
<i>Python</i>	<i>3.13</i>	<i>Ambiente de execução</i>
<i>uv</i>	<i>última</i>	<i>Gestor de packages</i>
<i>amqtt</i>	<i>==0.11.3</i>	<i>Client MQTT assíncrono</i>
<i>FastAPI</i>	<i>==0.129.0</i>	<i>Server HTTP do consumer webhook</i>
<i>loguru</i>	<i>==0.7.3</i>	<i>Log de logs</i>
<i>numpy</i>	<i>==2.3.4</i>	<i>Operações matriciais para transformações de coordenadas</i>
<i>pydantic</i>	<i>==2.12.3</i>	<i>Validação de dados</i>
<i>pydantic-settings</i>	<i>==2.11.0</i>	<i>Gestão de definições</i>
<i>pymap3d</i>	<i>==3.2.0</i>	<i>Transformações de coordenadas geográficas (ENU/ECEF)</i>

Tabela 65 - Versões Atuais

8.7 Resolução de Problemas

Problemas Comuns

Problem	Solution
O serviço termina imediatamente*	Verifique se config.toml existe e é TOML válido. Altere log_level para DEBUG para saída detalhada.
*Nenhuma mensagem recebida	Verifique se o backend do consumer corresponde à saída do sensor OutSight (MQTT ou webhook). Verifique detalhes de ligação (host, port, URL, topics).
Ligação MQTT Ve recusada*	Verifique a URL do broker nas secções [sender] e [consumer]. Verifique se o broker está a funcionar e acessível.
Webhook não recebe C dados*	Confirme que a porta do webhook (padrão 8000) não está bloqueada por uma firewall. Verifique se o path do request POST corresponde a base_path.
Mensagens convertidas com campos em falta	Assegure-se de que [converter.outsight_reference_plane] e [converter.atobe_reference_plane] estão configurados com coordenadas corretas para o seu deploy.
Erros de validação nos A logs*	Mensagem recebida não corresponde ao formato OutSight esperado. Ative o log DEBUG para ver a mensagem original e detalhes de validação.

Tabela 66 - Problemas Comuns

Registo de Logs.

Aumente a verbosidade definindo log_level = "DEBUG" em config.toml ou via variável de ambiente:

```
export SENSOR_DATA_CONVERSION_LOG_LEVEL="DEBUG"
```

Níveis de log disponíveis: CRITICAL, ERROR, WARNING, INFO, DEBUG.

Ajuda Adicional.

- Verifique config.toml contra config.example.toml para referência de campos
- Ative o registo DEBUG para ver saída de processamento detalhada

9 PLATAFORMA DE VISUALIZAÇÃO DE DADOS

9.1 Visão Geral

A Plataforma de Visualização de Dados é uma aplicação de visualização 3D que apresenta o estado do digital twin do DT4MOB. Construída com Unreal Engine 5.6 e o plugin Cesium for Unreal, recebe dados de entidades do Eclipse Ditto via WebSocket e apresenta-os num globo interativo.

9.2 Início de Sessão

68 Quando a aplicação é iniciada, é apresentado um ecrã de início de sessão antes de qualquer dado ser carregado. Preencha os seguintes campos para se ligar ao backend Ditto:

Campo	Descrição
Host	O endereço do servidor Ditto, sem prefixo de protocolo (e.g. ditto.example.com).
Username	O seu nome de utilizador Ditto.
Password	A sua palavra-passe Ditto.
Use HTTPS	Ative se o servidor exigir uma ligação segura.
Use OAuth	Ative a autenticação via Keycloak. Se desativado, é utilizada autenticação HTTP Basic.
WS Start Message	A mensagem de subscrição é enviada para o Ditto após a ligação WebSocket ser estabelecida. Esta está pré-configurada e normalmente não necessita de ser alterada.

Tabela 67 - Campos de Início de Sessão

Após a submissão das credenciais, a aplicação autentica-se no servidor e começa a carregar entidades para o globo. A cena é povoada progressivamente à medida que os dados das entidades são recebidos.

9.3 Navegação no Globo

A plataforma de visualização disponibiliza dois modos de câmara adequados a diferentes tarefas de inspeção. O modo ativo é exibido na barra de ferramentas e pode ser alternado a qualquer momento usando o botão de alternância de câmara.

9.3.1 Modo RTS

O modo RTS proporciona uma vista aérea do globo, adequada para uma visão geográfica ampla e localização de entidades numa área extensa. A câmara olha diretamente para baixo e mantém o alinhamento com o Norte e Este geográficos à medida que se move.

Ação	Controlo
Pan	W A S D
Rodar vista	Rato
Aumentar / diminuir zoom	Roda do rato
Mover para cima / baixo	Q / E

Tabela 68 - Controlos do Modo RTS

À medida que a câmara aproxima, a plataforma carrega automaticamente as entidades para a área geográfica visível e descarrega as que saem de vista. Afastar o zoom suficientemente carregará todas as entidades simultaneamente, independentemente da posição.

9.3.2 Modo FreeFly

O modo FreeFly coloca a câmara ao nível do solo com total liberdade de movimento, sendo

adequado para a inspeção detalhada de entidades individuais e dos seus arredores. A câmara mantém-se na vertical relativamente à superfície do globo à medida que se move.

Ação	Controlo
Avançar / recuar / esquerda / direita	W A S D
Mover para cima / baixo	Q / E
Olhar em redor	Rato (quando o rato está bloqueado)
Desbloquear o rato para interagir com a UI	Tab
Bloquear o rato para retomar a observação	Tab

Tabela 69 - Controlos do Modo FreeFly

Quando o rato está desbloqueado (Tab), o movimento do cursor deixa de controlar a direção da câmara, permitindo ao utilizador interagir com elementos da interface e entidades sem que a vista se desloque. Prima Tab novamente para rebloquear o rato e retomar a observação livre.

9.3.3 Mudança de Modos de Câmara

O botão de alternância do modo de câmara está localizado na barra de ferramentas no topo do ecrã. Clicar nele alterna entre os modos RTS e FreeFly. Ao alternar para FreeFly, a câmara ajusta-se para uma orientação vertical relativamente à superfície do globo na posição atual.

9.4 Compreender a Interface

2 Vista geral da interface de visualização do DT4MOB mostrando a barra de ferramentas, o painel de outline da cena e uma janela de inspeção de entidade aberta.

A interface é sempre composta por três elementos persistentes visíveis.

9.4.1 Barra de Ferramentas

A barra de ferramentas estende-se ao longo do topo do ecrã e disponibiliza controlos para:

- **Alternância do modo de câmara** — alterna entre os modos de câmara RTS e FreeFly.
- **Seletor de tipo de entidade** — um menu suspenso para escolher o tipo de entidade a colocar quando o modo de colocação está ativo.
- **Botão Colocar Entidade** — ativa o modo de colocação para o tipo de entidade selecionado (ver Secção 6).

9.4.2 Painel de Outline da Cena

O painel de outline da cena é um painel deslizante que lista todas as entidades atualmente carregadas na cena. Pode ser aberto e fechado usando o botão **Outline** na barra de ferramentas. O painel é atualizado em tempo real à medida que as entidades são carregadas e descarregadas durante a navegação no globo.

Um campo de pesquisa no topo do painel permite filtrar a lista por identificador de entidade ou nome

de exibição. Clicar em qualquer linha da lista abre ou traz para a frente a janela de inspeção dessa entidade.

9.4.3 Marcadores de Entidade

As entidades são representadas na cena 3D como malhas posicionadas nas suas coordenadas geográficas. Mover o cursor sobre uma entidade realça-a com um contorno colorido, indicando que pode ser selecionada. Clicar numa entidade realçada abre a sua janela de inspeção.

9.5 Interação com Entidades

9.5.1 Pairar e Selecionar

Mover o cursor sobre qualquer entidade na cena faz aparecer um contorno de realce à sua volta. Isto indica que a entidade está sob o cursor e pode ser selecionada.

Clicar na entidade realçada abre a sua janela de inspeção. Várias janelas de inspeção podem estar abertas simultaneamente, uma por entidade. Se uma janela de inspeção já estiver aberta para a entidade clicada, esta é trazida para a frente em vez de ser aberta novamente.

As janelas de inspeção são arrastáveis e redimensionáveis. Clique e arraste a barra de título para mover uma janela e arraste o controlo de redimensionamento no canto inferior direito para a redimensionar.

9.5.2 Janela de Inspeção

3 Janela de inspeção de entidade mostrando o cabeçalho e o separador Info.

O cabeçalho da janela de inspeção exhibe:

- **Identificador da entidade** — o thingId completo da entidade.
- **Marca de tipo** — um rótulo curto que indica o tipo de entidade (e.g. GEO, INSTR).
- **Indicador de estado** — mostra se os dados da entidade são **Reais** (dados de sensores em tempo real) ou **Simulados** (dados produzidos por um modelo de simulação).

O corpo da janela está dividido em quatro separadores.

9.5.2.1 Separador de Informação

O separador Info exhibe os atributos mais relevantes da entidade selecionada como uma lista de pares de valor etiquetados. Os valores são atualizados automaticamente à medida que os dados em tempo real chegam do servidor, sem necessitar de qualquer ação do utilizador.

Se o conjunto predefinido de campos não exibir a informação necessária, clique no botão **Config Panel** para abrir o painel de configuração de campos. Este painel lista todos os campos disponíveis para o tipo de entidade e permite selecionar qualquer combinação deles para exibição no separador Info.

9.5.2.2 Separador JSON Bruto

4 Separador Raw JSON mostrando o estado completo da entidade como uma árvore recolhível.

O separador Raw JSON mostra o estado bruto completo da entidade tal como recebido do backend Ditto. Os dados são apresentados como uma árvore recolhível, permitindo expandir ou recolher

qualquer parte da estrutura JSON. Este separador é útil para inspecionar campos que não são apresentados no separador Info.

9.5.3 Separador de Associações

5 Separador Associations listando entidades ligadas à entidade selecionada.

O separador Associations lista outras entidades do Gémeo Digital que estão ligadas à entidade selecionada. Cada linha mostra o identificador e o tipo da entidade associada. Clicar em Open em qualquer linha abre a janela de inspeção dessa entidade.

9.5.4 Separador de Modelos

6 Separador Models com alternâncias de visibilidade por camada.

O separador Models lista as camadas de malha visual associadas à entidade. Cada camada pode ser ativada ou desativada independentemente usando o botão no lado direito da sua linha. Isto é útil para entidades com múltiplos componentes visuais, como entidades de simulação de incêndio que exibem camadas de malha separadas para diferentes horizontes temporais.

9.6 Painel de Outline da Cena

7 Painel de outline da cena mostrando a lista de entidades e o campo de pesquisa.

O painel de outline da cena fornece uma lista completa de todas as entidades atualmente carregadas na cena. Pode ser aberto clicando no botão **Outline** na barra de ferramentas.

O campo de pesquisa no topo do painel filtra a lista em tempo real à medida que o texto é digitado. O filtro corresponde tanto ao identificador da entidade como ao seu nome de exibição, facilitando a localização de uma entidade específica numa cena grande.

Clicar em qualquer linha da lista abre ou foca a janela de inspeção dessa entidade, equivalente a clicar na entidade diretamente na cena 3D.

9.7 Colocação de Entidades

A plataforma de visualização permite que novas entidades sejam colocadas diretamente na cena. Apenas os tipos de entidade com suporte do lado do servidor podem ser colocados; atualmente, isto limita-se a pontos de ignição de incêndio, para os quais o servidor calcula e devolve dados de simulação de propagação de incêndio após a colocação.

Para colocar uma entidade:

1. Selecione o tipo de entidade desejado no menu suspenso **Type** na barra de ferramentas.
2. Clique no botão **Place Entity** para entrar no modo de colocação.
3. Mova o cursor sobre a cena — uma pré-visualização fantasma da entidade seguirá a superfície do terreno sob o cursor.
4. Clique para confirmar a posição de colocação.

A entidade aparece na cena imediatamente. O servidor processa a colocação em segundo plano e atualiza a entidade com os resultados da simulação assim que estiverem prontos; estas atualizações são refletidas automaticamente na janela de inspeção da entidade.

9.8 Atualizações em Tempo Real

Todos os dados das entidades exibidos na plataforma de visualização são continuamente sincronizados com o backend Ditto através de uma ligação WebSocket persistente. Não é necessária qualquer atualização manual em nenhum momento.

Quando novos dados chegam para uma entidade, qualquer janela de inspeção aberta para essa entidade atualiza automaticamente os seus valores exibidos. Os valores do separador Info são atualizados no local sem reconstruir o separador, e a árvore Raw JSON reflete o estado mais recente da entidade.

O **indicador de estado** no cabeçalho da janela de inspeção indica a natureza dos dados da entidade:

- **Real** — os dados da entidade originam-se de sensores físicos ou dispositivos em tempo real.
- **Simulado** — os dados da entidade são produzidos por um modelo de simulação em vez de uma fonte física.

10 DATA BRIDGE

10.1 Visão Geral

O Data Bridge é um serviço de integração de dados que consulta múltiplas fontes de dados rodoviários e meteorológicos (IPMA, Waze e ficheiros GeoJSON locais) e encaminha os dados normalizados para um registo de dispositivos IoT Eclipse Hono usando o protocolo Eclipse Ditto sobre HTTP.

10.2 Pré-requisitos

Antes de utilizar o Data Bridge, certifique-se de que tem:

Requisito	Versão/Detalhes
Python	3.13 ou superior
Eclipse Hono	Instância com um adaptador HTTP configurado usando autenticação baseada em certificado
Ambiente virtual Python	Um ambiente virtual configurado, seja usando uv ou qualquer outro sistema compatível com PEP-518
Acesso à API IPMA Open	Ter ligação à internet a <code>api.ipma.pt</code>
Acesso à API Waze CCP	Capacidade de utilizar a API Waze CCP

Tabela 70 - Pré-requisitos do Data Bridge

NOTA: A interação com o Waze está atualmente indisponível.

10.3 Configuração

O Data Bridge é configurável através de um ficheiro `config.toml`, que contém a informação necessária para o programa carregar os módulos esperados.

A estrutura deste ficheiro é a seguinte:

Configuração	Tipo	Predefinição
Hono	Object	null
Devices	Array	null

Tabela 71 - Estrutura de Configuração

10.3.1 Objeto Hono

No objeto Hono, os seguintes campos são definidos:

Configuração	Tipo	Predefinição	Descrição
<code>http_adapter</code>	URL	<code>https://localhost:8443</code>	URL do HTTP Adapter da instância Hono a ser utilizada
<code>tenant_id</code>	String	<code>"DEFAULT_TENANT"</code>	ID do inquilino para o qual as mensagens serão enviadas. O inquilino DEVE já existir antes de o Data Bridge ser iniciado, caso contrário ocorrerá um erro.
<code>server_cert_path</code>	Path	null	Este campo é OPCIONAL. Contém o caminho para o certificado x509 do servidor, caso seja necessário.

Tabela 72 - Campos do Objeto Hono

Todas as ligações feitas pelo Data Bridge ao Eclipse Hono são feitas via HTTPS, e o contexto SSL/TLS está ativo, significando que se o Data Bridge não conseguir verificar o certificado apresentado pelo HTTP Adapter, o programa será interrompido. Para evitar isto, caso o endpoint não contenha um certificado x509 público válido, pode ser passado um certificado raiz para a aplicação usando `server_cert_path`.

10.3.2 Array de Dispositivos

O array de dispositivos é uma lista que contém vários objetos de dispositivo. Cada objeto é definido da seguinte forma:

Configuração	Tipo	Predefinição	Descrição
<code>cert_path</code>	Path	null	Caminho para o certificado x509 do dispositivo, utilizado para autenticar o dispositivo específico no HTTP Adapter no Eclipse Hono.

Configuração	Tipo	Predefinição	Descrição
private_key	Path	null	Caminho para a chave privada correspondente ao certificado.
policy_id	String	null	policy que o Eclipse Ditto aplicará sobre os comandos que o Data Bridge enviará através do Eclipse Hono.
subject	str	null	Subject a ser usado pelo dispositivo no Eclipse Ditto.
strategies	Array	[]	Explicado em maior detalhe na secção seguinte

Tabela 73 - Campos do Array de Dispositivos

Os ThingIds no Eclipse Ditto seguem o padrão já estabelecido namespace:subject:id existente na infraestrutura. O namespace é definido pelo dispositivo, e o dispositivo terá controlo apenas sobre os Things neste namespace. Este tem de corresponder ao campo Common Name do certificado x509 fornecido.

10.3.3 Estratégias

A arquitetura deste programa baseia-se no que é chamado de strategies, que é um módulo reutilizável que define os dados que o dispositivo deve enviar para o Eclipse Ditto.

Cada estratégia terá de definir pelo menos um type, que é o discriminador entre as diferentes estratégias, e um subject, que definirá a parte subject do ThingId.

Campo	Tipo	Descrição
type	String	Discriminador da estratégia a utilizar.
namespace	String	Namespace a ser usado no ThingId

Tabela 74 - Estrutura de Estratégia

Atualmente, existem as seguintes estratégias:

- meteo
- meteo_warnings
- geojson

10.3.3.1 Estratégia Meteorológica

A estratégia meteo é responsável por consultar a API IPMA e criar/atualizar Things no Eclipse Ditto que contêm os dados das várias estações meteorológicas que a IPMA disponibiliza. Esta estratégia criará Things com um ThingId formatado como namespace:subject:station_id, onde station_id é um número fornecido pela API da IPMA.

76 Para fazer com que um dispositivo utilize esta estratégia, o seguinte objeto deve ser configurado e adicionado ao seu array de estratégias:

Campo	Tipo	Descrição
type	Literal	meteo
namespace	String	Namespace a ser usado no ThingId

Tabela 75 - Definição da estratégia meteorológica

10.3.3.2 Estratégia de Avisos Meteorológicos

A estratégia `meteo_warnings` é responsável por consultar a API IPMA e atualizar os Things das estações meteorológicas existentes com uma nova feature `events` que conterá avisos meteorológicos conforme fornecidos pela IPMA. Um determinado aviso será adicionado às 3 estações meteorológicas mais próximas (num máximo de 100 km) da área do aviso. [IPMA API](https://api.ipma.pt)

Para fazer com que um dispositivo utilize esta estratégia, o seguinte objeto deve ser configurado e adicionado ao seu array `strategies`:

Campo	Tipo	Descrição
type	Literal	warnings
namespace	String	Namespace a ser usado no ThingId

Tabela 76 - Definição da estratégia de avisos meteorológicos

10.3.3.3 Estratégia de Tráfego

NOTA: À data de redação deste manual, esta estratégia não está a funcionar como pretendido e não deve ser ativada. É um componente legado e poderá ser atualizado no futuro.

A estratégia `traffic` é responsável por, dada uma localização geográfica e um raio, simular um Ponto Sensor nessa localização e publicar os dados de tráfego retornados pelo Waze. Também aceita um `sensor_name` que será usado no formato do ThingId.

Para fazer com que um dispositivo utilize esta estratégia, o seguinte objeto deve ser configurado e adicionado ao seu array de estratégias:

Campo	Tipo	Descrição
type	Literal	traffic
namespace	String	Namespace a ser usado no ThingId
sensor_name	String	O nome do sensor a ser usado no formato ThingId.
road	String	O nome da estrada onde este sensor está localizado.
latitude	Float	A latitude geográfica, em WGS84, onde o sensor será localizado.
longitude	Float	A longitude geográfica, em WGS84, onde o sensor será localizado.
radius	Integer	O raio (em metros) que o sensor utilizará para

Campo	Tipo	Descrição
		consultar a API do Waze para obter alerts e jams

Tabela 77 - Definição da estratégia de tráfego

10.3.3.4 estratégia GeoJSON

A estratégia geojson é responsável por, dado um caminho GeoJSON OU diretório contendo ficheiros GeoJSON, recuperar todas as features dentro desse GeoJSON e criar um Thing no Eclipse Ditto com a informação contida nas properties coordinates no caso de um ponto único, ou geometry no caso de múltiplos pontos.

NOTA: À data de redação deste manual, esta estratégia espera que a informação geográfica no GeoJSON seja expressa na projeção ETRS89, e converte-a automaticamente para a projeção WGS84. Isto poderá mudar no futuro, e a estratégia poderá ser atualizada para recuperar automaticamente a projeção original do ficheiro GeoJSON. O resultado será sempre em WGS84.

Para fazer com que um dispositivo utilize esta estratégia, o seguinte objeto deve ser configurado e adicionado ao seu array strategies:

Campo	Tipo	Descrição
type	Literal	geojson
namespace	String	Namespace a ser usado no ThingId
file	Path	A localização do ficheiro para a estratégia ler
dir	Path	A localização do diretório que contém os ficheiros GeoJSON para a estratégia ler

Tabela 78 - Definição da estratégia GeoJSON

É importante notar que estes campos são MUTUAMENTE EXCLUSIVOS, significando que se file estiver definido, dir deve estar indefinido e vice-versa.

10.4 Nota sobre geotiles

De acordo com o padrão existente do sistema, este Data Bridge adiciona um expiry_ts e um geotile a todos os Things que cria, onde o primeiro é uma indicação para o garbage collector se um Thing deve ou não ser eliminado, enquanto o segundo é um atributo que permite a pesquisa geográfica rápida de Things numa determinada área (um geotile).

10.5 Guia de Deployment

O Data Bridge é uma aplicação Python e pode ser implantado de 3 formas diferentes: - Instanciação direta da aplicação - Utilização do contentor Docker fornecido - Utilização de um chart Helm (para implantação em Kubernetes).

Cada invocação executa uma única passagem de execução. Para operação periódica, agende via CronJob ou escalonador externo. O chart Helm fornecido configura automaticamente um Kubernetes CronJob. Para outros métodos de implantação, configure o escalonamento externamente (e.g., cron do Linux ou Windows Task Scheduler).

10.5.1 Instanciação Direta

A aplicação Python foi desenvolvida num ambiente gerido por [uv](#). No entanto, é compatível com PEP-518, significando que a ferramenta uv não é necessária para executar a aplicação, uma vez que as dependências podem ser geridas e instaladas usando pip num ambiente virtual configurado, ou venv.

Usar a instanciação direta é tão simples como executar o ficheiro `main.py` no ambiente gerido (usando `uv run main.py` se utilizar uv ou executando `python main.py` na venv se utilizar qualquer outra ferramenta compatível com PEP-518).

Neste caso, o ficheiro de configuração `config.toml` deve ser colocado na raiz do projeto, que será o diretório onde o ficheiro `main.py` está localizado. O Data Bridge carregará automaticamente esse ficheiro e aplicará as configurações nele contidas. Além disso, dado que este projeto utiliza `pydantic-settings`, estas também podem ser definidas usando variáveis de ambiente. Estas são nomeadas exatamente como os campos, usando um duplo underscore (`__`) para objetos aninhados. Por exemplo, o objeto `http_adapter` do objeto Hono é definido como `HONO__HTTP_ADAPTER`. Para definir arrays e tipos mais complexos, pode ser usada uma string codificada em JSON. Como exemplo, para definir um dispositivo, pode ser usada a seguinte variável de ambiente:

```
DEVICES="[
  {
    cert_path:<path>,
    private_key:<path>,
    policy_id:<str>,
    subject:<str>,
    strategies:[
      {
        type:<str>,namespace:<str>,
      },
    ],
  }
]"
```

10.5.2 Dockerfile

A utilização do Dockerfile é mais simples do que a instanciação direta, pois a imagem apenas precisa de ser construída (ou usar a imagem pré-construída em `atnog-harbor.av.it.pt/dt4mob/data-bridge`), montando o ficheiro `config.toml` no diretório `/app/config.toml`.

Isto pode ser feito com o comando `docker run -v config.toml:/app/config.toml atnog-harbor.av.it.pt/dt4mob/data-bridge`. Nota: Isto executa uma única passagem de execução do Data Bridge e atualiza os Things uma vez. O escalonamento periódico deve ser configurado pelo administrador.

Adicionalmente, tal como na instanciação direta, a configuração pode ser feita com variáveis de ambiente.

Quaisquer outros ficheiros que possam ser necessários ao Data Bridge (como no caso da estratégia `geojson`) devem também ser montados no contentor Docker, e no caminho configurado no ficheiro de configuração `config.toml`. Este caminho DEVE ser igual ao caminho do ficheiro dentro do Contentor Docker, e não o do host.

10.5.3 Chart Helm

O chart Helm está disponível no repositório e pode ser instalado usando o instalador Helm (`helm install data-bridge <localização_do_chart> -f <localização_do_values.yml>`). A configuração neste

caso é feita através do ficheiro `values.yml`, mas segue a mesma estrutura do ficheiro de configuração `config.toml`.

Quaisquer outros ficheiros que possam ser necessários ao Data Bridge (como no caso da estratégia `geojson`) devem ser montados no pod Kubernetes, e no caminho configurado no ficheiro de configuração `values.yml`.

11 METEO POPULATOR

11.1 Visão Geral

O Meteo Populator é um serviço Python que associa automaticamente Things do Eclipse Ditto às suas estações meteorológicas mais próximas do IPMA (Instituto Português do Mar e da Atmosfera).

O serviço obtém as estações meteorológicas ativas da API IPMA, recupera todos os Things do Eclipse Ditto que têm dados de localização, geometria ou coordenadas, calcula as estações meteorológicas mais próximas para cada Thing com base na proximidade geográfica, e adiciona um atributo com referências para as suas estações mais próximas, na forma de IDs de Thing Ditto. Usa OIDC para autenticação com a infraestrutura.

11.2 Pré-requisitos

Antes de utilizar o Meteo Populator, certifique-se de que tem:

Requisito	Versão/Detalhes
Python	3.12 ou superior
Eclipse Ditto	Instância com uma integração OIDC configurada
Ambiente virtual Python	Um ambiente virtual configurado, seja usando <code>uv</code> ou qualquer outro sistema compatível com PEP-518
Acesso à API IPMA Open	Ter ligação à internet a <code>api.ipma.pt</code>

Tabela 79 - Pré-requisitos do Meteo Populator

11.3 Configuração

O Meteo Populator é configurável através de um ficheiro `config.toml`, que contém a informação necessária para o programa carregar os módulos esperados.

A estrutura deste ficheiro é a seguinte:

Configuração	Tipo	Predefinição
<code>populator</code>	Object	<code>{ dry_run = false }</code>
<code>logging</code>	Object	<code>{ level = "INFO" }</code>
<code>station</code>	Object	<code>{ namespace = "meteo", subject = "internal" }</code>
<code>filter</code>	Object	Ver abaixo
<code>oidc</code>	Object	<code>{ scope = "openid" }</code>

Tabela 80 - Estrutura da configuração do Meteo Populator

11.3.1 Objeto Populator

80 O objeto populator contém um único campo:

Configuração	Tipo	Predefinição	Descrição
dry_run	Boolean	false	Quando true, o Meteo Populator NÃO atualizará quaisquer Things no Eclipse Ditto, mas em vez disso registrará a operação que normalmente executaria, nomeadamente que Thing seria atualizado e que atributo seria adicionado.

Tabela 81 - Campos do Objeto Populator

11.3.2 Objeto Logging

O objeto logging contém um único campo:

Configuração	Tipo	Predefinição	Descrição
level	String	"INFO"	O nível de registo da aplicação. A aplicação apenas mostrará registos cuja gravidade seja desse nível ou superior.

Tabela 82 - Campos do Objeto Logging

11.3.3 Objeto Station

O objeto station contém informações sobre os Things que representam as estações meteorológicas:

Configuração	Tipo	Predefinição	Descrição
namespace	String	"meteo"	Namespace usado no ThingId dos Things das estações meteorológicas.
subject	String	"internal"	Subject usado no ThingId dos Things das estações meteorológicas.

Tabela 83 - Campos do Objeto Station

Os ThingIds seguem o padrão namespace:subject:id, pelo que os ThingIds das estações serão formatados como namespace:subject:<station_id>, onde station_id é um número fornecido pela API da IPMA.

11.3.4 Objeto Filter

Tabela 83 – O objeto filter contém os filtros usados na API Thing Search do Eclipse Ditto para

recuperar os Things que precisam de ser atualizados com a informação das estações meteorológicas mais próximas:

Configuração	Tipo	Predefinição	Descrição
filter	String	“and(or(exists(attributes/location),exists(attributes/geometry),exists(attributes/coordinates)),not(eq(namespace,"meteo")))”	Uma expressão Resource Query Language usada na API Thing Search do Eclipse Ditto. A expressão predefinida recupera todos os Things que contêm os atributos location, geometry ou coordinates e cujo Namespace não seja meteo, para evitar adicionar o atributo das estações meteorológicas mais próximas a um Thing que já representa uma estação meteorológica. Resource Query Language
fields	String	"thingId,attributes(location,geometry,coordinates)"	Os campos a recuperar do Ditto. Apenas o ThingId e o atributo de localização geográfica são necessários, mas o campo fica configurável para preparação para o futuro.

Tabela 84 - Campos do Objeto Filter

11.3.5 Objeto OIDC

O objeto OIDC configura a autenticação com o fornecedor OIDC da infraestrutura:

Configuração	Tipo	Predefinição	Descrição
base_url	String	""	O URL base da infraestrutura.
realm	String	""	O realm para autenticação OIDC.
username	String	""	O nome de utilizador para autenticação.
password	String	""	A palavra-passe para autenticação.
client_id	String	""	O ID do cliente que identifica o serviço que está a autenticar-se.
scope	String	"openid"	O âmbito do processo de autenticação.

Tabela 85 - Campos do Objeto OI DC

11.4 Nota sobre geotiles

De acordo com o padrão existente do sistema, este Meteo Populator adiciona um `expiry_ts` e um `geotile` a todos os Things que cria, onde o primeiro é uma indicação para o garbage collector se um Thing deve ou não ser eliminado, enquanto o segundo é um atributo que permite a pesquisa geográfica rápida de Things numa determinada área (um geotile).

11.5 Guia de Deployment

O Meteo Populator é uma aplicação Python e pode ser implantado de 3 formas diferentes:

- Instanciação direta da aplicação
- Utilização do contentor Docker fornecido
- Utilização de um chart Helm (para implantação em Kubernetes)

Cada invocação executa uma única passagem de execução. Para operação periódica, agende via CronJob ou escalonador externo. O chart Helm fornecido configura automaticamente um Kubernetes CronJob. Para outros métodos de implantação, configure o escalonamento externamente (e.g., cron do Linux ou Windows Task Scheduler).

11.5.1 Instanciação Direta

A aplicação Python foi desenvolvida num ambiente gerido por [uv](#). No entanto, é compatível com PEP-518, significando que a ferramenta `uv` não é necessária para executar a aplicação, uma vez que as dependências podem ser geridas e instaladas usando `pip` num ambiente virtual configurado, ou `venv`.

Usar a instanciação direta é tão simples como executar o ficheiro `main.py` no ambiente gerido (usando `uv run main.py` se utilizar `uv` ou executando `python main.py` na `venv` se utilizar qualquer outra ferramenta compatível com PEP-518).

Neste caso, o ficheiro de configuração `config.toml` deve ser colocado na raiz do projeto, que será o diretório onde o ficheiro `main.py` está localizado. O Meteo Populator carregará automaticamente esse ficheiro e aplicará as configurações nele contidas. Além disso, dado que este projeto utiliza `pydantic-settings`, estas também podem ser definidas usando variáveis de ambiente. Estas são nomeadas exatamente como os campos, usando um duplo underscore (`__`) para objetos aninhados. Por exemplo, o campo `base_url` do objeto `oidc` é definido como `OI DC__BASE_URL`. Para definir arrays e tipos mais complexos, pode ser usada uma string codificada em JSON.

11.5.2 Dockerfile

A utilização do Dockerfile é mais simples do que a instanciação direta, pois a imagem apenas precisa de ser construída (ou usar a imagem pré-construída em `atnog-harbor.av.it.pt/dt4mob/meteo-populator`), montando o ficheiro `config.toml` no diretório `/app/config.toml`.

Isto pode ser feito com o comando `docker run -v config.toml:/app/config.toml atnog-harbor.av.it.pt/dt4mob/meteo-populator`.

Nota: Isto executa uma única passagem de execução do Meteo Populator e atualiza os Things uma vez. O escalonamento periódico deve ser configurado pelo administrador.

Adicionalmente, tal como na instanciação direta, a configuração pode ser feita com variáveis de ambiente.

11.5.3 Chart Helm

O chart Helm está disponível no repositório e pode ser instalado usando o instalador Helm (`helm install meteo-populator <caminho_para_chart> -f <caminho_para_values.yml>`). A configuração neste caso é feita através do ficheiro `values.yml`, mas segue a mesma estrutura do ficheiro de configuração `config.toml`

12 NAMESPACE REMOVER

12.1 Visão Geral

O Namespace Remover é um utilitário de linha de comandos que elimina em massa Things do Eclipse Ditto de uma instância Ditto. Autentica via OIDC (password grant), pesquisa Things que correspondam a um namespace e/ou consulta de filtro RQL, e elimina-os com paginação automática e renovação de token.

É útil para limpar entidades órfãs ou desatualizadas (e.g., sinais de trânsito estáticos ou barreiras de segurança) que não possuem o atributo de ciclo de vida `expiry_ts` e, portanto, não podem ser removidas pelo garbage collector através do mecanismo normal de expiração.

12.2 Pré-requisitos

Antes de utilizar o Namespace Remover, certifique-se de que tem:

Requisito	Versão/Detalhes
Python	3.13 ou superior
Eclipse Ditto	Uma instância com o módulo Things-Search ativado e acessível via HTTPS
Credenciais OIDC	Um utilizador Keycloak com acesso password grant ao cliente Ditto
Ambiente virtual Python	Um ambiente virtual configurado, seja usando <code>uv</code> ou qualquer outro sistema compatível com PEP-518

Tabela 86 - Pré-requisitos do Namespace Remover

12.3 Configuração

O Namespace Remover é configurado através de um ficheiro `config.toml` colocado na raiz do projeto (onde se encontra `main.py`). A estrutura é a seguinte:

Configuração	Tipo	Predefinição	Descrição
<code>base_url</code>	URL	""	URL base da instância Eclipse Ditto (e.g. <code>https://ditto.example.com</code>). Isto é usado tanto para o endpoint de token OIDC como para a API Ditto.

Configuração	Tipo	Predefinição	Descrição
[oidc]	Object	—	Credenciais de autenticação OIDC. Ver abaixo.

Tabela 87 - Configuração do Namespace Remove

12.3.1 Objeto OIDC

Configuração	Tipo	Predefinição	Descrição
realm	String	""	Realm do Keycloak a que o utilizador pertence.
username	String	""	Nome de utilizador OIDC.
password	String	""	Palavra-passe OIDC.
client_id	String	""	ID do cliente OIDC.
scope	String	"openid"	Âmbito OIDC a solicitar. O openid predefinido é suficiente para a maioria das configurações.

Tabela 88 - Campos do Objeto OIDC

12.3.2 Exemplo de config.toml

```
base_url = "https://ditto.example.com"

[oidc]
realm = "my-realm"
username = "my-user"
password = "s3cret"
client_id = "ditto"
# scope = "openid" # opcional, predefinido como "openid"
```

12.4 Argumentos de CLI

88 O programa aceita os seguintes argumentos de linha de comandos:

Argumento	Tipo	Descrição
--namespaces	Lista de strings (zero ou mais)	Namespace(s) para filtrar. Apenas os Things cujo thingId corresponda a algum destes namespaces serão eliminados. Se nenhum --query for fornecido, todos os Things nos namespaces especificados são eliminados.

Argumento	Tipo	Descrição
--query	String	Uma string de filtro RQL Ditto . Apenas os Things que corresponderem ao filtro serão eliminados. Quando combinado com --namespaces, é utilizada a interseção de ambos os critérios.

Tabela 89 - Argumentos de CLI do Namespace Remover

NOTA: Se nem --namespaces nem --query forem fornecidos, o programa será encerrado sem fazer nada, pois não há forma de determinar quais Things eliminar.

12.5 Fluxo de Execução

O programa segue esta sequência de operações:

1. **Autenticar** — Executa um OIDC password grant contra `<base_url>/auth/realms/<realm>/protocol/openid-connect/token` para obter um access token e um refresh token.
2. **Pesquisar** — Envia um pedido POST para `<base_url>/api/2/search/things` com os namespaces fornecidos e/ou consulta de filtro. A resposta contém uma lista de valores thingId e, se houver mais resultados, um cursor para paginação.
3. **Eliminar** — Envia um pedido DELETE para cada thingId retornado para `<base_url>/api/2/things/<thingId>`.
4. **Paginar** — Se um cursor foi retornado, repete os passos 2–3 com o cursor até não restarem mais resultados.
5. **Renovar** — Antes de cada lote paginado, verifica se o access token está prestes a expirar. Em caso afirmativo, renova o token usando o refresh token armazenado antes de continuar.

12.6 Utilização

12.6.1 Utilização Básica

Eliminar todos os Things num único namespace:

```
uv run main.py --namespaces example
```

Eliminar Things em múltiplos namespaces:

```
uv run main.py --namespaces example org.other
```

Eliminar Things que correspondam a uma consulta de filtro dentro de um namespace:

```
uv run main.py --namespaces example --query 'eq(attributes/type,"road_sign")'
```

Eliminar Things que correspondam a uma consulta de filtro em todos os namespaces (usar com precaução):

```
uv run main.py --query 'like(thingId,"*test*")'
```

12.7 Configuração de Variáveis de Ambiente

Como o projeto utiliza pydantic-settings, todos os campos do config.toml podem ser substituídos

através de variáveis de ambiente. Campos aninhados usam um duplo underscore (__) como delimitador.

Variável de ambiente	Campo correspondente
BASE_URL	base_url
OIDC__REALM	oidc.realm
OIDC__USERNAME	oidc.username
OIDC__PASSWORD	oidc.password
OIDC__CLIENT_ID	oidc.client_id
OIDC__SCOPE	oidc.scope

Tabela 90 - Variáveis de ambiente para configuração do Namespace Remover

Exemplo:

```
BASE_URL="https://ditto.example.com" \
OIDC__REALM="my-realm" \
OIDC__USERNAME="admin" \
OIDC__PASSWORD="s3cret" \
OIDC__CLIENT_ID="ditto" \
uv run main.py --namespaces example
```

12.8 Notas de Deployment

```
uv run main.py --namespaces example
```

Ou, se usar uma venv padrão com pip:

```
python main.py --namespaces example
```

O ficheiro `config.toml` deve estar no diretório raiz do projeto (juntamente com `main.py`). As substituições por variáveis de ambiente podem ser usadas adicionalmente ou em vez do ficheiro de configuração. Manual de Utilizador de Criação de Texturas.

Este programa gera duas malhas 3D texturizadas (formato `.gltf`) a partir de uma malha `.ply` e dados de nuvem de pontos obtidos de uma API. Produz uma textura baseada em indicadores de qualidade (alarmista) e outra baseada em valores de deslocamento. # Texture Creation

12.9 Visão Geral

O Texture Creation é um pipeline Python que cria texturas GLTF coloridas a partir de dados de nuvem de pontos e malhas 3D. Obtém malhas PLY e nuvens de pontos CSV de uma API, calcula cores de vértices, exporta como GLTF e carrega os resultados para armazenamento S3 com lógica de cache.

12.10 Pré-requisitos

Antes de utilizar o Gerador de Texturas de Malha, certifique-se de que tem:

Requisito	Versão/Detalhes
Python	3.12 ou superior
Armazenamento compatível com S3	Endpoint acessível com um bucket para malhas de entrada e texturas de saída
Acesso à API	API acessível com credenciais para dados de nuvem de pontos e malha
Ambiente virtual Python	Um ambiente virtual configurado, utilizando uv ou qualquer outro sistema compatível com PEP-518

Tabela 91 - Pré-requisitos do Texture Creations

12.11 Configuração

O Texture Creation é configurável através de um ficheiro `config.toml` na raiz do projeto. Copie `config.toml.example` para `config.toml` e preencha os valores para o seu ambiente. Adicionalmente, também pode ser configurado através de variáveis de ambiente, que podem ser definidas na shell ou carregadas a partir de um ficheiro `.env`.

Fonte	Prioridade
Variáveis de ambiente	Mais alta
Ficheiro <code>.env</code>	Média
<code>config.toml</code>	Mais baixa (configuração base)

Tabela 92 - Prioridades de Configuração

Os valores de configuração aninhados podem ser substituídos através de variáveis de ambiente ou de um ficheiro `.env` utilizando `__` como delimitador (por exemplo, `AUTH__TENANT=mycompany`).

A estrutura do `config.toml` é a seguinte:

- `api` (Object)
- `auth` (Object)
- `s3` (Object)
- `mesh` (Object)

12.11.1 API Object

Contém as informações de ligação para a API que fornece os dados de nuvem de pontos e malha.

Definição	Tipo	Descrição
base_url	URL	O URL base da API.

Tabela 93 - Configurações do Objeto API

12.11.2 AUTH Object

Contém as credenciais de autenticação e identificadores de recurso.

Definição	Tipo	Descrição
tenant	String	O identificador de inquilino para a API.
resource_id	String	O ID do recurso do ativo a processar.
username	String	O nome de utilizador para autenticação na API.
password	String	A palavra-passe para autenticação na API.

Tabela 94 - Configurações do Objeto AUTH

12.11.3 S3 Object

Contém as definições de ligação para o armazenamento compatível com S3 utilizado tanto para as malhas de entrada como para as texturas de saída.

Definição	Tipo	Descrição
endpoint	URL	O URL do endpoint compatível com S3.
access_key	String	A chave de acesso S3.
secret_key	String	A chave secreta S3.
bucket	String	O bucket S3 onde tanto os ficheiros .ply de entrada como .gltf de saída são armazenados.

Tabela 95 - Configurações do Objeto S3

12.11.4 MESH Object

Definição	Tipo	Predefinição	Descrição
path	Path	null	Caminho de ficheiro local opcional para um ficheiro de malha PLY. Se definido, o programa carregará a malha deste ficheiro e fará o upload

Definição	Tipo	Predefinição	Descrição
			para o S3, ignorando a recuperação por API e S3.

Tabela 96 - Configurações do Objeto MESH

12.11.5 Substituição por Variáveis de Ambiente

Os campos aninhados podem ser substituídos com __ como delimitador:

Variável de Ambiente	Caminho de Configuração
API__BASE_URL	api.base_url
AUTH__TENANT	auth.tenant
AUTH__RESOURCE_ID	auth.resource_id
AUTH__USERNAME	auth.username
AUTH__PASSWORD	auth.password
S3__ENDPOINT	s3.endpoint
S3__ACCESS_KEY	s3.access_key
S3__SECRET_KEY	s3.secret_key
S3__BUCKET	s3.bucket
MESH__PATH	mesh.path

Tabela 97 - Substituição por Variáveis de Ambiente

12.12 Como Funciona

O programa segue um pipeline sequencial:

12.12.1 Autenticação

O programa autentica-se na API utilizando as credenciais fornecidas na secção auth da configuração. Este chama o endpoint de início de sessão da API em {base_url}/{tenant}/api/accounts/login e recebe um bearer token. Este token é utilizado para todos os pedidos subsequentes à API.

12.12.2 Obtenção da Malha

A malha PLY de entrada é obtida utilizando a seguinte ordem de prioridade:

1. **Ficheiro local** — Se mesh.path estiver definido, a malha é carregada diretamente do

sistema de ficheiros local e enviada para o bucket S3 como {resource_id}.ply.

2. **Obtenção por API** — Se não houver caminho local definido, o programa tenta descarregar a malha da API em {base_url}/{tenant}/api/v1/ativos-geotecnicos/{resource_id}/modelo/{POLYGON_TYPE}/download. A malha descarregada é armazenada em cache no S3 para execuções subsequentes.
3. **Recurso a S3** — Se tanto a obtenção local como por API falharem, o programa tentará utilizar um ficheiro PLY existente previamente armazenado em cache no S3.

Se não for possível obter uma malha de nenhuma fonte, o programa termina com um erro.

12.12.3 Verificação de Cache

Antes de processar, o programa verifica se pode saltar o trabalho totalmente. Este compara o timestamp Last-Modified da fonte CSV da API com os timestamps dos ficheiros .gltf de saída existentes no bucket S3. Se os resultados forem mais recentes ou iguais à fonte, o processamento é ignorado e o programa termina.

Isto permite que a ferramenta seja executada de forma idempotente, onde execuções repetidas não produzem trabalho a menos que os dados de origem tenham sido alterados. Adicionalmente, isto poupa trabalho computacional.

12.12.4 Obtenção de Dados

Se for necessário processar, o programa obtém um ficheiro CSV da API em:

```
{base_url}/{tenant}/api/v1/ativos-geotecnicos/{resource_id}/modelo/{CSV_TYPE}/download
```

O CSV contém as seguintes colunas relevantes:

- X, Y, Z — Coordenadas espaciais de cada ponto
- Quality_flag — Classificação de qualidade (alert, alarm, OK)
- dist_3d_m — Magnitude de deslocamento 3D (com sinal, em metros)
- dx_m, dy_m, dz_m — Componentes de deslocamento

12.12.5 Cálculo de Cores

Dois mapas de cores são calculados em paralelo:

12.12.5.1 Textura Alarmista

Baseado no campo Quality_flag:

Flag	Cor	RGB
alert	Amarelo	(1.0, 1.0, 0.0)
alarm	Vermelho	(1.0, 0.0, 0.0)
OK (predefinição)	Verde	(0.0, 1.0, 0.0)

Tabela 98 - Cores da Textura Alarmista

12.12.5.2 Textura de Deslocamento

Baseado no campo `dist_3d_m` com valores limitados a ± 20 mm:

<code>dist_3d_m</code>	Cor	RGB
-20 mm (máx. negativo)	Rosa	(1.0, 0.0, 1.0)
0 mm (nenhum)	Verde	(0.0, 1.0, 0.0)
+20 mm (máx. positivo)	Vermelho	(1.0, 0.0, 0.0)

Tabela 99 - Cores da Textura de Deslocamento

Os valores entre estes extremos são interpolados linearmente, produzindo um gradiente suave. Os valores de `dist_3d_m` em falta são predefinidos para verde (sem deslocamento).

12.12.6 Interpolação

A nuvem de pontos colorida é interpolada na superfície da malha utilizando o método `interpolate` do `pyvista` com um único vizinho mais próximo (`n_points=1`) e um raio de pesquisa de 0,5 metros. As faces da malha são invertidas antes da interpolação para garantir normais de superfície corretas.

12.12.7 Exportação

Dois ficheiros `.gltf` são gerados e enviados para o bucket S3:

- `{resource_id}_alarmist.glTF` — Textura baseada no indicador de qualidade
- `{resource_id}_displacement.glTF` — Textura baseada no deslocamento

Ambos os ficheiros utilizam cores de vértice (RGB) incorporadas nos dados da malha.

12.13 Formato de Saída

Os ficheiros de saída são ficheiros `.glTF` (GL Transmission Format) padrão que contêm a geometria original da malha com dados de cor por vértice. Podem ser visualizados em qualquer visualizador compatível com GLTF (por exemplo, Babylon.js, Three.js, Windows 3D Viewer).

12.14 Guia de Implantação

O pipeline de criação de texturas é uma aplicação Python e pode ser implementado de 3 formas diferentes: - Instanciação direta da aplicação - Utilização do contentor Docker fornecido - Utilização de um Helm chart (para implementação em Kubernetes).

Cada invocação executa uma única passagem de execução. Para operação periódica, agende através de CronJob ou de um agendador externo. O Helm chart fornecido configura automaticamente um CronJob Kubernetes. Para outros métodos de implantação, configure o agendamento externamente (por exemplo, cron Linux ou Windows Task Scheduler).

12.14.1 Instanciação Direta

A aplicação Python foi desenvolvida num ambiente gerido por [uv](#). No entanto, é compatível com D3| PU (public) | Version 1 | Final

PEP-518, o que significa que a ferramenta uv não é necessária para executar a aplicação, pois as dependências podem ser geridas e instaladas utilizando pip num ambiente virtual configurado, ou venv.

A utilização da instanciação direta é tão simples como executar o ficheiro [main.py](#) no ambiente gerido:

```
# Utilizando uv
uv run main.py

# Ou utilizando pip/venv numa ferramenta compatível com PEP 518
python main.py
```

Neste caso, o ficheiro de configuração config.toml deve ser colocado na raiz do projeto, que é o diretório onde o ficheiro main.py está localizado. A aplicação carregará automaticamente esse ficheiro e aplicará as configurações nele contidas. Adicionalmente, dado que este projeto utiliza pydantic-settings, estas também podem ser definidas utilizando variáveis de ambiente. Estas têm o nome dos campos, utilizando um duplo underscore (__) para objetos aninhados. Por exemplo, o endpoint do objeto S3 é definido como S3__ENDPOINT. Docker

A utilização do Dockerfile é mais simples do que a instanciação direta, pois a imagem apenas precisa de ser construída (ou utilizar a imagem pré-construída em atnog-harbor.av.it.pt/dt4mob/texture-creation).`

```
docker build -t texture-creation .
# Ou utilizar a imagem pré-construída:
docker run atnog-harbor.av.it.pt/dt4mob/texture-creation
```

O config.toml e quaisquer outros ficheiros necessários devem ser montados no contentor nos caminhos configurados no config.toml. Estes caminhos DEVEM corresponder aos caminhos dentro do contentor, não aos do hospedeiro.

```
docker run -v /path/to/config.toml:/app/config.toml atnog-harbor.av.it.pt/dt4mob/texture-creation
```

Nota: Isto executa uma única passagem de execução do Gerador de Texturas. O agendamento periódico deve ser configurado pelo administrador.

12.14.2 Helm Chart

O Helm chart está disponível no repositório GitHub [dt4mob-platform](#) e pode ser instalado utilizando o instalador Helm:

```
helm install texture-creation <path_to_chart> -f <path_to_values.yml>
```

A configuração é feita através do ficheiro values.yml, mas segue a mesma estrutura do ficheiro de configuração config.toml

13 SIMULAÇÃO DA FAIXA RESERVADA A5

13.1 Visão Geral

Esta aplicação permite avaliar o impacto da implementação de uma faixa reservada com acesso pago na autoestrada A5 através de simulação de tráfego.

O utilizador pode definir um conjunto de parâmetros relativos ao cenário a analisar e iniciar uma simulação. Após a conclusão da execução, a aplicação apresenta os resultados obtidos para quatro cenários distintos, permitindo comparar indicadores operacionais e económicos, como a utilização da faixa reservada, as velocidades médias e a receita estimada.

13.2 Aceder à aplicação

A aplicação encontra-se disponível através de um navegador Web.

Após aceder ao endereço fornecido, é apresentada a página inicial da aplicação, onde se encontram os parâmetros da simulação e a área de apresentação dos resultados.

13.3 Configuração da simulação

Antes de iniciar uma simulação devem ser definidos os parâmetros de entrada.

13.3.1 Percentagem de Veículos

Define a percentagem de veículos elegíveis que optam por pagar para utilizar a faixa reservada.

Valores mais elevados representam uma maior adesão ao sistema de portagem.

13.3.2 Preço da Portagem (€)

Corresponde ao valor cobrado aos veículos que utilizam a faixa reservada.

Este parâmetro influencia diretamente a receita estimada apresentada nos resultados.

13.3.3 Fator de Escala do Tráfego

Permite aumentar ou reduzir o volume de tráfego relativamente aos dados históricos utilizados como base da simulação.

Por exemplo:

- **1.0** – tráfego correspondente aos dados históricos;
- **2.0** – aproximadamente o dobro do volume de tráfego;
- **3.0** – aproximadamente o triplo do volume de tráfego.

Após configurar os parâmetros pretendidos, deve ser selecionado o botão Start Simulation.

13.4 Execução da Simulação

Depois da submissão do pedido, a aplicação inicia automaticamente o processamento da simulação.

Durante este processo poderá ser apresentado um dos seguintes estados:

- **Submitting simulation...** – o pedido está a ser enviado para o servidor;
- **Waiting for a free simulation slot...** – o pedido aguarda disponibilidade para iniciar a execução;
- **Simulations running...** – os cenários estão a ser simulados;
- **Finished!** – a execução terminou e os resultados encontram-se disponíveis.

13.5 Interpretação dos resultados

Após a conclusão da execução é apresentada uma tabela comparativa com os resultados dos quatro cenários simulados.

Os cenários considerados são:

- **Real** – reprodução do tráfego histórico observado;
- **All Open** – todas as faixas permanecem disponíveis para circulação;
- **Closed Lane** – a faixa reservada encontra-se encerrada ao tráfego geral;
- **Paid Lane** – a faixa reservada encontra-se disponível apenas para os veículos autorizados mediante pagamento.

Para cada cenário são apresentados os seguintes indicadores:

- **Total de veículos** – número total de veículos simulados;
- **Veículos na faixa reservada** – número de veículos que utilizaram a faixa reservada;
- **Utilização da faixa** – percentagem de utilização da faixa reservada;
- **Receita** – valor estimado obtido através da cobrança da portagem;
- **Velocidade média por faixa** – velocidade média observada em cada uma das faixas monitorizadas.

A comparação entre os quatro cenários permite avaliar o impacto da implementação da faixa reservada em diferentes condições de procura e diferentes estratégias de tarifação.

14 SIMULAÇÃO DE TRÁFEGO NA VCI

14.1 Visão Geral

A Simulação de Tráfego da VCI permite reproduzir o comportamento do tráfego na Via de Cintura Interna (VCI) utilizando um modelo microscópico desenvolvido no Eclipse SUMO.

A simulação foi calibrada a partir de contagens históricas de tráfego disponibilizadas pela Infraestruturas de Portugal, permitindo gerar cenários representativos da circulação real.

O utilizador pode executar a simulação previamente calibrada, visualizar a circulação dos veículos no SUMO GUI e analisar os resultados produzidos pelos detetores virtuais.

14.2 Estrutura da Simulação

A simulação é composta pelos seguintes ficheiros:

Ficheiro	Descrição
osm.net.xml	Rede rodoviária da VCI.
detectors.det.xml	Definição dos detetores virtuais.
edgedata4routeSampler_clean.xml	Contagens históricas convertidas para o formato EdgeData do SUMO.

randomRoutes*.rou.xml	Rotas candidatas geradas automaticamente.
sampledRoutes*.rou.xml	Rotas calibradas utilizadas na simulação.

Os ficheiros de rotas utilizados correspondem ao resultado do processo de calibração.

14.3 Execução

Antes da execução deverá confirmar que o Eclipse SUMO se encontra instalado e a variável de ambiente SUMO_HOME está configurada. A aplicação utiliza os seguintes ficheiros principais:

Ficheiro	Descrição
osm.net.xml	Rede rodoviária da VCI.
detectors.det.xml	Definição dos detetores virtuais.
sampledRoutes2.rou.xml	Rotas calibradas utilizadas na simulação. (geradas pelos scripts em randomTrips/)
main.py	Execução da simulação.
randomTrips/	Scripts para geração e calibração das rotas.

É necessário criar as rotas. Em randomTrips/ executar: `./random.sh` . Este passo gera um conjunto inicial de viagens sobre a rede da VCI. Em randomTrips/routeSampler executar: `./sampler.sh` . Este procedimento utiliza as contagens históricas para selecionar o conjunto de rotas que melhor representa o tráfego observado.

No final é produzido o ficheiro sampledRoutes2.rou.xml, que será utilizado na simulação.

Após confirmar que o ficheiro sampledRoutes2.rou.xml se encontra disponível, a simulação pode ser iniciada:

```
docker build -t vci-simulation .
docker run --rm vci-simulation
```

Durante a execução é carregada a rede da VCI, os detetores e o ficheiro de rotas calibradas, iniciando-se automaticamente a simulação.

14.4 Resultados

No final de cada execução são produzidos vários ficheiros contendo informação sobre a simulação.

Entre os principais resultados encontram-se:

- estatísticas gerais da execução (stats.xml);
- medições dos detetores virtuais;
- tempos de execução da simulação;

- ficheiros gerados pelo SUMO.

14.5 Atualização do modelo

Sendo disponibilizadas novas contagens de tráfego, a procura poderá ser reconstruída repetindo o processo de geração de rotas.

O procedimento consiste em:

1. Atualizar o ficheiro `edgedata4routeSampler_clean.xml` com os novos dados de tráfego;
2. Executar o `randomTrips.py` para gerar um novo conjunto de rotas candidatas;
3. Executar o `routeSampler.py` para reconstruir a procura;
4. Executar novamente a simulação.
- 5.

15 MODELOS ML PARA PREVISÃO DO FLUXO DE TRÁFEGO

15.1 Visão Geral

Esta aplicação permite treinar e avaliar modelos de previsão de fluxo de tráfego a partir de dados históricos provenientes de detetores.

O sistema executa automaticamente todas as etapas necessárias, incluindo:

- pré-processamento dos dados;
- imputação de valores em falta;
- Feature engineering;
- otimização de hiperparâmetros;
- treino dos modelos;
- avaliação do desempenho;
- armazenamento dos modelos treinados;
- registo dos resultados no MLflow.

15.2 Execução

No root do projeto execute:

```
docker compose up --build
```

Na primeira execução serão descarregadas as imagens Docker necessárias e instaladas automaticamente todas as dependências. O treino inicia-se automaticamente.

15.3 Monitorização

Durante a execução é possível acompanhar o progresso através dos logs apresentados no terminal.

Em simultâneo, todas as experiências são registadas no MLflow.

A interface encontra-se disponível em: <http://localhost:5000> .

Nesta interface podem ser consultados:

- modelos treinados;

- métricas obtidas;
- hiperparâmetros selecionados.

15.4 Resultados

Após a conclusão da execução, os resultados são armazenados automaticamente na pasta: output/. Para cada detetor são gerados:

- métricas de desempenho;
- previsões;
- gráficos de análise e resíduos;
- comparação entre modelos.

Os modelos treinados e scalers são igualmente exportados para utilização futura.

15.5 Modelos Treinados

O sistema treina automaticamente quatro modelos distintos para cada detetor:

- SARIMA;
- Random Forest;
- XGBoost;
- LSTM.

16 MODELO AGÊNTICO

16.1 Visão Geral

O modelo agêntico foi desenvolvido para monitorizar autonomamente o estado do tráfego na Via de Cintura Interna (VCI), através da comparação entre os fluxos de tráfego previstos pelos modelos de machine learning e os valores efetivamente observados pelos detetores. O sistema integra dados históricos de tráfego, condições meteorológicas e observações em tempo real para identificar desvios significativos, distinguir eventos localizados de alterações globais do tráfego e produzir uma avaliação contínua do estado da rede.

A arquitetura multiagente, composto por um agente coordenador e por múltiplos agentes especializados, correspondendo cada um a um par detetor-direção. A coordenação entre agentes é realizada através da framework Goose, enquanto a comunicação entre o modelo de linguagem e as funcionalidades da aplicação é efetuada recorrendo ao Model Context Protocol (MCP), implementado com a biblioteca FastMCP.

16.2 Execução

Antes da execução é necessário configurar o ficheiro .env com as credenciais de acesso aos diferentes serviços externos, nomeadamente a plataforma Eclipse Ditto, o repositório Amazon S3 onde se encontram armazenados os modelos preditivos e o fornecedor do modelo de linguagem. Devem ainda ser definidos os parâmetros de execução, como o modo de funcionamento (RUN_MODE) e, caso seja utilizado o modo de teste, o intervalo temporal (TEST_START e TEST_END).

```
docker compose up -build
```

16.3 Fluxo de Execução

Durante a execução, os subagentes descarregam automaticamente do Amazon S3 os modelos correspondentes aos detetores em análise, obtêm os dados históricos e observados através do Eclipse Ditto, executam as previsões e calculam os desvios relativamente aos valores observados. Os resultados individuais são agregados pelo agente coordenador, que produz uma avaliação global do estado da rede.

16.4 Resultados e Análise

Todas as métricas, parâmetros, e *traces* das execuções são registados automaticamente no MLflow, podendo ser consultados através da interface Web disponível em <http://localhost:5000>.

17 DT4MOB PLATFORM

17.1 Introdução

Esta secção fornece instruções sobre como instalar, operar e manter a plataforma Digital Twins for Mobility, doravante designada por plataforma dt4mob ou simplesmente a plataforma.

17.1.1 Visão geral

A plataforma dt4mob é construída sobre uma arquitetura cloud native para poder escalar de forma fácil e eficiente para satisfazer a demanda, o que se traduz num conjunto de micro-serviços independentes que são executados e escalados num cluster Kubernetes.

Adicionalmente, a própria plataforma é concebida para ser modular através da separação de um subconjunto principal de serviços necessários para o funcionamento básico e de um conjunto de addons que contêm os seus próprios serviços para fornecer um conjunto alargado de funcionalidades e responder a casos de usos existentes e emergentes.

Na prática, e para simplificar a operação, isto é conseguido através de um conjunto de charts Helm que formam uma hierarquia.

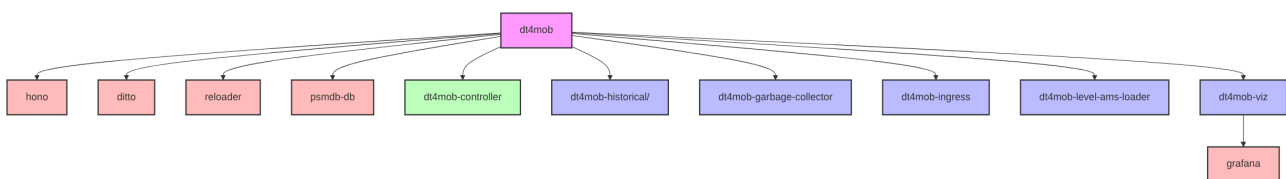


Figura 3 - Helm dependency graph.

O chart raiz dt4mob inclui o núcleo completo da plataforma e todos os addons existentes preparados para fácil instalação. No entanto, se nem todos os componentes forem necessários ou se for necessário mais controlo sobre o processo de instalação, os charts também podem ser instalados individualmente. Mais detalhes sobre todas as opções de instalação disponíveis são fornecidos na Secção 17.2.

17.1.2 Implementação de gémeos digitais

Os gémeos digitais são representações virtuais de objetos, estruturas e sistemas físicos, podendo ser utilizados para um vasto conjunto de finalidades, como monitorização e simulação. Estes casos de uso dependem de informação precisa e atualizada do gémeo físico.

A plataforma dt4mob baseia-se nos projetos Eclipse Ditto™ e Eclipse Hono™ para a sua oferta principal de gémeos digitais. O Ditto define a infraestrutura de backend para armazenar e consultar os gémeos digitais, ou *Things* na nomenclatura do projeto, e o Hono funciona como a camada de conectividade entre os dispositivos na periferia e o Ditto.

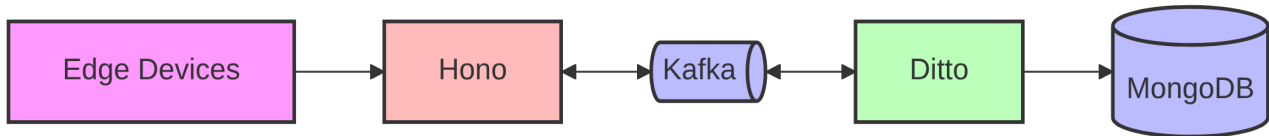


Figura 4 - Digital twins components architecture.

A conectividade com os dispositivos é feita através de um conjunto de adaptadores fornecidos pelo Hono e colocados na periferia, estes fornecem interfaces baseadas num conjunto diverso de protocolos como HTTP, AMQP e MQTT. A comunicação entre o Hono e o Ditto é feita através de uma fila de mensagens Kafka.

Todos estes são protegidos através da utilização de TLS e autenticação mútua baseada em certificados (mTLS). Os certificados são provisionados através de uma Autoridade Certificadora (CA) que reside dentro do cluster, com o seu ciclo de vida e renovação geridos automaticamente.

Para reconciliar a configuração e os certificados durante a operação normal, um serviço chamado dt4mob-controller monitoriza eventos do cluster e, através das APIs de administração oferecidas pelo Hono e pelo Ditto, atualiza a configuração dos componentes para corresponder ao novo estado do cluster.

17.2 Instalação

17.2.1 Pré-requisitos

É necessário um cluster Kubernetes a executar uma das três versões menores mais recentes. Além disso, devido a limitações do empacotamento em charts Helm, os seguintes componentes precisam de ser instalados previamente no cluster:

- [Ingress NGINX Controller](#)
- [cert-manager](#)
- [Strimzi Kafka Operator](#)
- [Percona Operator for MongoDB](#)
- [CloudNativePG](#)
- [SeaweedFS Operator](#)

O tamanho mínimo do cluster necessário para executar todos os componentes essenciais é de 8 vCPUs, 16 GiB de RAM e 200 GiB de armazenamento. Podem ser necessários mais recursos para escalar o cluster de modo a satisfazer a procura e para instalar os addons. Não existe restrição quanto ao número mínimo de nós no cluster, exceto em cenários de alta disponibilidade, onde são necessários pelo menos 3 nós.

17.2.2 Instalação do release

A plataforma dt4mob pode ser implantada de uma só vez utilizando o comando padrão helm install:

```
$ helm install dt4mob --values values.yaml ./dt4mob-platform/charts/dt4mob
```

Infelizmente, devido a limitações técnicas, a release deve ser nomeada dt4mob (É tecnicamente possível usar um nome diferente, mas requer uma grande quantidade de reconfiguração e não é suportado).

O ficheiro values.yaml é obrigatório e deve, no mínimo, ter a seguinte estrutura:

```
global:
  host: dt4mob.av.it.pt

ditto:
  ingress:
    host: dt4mob.av.it.pt

gateway:
  config:
    authentication:
      oauth:
        openidConnectIssuers:
          keycloak:
            issuer: "dt4mob.av.it.pt/auth/realms/dt4mob"
```

O valor de host em ambas as instâncias é o nome de domínio onde a plataforma será exposta. Pode ser um domínio público, um domínio interno ou até mesmo um domínio temporário, mas os valores tem de ser definidos. O valor de issuer deve ser definido manualmente, devido a limitações do empacotamento Helm, para o mesmo valor de global.host com /auth/realms/dt4mob acrescentado no final.

Poderá também ser relevante alterar o valor de global.tenant do seu valor predefinido test-tenant para um nome mais apropriado. Isto alterará o nome do tenant criado no Hono e também o nome das conexões criadas no Ditto. Este valor pode ser alterado posteriormente, mas requer trabalho manual para migrar todas as utilizações anteriores do nome do tenant antigo para o novo.

17.2.3 Abertura da plataforma ao exterior

A plataforma tem duas partes que podem ser expostas separadamente e através de mecanismos diferentes:

- Adaptadores do Hono
- APIs da plataforma

As seguintes secções abordam cada parte individualmente.

17.2.4 Adaptadores Hono

A instalação padrão da plataforma expõe os adaptadores Hono através da utilização de serviços Kubernetes NodePort; as portas são alocadas dinamicamente pelo cluster para garantir que não entram em conflito com alocações existentes. O seguinte comando pode ser utilizado para obter as portas alocadas.

```
$ kubectl get svc dt4mob-hono-adapter-http dt4mob-hono-adapter-mqtt dt4mob-hono-adapter-amqp
```

Contactar qualquer um dos nós do Cluster Kubernetes nestas portas encaminhará o tráfego para o respetivo adaptador Hono. Se, em alternativa, for desejável expor estes serviços utilizando um balanceador de carga (cloud ou on-prem), a seguinte alteração ao ficheiro de valores deve ser aplicada.

```
hono:
  useDynamicNodePorts: false
  useLoadBalancer: true
```

A comunicação com os adaptadores deve é feita através de TLS para todos os protocolos, isto é necessário não só para a proteção dos dados enviados e recebidos, mas também para a autenticação dos dispositivos que é feita através da utilização de certificados X.509. Todos os certificados utilizados para comunicação com o Hono são assinados por um certificado raiz autoassinado cujo ciclo de vida é gerido pelo cluster e, como tal, deve ser adicionado ao trust store dos dispositivos de perímetro. O certificado raiz pode ser obtido utilizando o seguinte comando:

```
$ kubectl get secrets dt4mob-tenant-ca-secret -o
jsonpath="{.data['ca\.crt']}" | base64 -d
```

Os certificados dos adaptadores incluirão sempre o nome de domínio configurado na opção `global.host` nos valores da extensão Subject Alternative Name (SAN). No entanto e em alguns casos, o DNS pode não estar disponível, não ser desejado, ou o domínio pode não resolver externamente, e como tal pode ser necessário conexão direta aos adaptadores através do endereço IP dos nós ou dos balanceadores de carga. Para tais cenários, os valores SAN podem ser estendidos com endereços IP arbitrários através da seguinte modificação ao ficheiro `values.yaml`.

```
dt4mob-ingress:
  addresses:
    - 193.136.93.66
```

Em certas configurações de rede e especialmente de firewall, o acesso a serviços através de portas não padrão pode ser impossível. Para combater este problema, é possível implementar ingressos que encaminham o tráfego com base no Server Name Indication (SNI) do TLS Client Hello. Na prática, isto permite que os adaptadores sejam expostos através da porta HTTPS padrão 443 em diferentes domínios que são então encaminhados para os seus respetivos adaptadores sem necessidade de qualquer encapsulamento. Para ativar isto, o seguinte excerto deve ser adicionado ao ficheiro de valores. Cada adaptador pode ter o seu ingresso ativado individualmente. Estes tem necessariamente de ter nomes de domínio diferentes para o host, que também tem de ser diferentes do configurado para a plataforma em `global.host`.

```
dt4mob-ingress:
  http:
    enabled: true
    host: "dt4mob-ingress.av.it.pt"
  mqtt:
    enabled: true
    host: "mqtt.dt4mob-ingress.av.it.pt"
  amqp:
    enabled: true
    host: "amqp.dt4mob-ingress.av.it.pt"
```

Não são necessárias mais alterações à configuração dos certificados dos adaptadores, pois estes irão automaticamente recolher os valores configurados para os ingressos nos seus valores SAN.

17.2.5 APIs da plataforma

As APIs da plataforma são expostas através do nome de domínio configurado na opção `global.host`. No entanto, isto, por predefinição, é feito através de um ingresso utilizando um certificado autoassinado. O `cert-manager` é utilizado para a emissão e ciclo de vida do certificado. Assim, para utilizar um certificado emitido por uma CA, o emissor tem de ser configurado. Isto pode ser feito de duas formas, utilizando um emissor presente no cluster:

```
ingressTLS:
  issuer:
    existingIssuer:
      name: "my-cluster-issuer"
      kind: ClusterIssuer
      group: cert-manager.io
```

Ou modificando o emissor existente para não utilizar certificados autoassinados e, em vez disso, utilizar, por exemplo, um Automatic Certificate Management Environment (ACME), como o Let's Encrypt:

```
ingressTLS:
  issuer:
    spec:
      selfSigned: null
      acme:
        server: https://acme-v02.api.letsencrypt.org/directory
        privateKeySecretRef:
          name: dt4mob-letsencrypt-issuer-account-key
        solvers:
          - http01:
              ingress:
                ingressClassName: nginx
```

Para mais detalhes, consulte a documentação do `cert-manager` para emissores.

17.2.6 Configuração de backups

O estado da plataforma é mantido principalmente numa base de dados MongoDB. Várias cópias de segurança devem ser feitas periodicamente para garantir tolerância a falhas e continuidade de negócio. As cópias de segurança são geridas através do `Percona Operator for MongoDB` e requerem configuração adicional nos valores do chart para configurar as cópias de segurança e a onde estas vão ser armazenadas. O exemplo seguinte mostra a configuração para cópias de segurança guardadas num servidor Network File System (NFS) com recuperação Point-In-Time (PITR) e compressão `zstd`.

```
psmdb-db:
  replsets:
    rs0:
      sidecarVolumes:
        - name: backup-nfs-vol
          nfs:
            server: "192.168.100.100"
            path: "/var/nfs/export/staging-mongodb-backup"
      backup:
        enabled: true
        volumeMounts:
          - mountPath: /home/mongodb/nfs
            name: backup-nfs-vol
```

```

storages:
  backup-nfs:
    type: filesystem
    filesystem:
      path: /home/mongodb/nfs
pitr:
  enabled: true
  compressionType: zstd
  compressionLevel: 2

```

Esta configuração permite a criação de cópias de segurança a pedido. Para cópias de segurança periódicas, a opção `psmdb-db.backup.tasks` pode ser utilizada para definir um agendamento periódico das cópias de segurança e até múltiplos agendamentos para diferentes tipos de cópia de segurança. O exemplo seguinte mostra uma configuração para cópias de segurança incrementais diárias com cópias de base completas no início de cada semana e com períodos de retenção configurados.

```

psmdb-db:
  backup:
    tasks:
      - name: daily-incremental
        enabled: true
        schedule: "0 1 * * *"
        type: incremental
        retention:
          count: 25
          type: count
          deleteFromStorage: true
        storageName: backup-nfs
        compressionType: zstd
        compressionLevel: 2
      - name: weekly-base
        enabled: true
        schedule: "0 5 * * 0"
        type: incremental-base
        retention:
          count: 5
          type: count
          deleteFromStorage: true
        storageName: backup-nfs
        compressionType: zstd
        compressionLevel: 2

```

Para mais detalhes, consulte a documentação do Percona Operator for MongoDB sobre cópias de segurança.

17.2.7 Configuração do bucket S3

A instalação padrão, por predefinição, espera um recurso CRD Seaweed no namespace `seaweedfs` com o nome `seaweed` para criar o seu bucket. A localização do recurso Seaweed pode ser alterada adicionando e configurando os seguintes valores no ficheiro `values.yaml`:

```

bucketConfig:
  clusterRef:
    name: seaweed
    namespace: seaweedfs

```

O bucket, por predefinição, está configurado para ser acessível ao público sem autenticação. Isto pode ser alterado utilizando o seguinte excerto no ficheiro de valores:

```
bucketConfig:  
  extraConfig:  
    anonymousRead: false
```

17.2.8 Outras configurações

Outras configurações tipicamente esperadas de um Helm Chart, como o número de réplicas, recursos do pod, afinidades, seletores de nós, entre outros, também são possíveis utilizando as convenções normais do Helm. Para mais detalhes sobre todas as configurações disponíveis, consulte o ficheiro `values.yaml` do chart ou obtenha-as executando o seguinte comando:

```
$ helm show values ./dt4mob-platform/charts/dt4mob
```

17.3 Operação

As seguintes secções documentam procedimentos padrão para a operação regular da plataforma.

17.3.1 Gestão de utilizadores

A plataforma utiliza uma abordagem de gestão centralizada de utilizadores construída sobre o Keycloak e o OpenID Connect (OIDC daqui em diante) para permitir que os utilizadores tenham uma experiência de Single Sign On (SSO) entre todos os diferentes serviços da plataforma. Para os serviços partilham lógica de autenticação e autorização, e para os administradores poderem gerir facilmente os utilizadores e os seus cargos.

Ao instalar ou atualizar a plataforma, uma tarefa de manutenção é iniciada automaticamente, que criará as configurações necessárias do Keycloak, incluindo um realm, um cliente e cargos para administração e para os serviços do sistema funcionarem corretamente. Como tal, os administradores não precisam de configurar contas de serviço e/ou cargos para a plataforma começar a funcionar normalmente, necessitando apenas de configurar contas de utilizador para participantes externos conforme necessário.

17.3.2 Aceder à consola de administração do Keycloak

O Keycloak é exposto no caminho <https://<host>/auth>, onde <host> é o nome de domínio configurado na Secção 17.2. Navegar diretamente para /auth redirecionará para um formulário de login que controla o acesso à consola de administração. Uma conta de administração é criada automaticamente durante a instalação com o nome de utilizador `admin` e uma palavra-passe gerada de forma segura, que pode ser recuperada com acesso ao cluster Kubernetes executando o seguinte comando:

```
$ kubectl get secrets dt4mob-keycloak-admin -o jsonpath="{.data.password}" |  
base64 -d
```

Isto imprimirá a palavra-passe no standard output do terminal. Utilizando as credenciais obtidas e submetendo o formulário de login, é permitido o acesso à consola de administração do Keycloak.

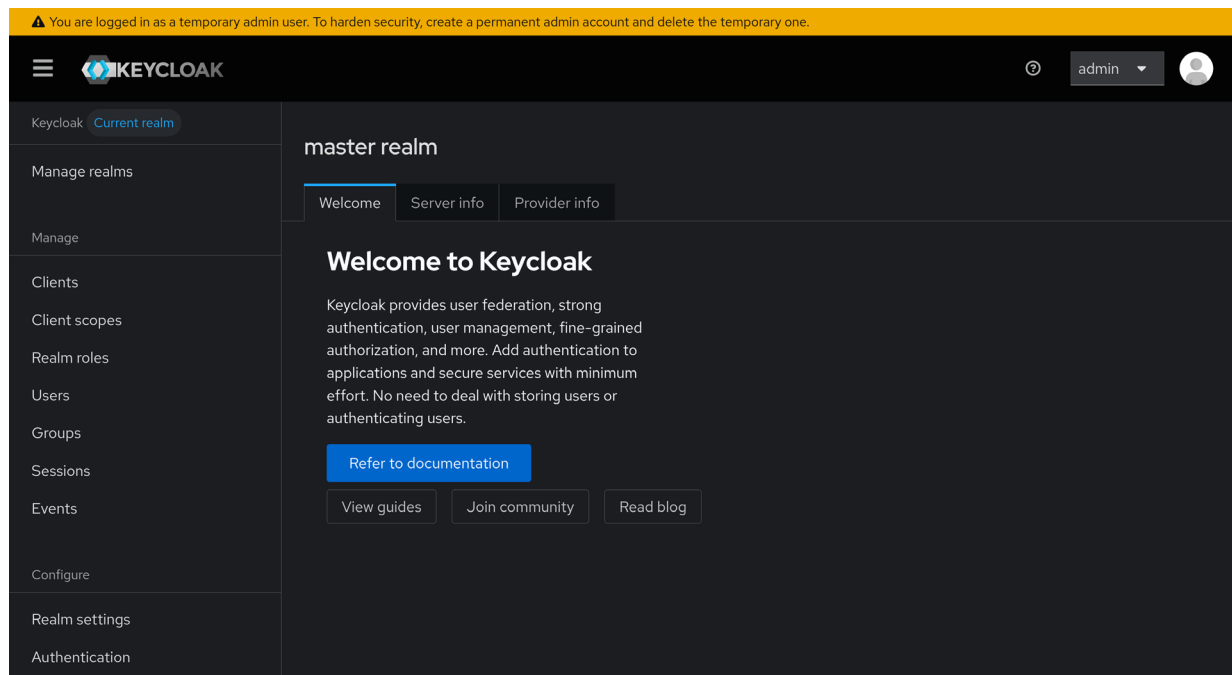


Figura 5 - Consola de administração do Keycloak

O Keycloak exibirá um aviso a indicar que a conta é um utilizador administrador temporário e para criar uma conta de administrador permanente. O Keycloak é provisionado exclusivamente para a plataforma, e as credenciais para o utilizador admin são geradas de forma segura; como tal, não é estritamente necessário criar uma conta de administrador permanente.

No entanto, pode ser útil fazê-lo para definir uma conta de administração mais granular ou aumentar a segurança da conta de administrador, por exemplo, adicionando 2FA. Para tal, consulte a documentação do Keycloak para mais instruções.

As próximas secções vão incidir todas sobre o realm dt4mob; para trocar o realm selecionado, clique no link "Manage realms" na barra lateral esquerda e depois no nome do realm dt4mob, que deve estar destacado como um link.

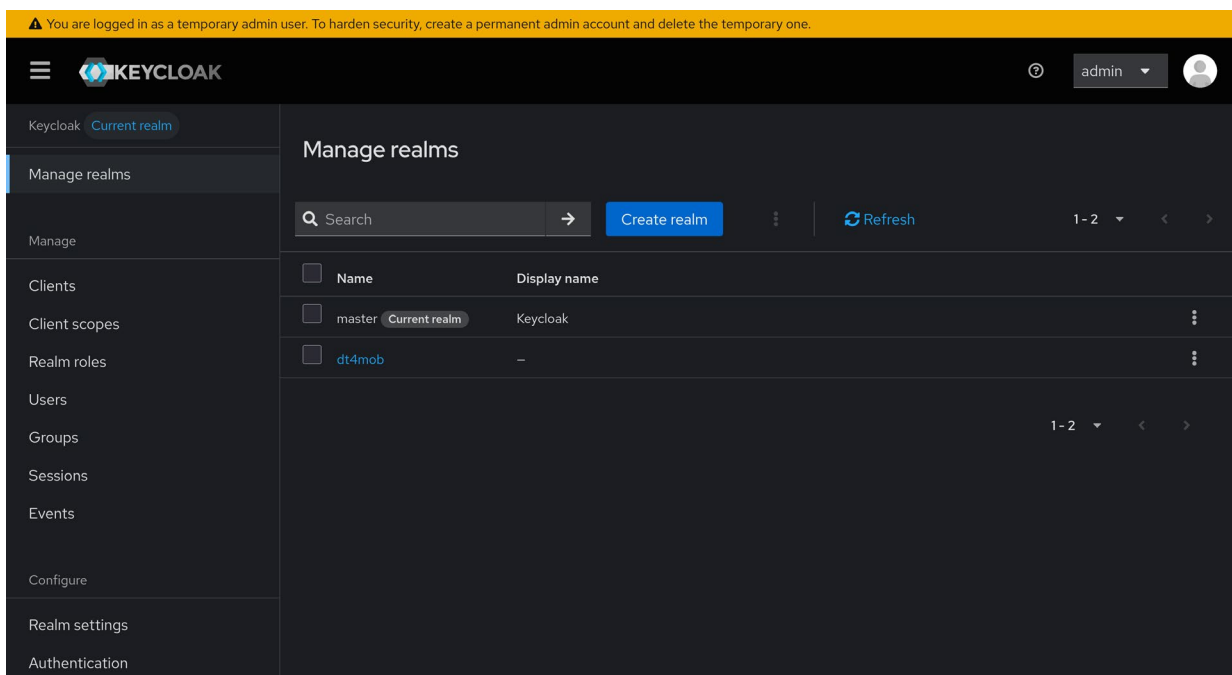


Figura 6 - Seleção de realms no Keycloak

17.3.3 Adicionar um utilizador

Uma instalação nova conterá apenas os utilizadores dos serviços da plataforma. Para criar uma conta de utilizador para uma entidade externa poder interagir com a plataforma, comece por navegar para o link “Users” na barra lateral esquerda (certifique-se de que o realm atual é o realm dt4mob).

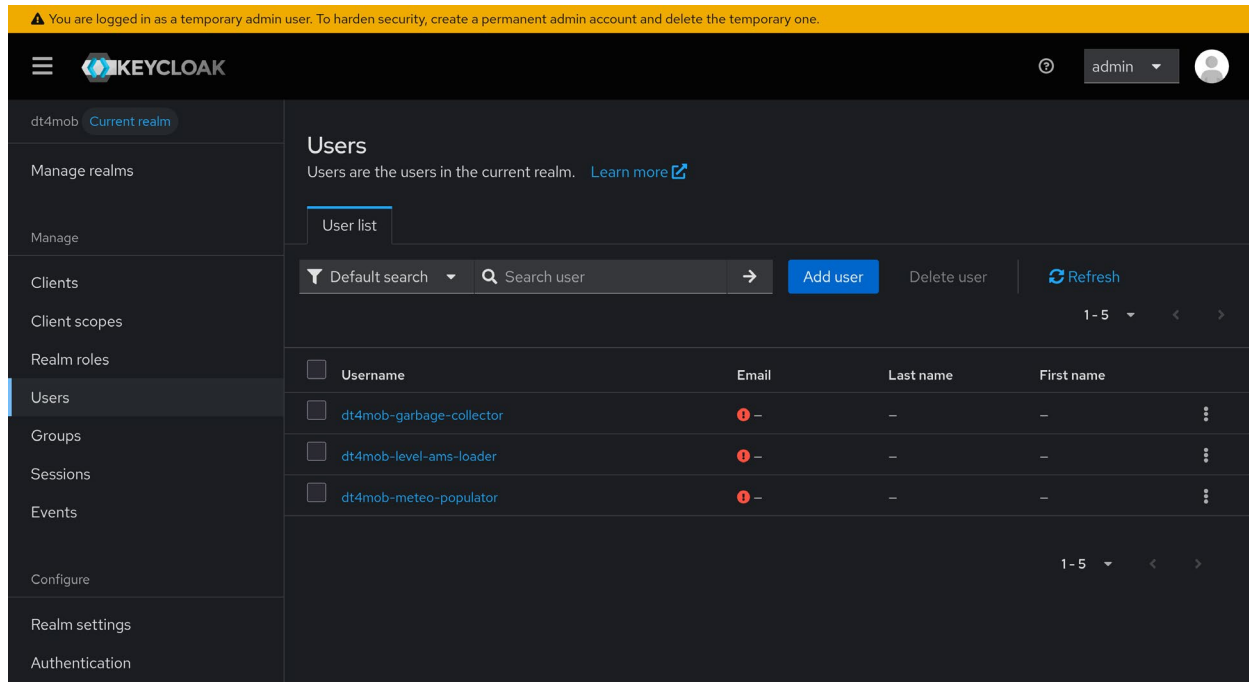


Figura 7 - Listagem de utilizadores do realm

Em seguida, pressione o botão “Add user” na interface; isto abrirá um formulário para criar um novo utilizador. O único campo obrigatório é o nome de utilizador, que deve ser único e será utilizado na autenticação. No entanto, se o utilizador tiver de ter acesso à instância de visualização Grafana, então um email também tem de ser especificado, caso contrário o utilizador não conseguirá aceder mesmo com os cargos corretos. Os outros campos podem ser deixados vazios ou preenchidos conforme necessário; para mais detalhes, consulte a documentação do Keycloak.

Submeter o formulário criará o utilizador e redirecionará para a sua página de detalhes.

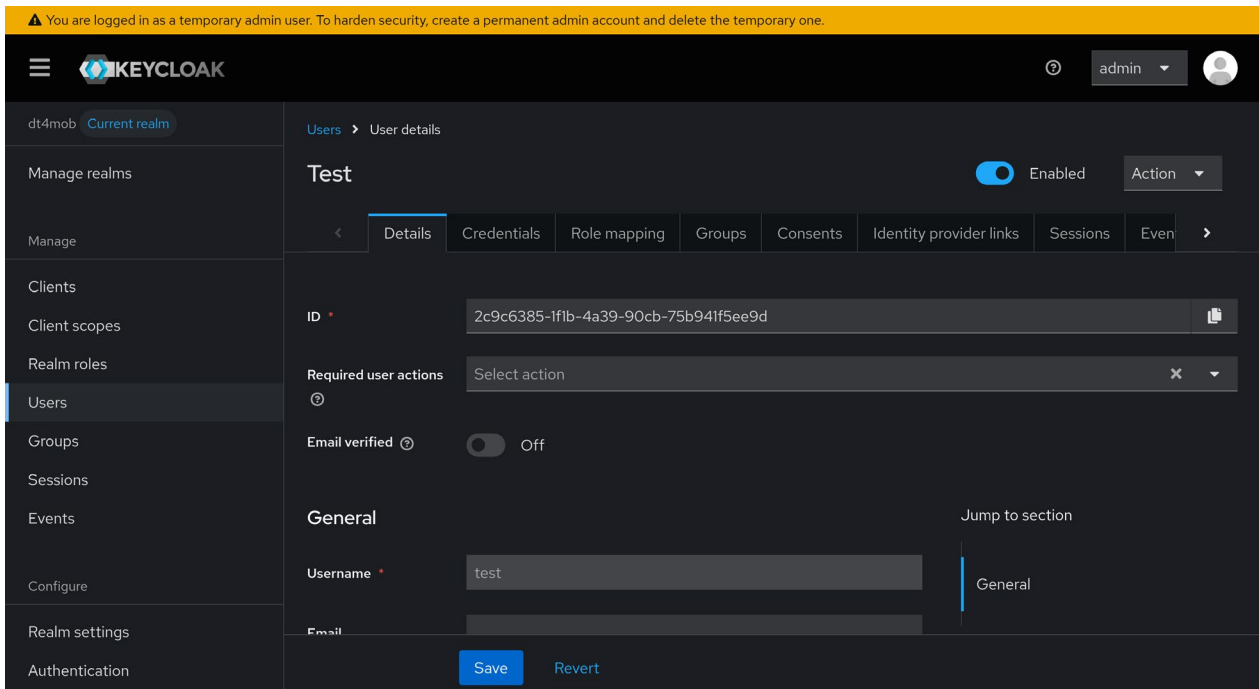


Figura 8 - User details page

Neste ponto, ainda não será possível utilizar a conta, pois não tem quaisquer credenciais para usar na autenticação. Para atribuir uma palavra-passe, navegue para o separador “Credentials” na página de detalhes do utilizador e pressione o botão “Set password”. Isto abrirá um popup com um formulário para definir uma palavra-passe para o utilizador.

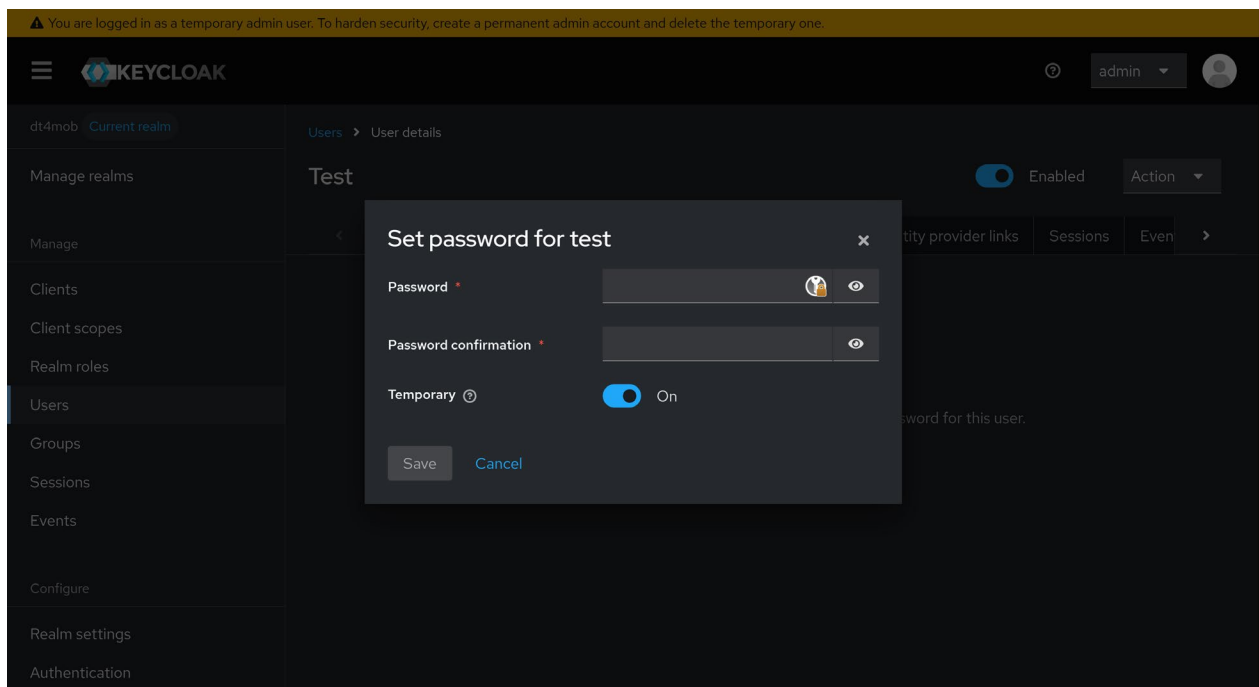


Figura 9 - Setting the user's password

A palavra-passe pode ser definida como temporária (opção por predefinição), que implicará a solicitação ao utilizador da sua alteração após o primeiro login. Configure esta opção conforme relevante para o utilizador criado. Preencha o formulário com uma palavra-passe adequada, confirme-a e submeta o formulário. O utilizador terá agora credenciais na forma do seu nome de utilizador e da palavra-passe que acabou de ser configurada. Estas devem ser comunicadas à entidade externa através de um canal seguro.

17.3.4 Atribuição de cargos

Uma conta recém-criada não terá quaisquer cargos na plataforma atribuídos. Isto permite-lhe aceder ao Ditto e ao serviço de visualização e interagir com things que são permitidas pelas suas respetivas políticas.

No entanto, para aceder a outros serviços, como a API de dados históricos, o serviço de emissão de certificados, ou até mesmo para ter acesso administrativo, tem de ser atribuídos cargos ao utilizador.

Para atribuir um cargo ao utilizador, navegue para a sua página de detalhes, depois para o separador “Role mapping” e pressione o botão “Assign Role”. Isto abrirá um dropdown com duas opções; selecione a opção “Client roles”.

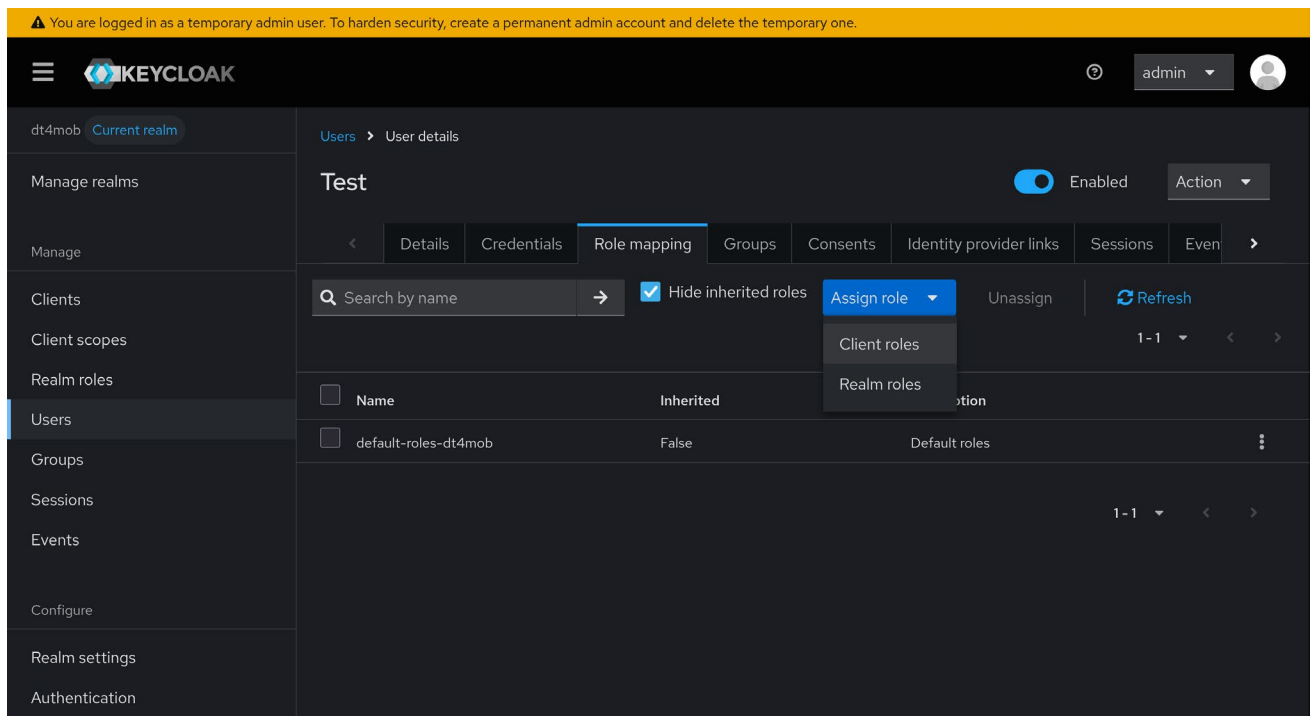


Figura 10 - Atribuição de cargos

Isto abrirá um popup com uma lista de cargos que podem ser atribuídos ao utilizador; destes, apenas os que têm o Client ID ditto são relevantes para autorização no contexto dos serviços da plataforma.

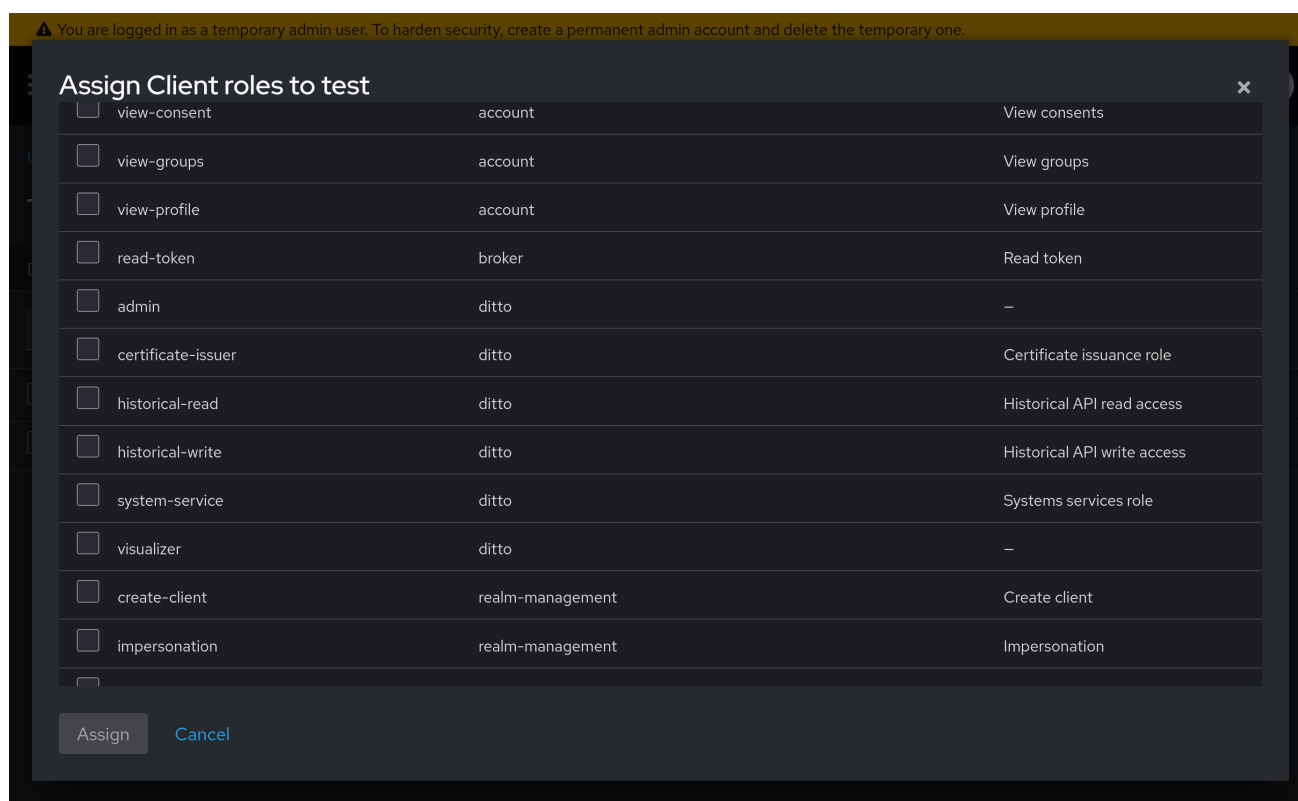


Figura 11 - Seleção de cargos da plataforma

Os cargos podem ser criados por administradores para uso em políticas Ditto, e a própria plataforma define um conjunto de cargos especiais que concedem acesso a serviços extra dentro da plataforma. A tabela abaixo lista estes cargos especiais e o acesso extra que concedem.

Cargo	Acesso concedido
admin	Concede acesso total à plataforma, incluindo todas as things, todos os serviços e capacidade de gerar certificados personificando outros utilizadores.
historical-read	Concede acesso de leitura à API de dados históricos. Deve notar-se que não existe noção de políticas nos dados históricos e, como tal, todas as things são consultáveis por qualquer pessoa com este cargo.
historical-write	Concede acesso de escrita à API de dados históricos, permitindo o preenchimento retrospectivo de informações de outros sistemas.
certificate-issuer	Concede acesso ao serviço de emissão de certificados, permitindo ao utilizador obter certificados de cliente válidos que podem ser usados para conetar dispositivos ao Hono.

Tabela 1: Cargos especiais da plataforma

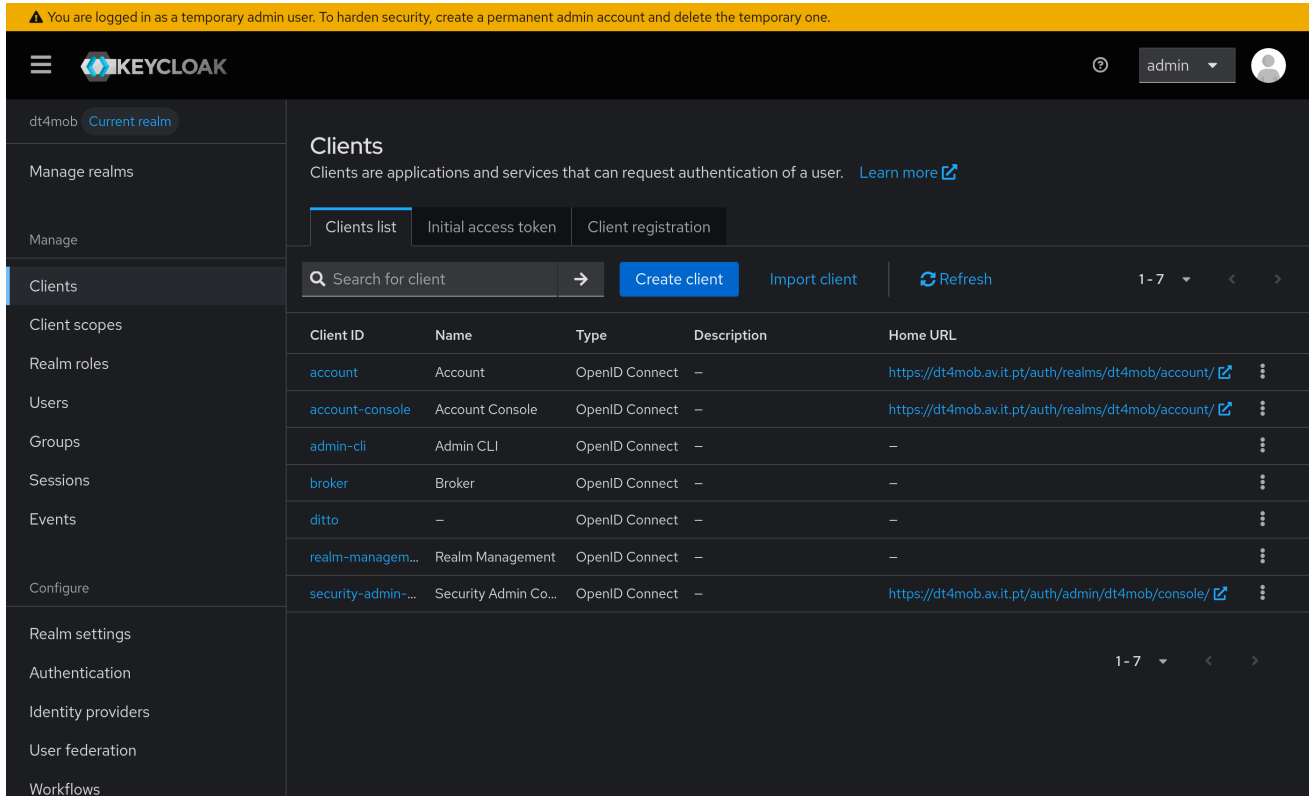
Basta seleccionar os cargos que o utilizador deve ter e finalizar clicando em “Assign” na parte inferior do popup. Apesar dos cargos serem atribuídos instantaneamente, as sessões mais antigas ainda estarão a usar tokens mais antigos que não têm esses cargos. Isto pode ser resolvido aguardando que o token seja atualizado, momento em que terá os cargos, ou fazendo logout e login novamente. Isto fará com que um novo token seja gerado, que conterá a nova atribuição de cargos.

17.3.5 Criação de cargos

Como mencionado anteriormente, os administradores podem criar cargos adicionais que podem então ser usados em políticas para controlar o acesso a things dentro da plataforma. Isto pode ser

útil, pois permite definir políticas de autorização em termos de cargo do utilizador em vez de identidade. Por exemplo, os técnicos podem precisar de acesso a todas as portagens da empresa no país. Em vez de definir cada técnico na política para as portagens, pode ser criada um cargo e atribuído aos técnicos conforme necessário, e a política só precisa de conceder acesso ao cargo uma vez.

Para definir uma novo cargo, comece por navegar para o link “Clients” na barra lateral esquerda (certifique-se de que o realm atual é o realm dt4mob). Deve aparecer uma lista de clients, incluindo os usados internamente pelo Keycloak e o client ditto, que é o client usado pela plataforma.



Keycloak Admin Console - Clients

dt4mob **Current realm**

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Clients

Clients are applications and services that can request authentication of a user. [Learn more](#)

Clients list Initial access token Client registration

Search for client → Create client Import client Refresh 1 - 7

Client ID	Name	Type	Description	Home URL
account	Account	OpenID Connect	–	https://dt4mob.av.it.pt/auth/realms/dt4mob/account/
account-console	Account Console	OpenID Connect	–	https://dt4mob.av.it.pt/auth/realms/dt4mob/account/
admin-cli	Admin CLI	OpenID Connect	–	–
broker	Broker	OpenID Connect	–	–
ditto	–	OpenID Connect	–	–
realm-managem...	Realm Management	OpenID Connect	–	–
security-admin-...	Security Admin Co...	OpenID Connect	–	https://dt4mob.av.it.pt/auth/admin/dt4mob/console/

1 - 7

Figura 12 - Listagem de clientes no realm

Clicando no link para o cliente ditto abrirá a sua página de detalhes do cliente; a partir daqui, navegue para o separador “Roles”, que exibirá uma lista de todos os cargos do cliente.

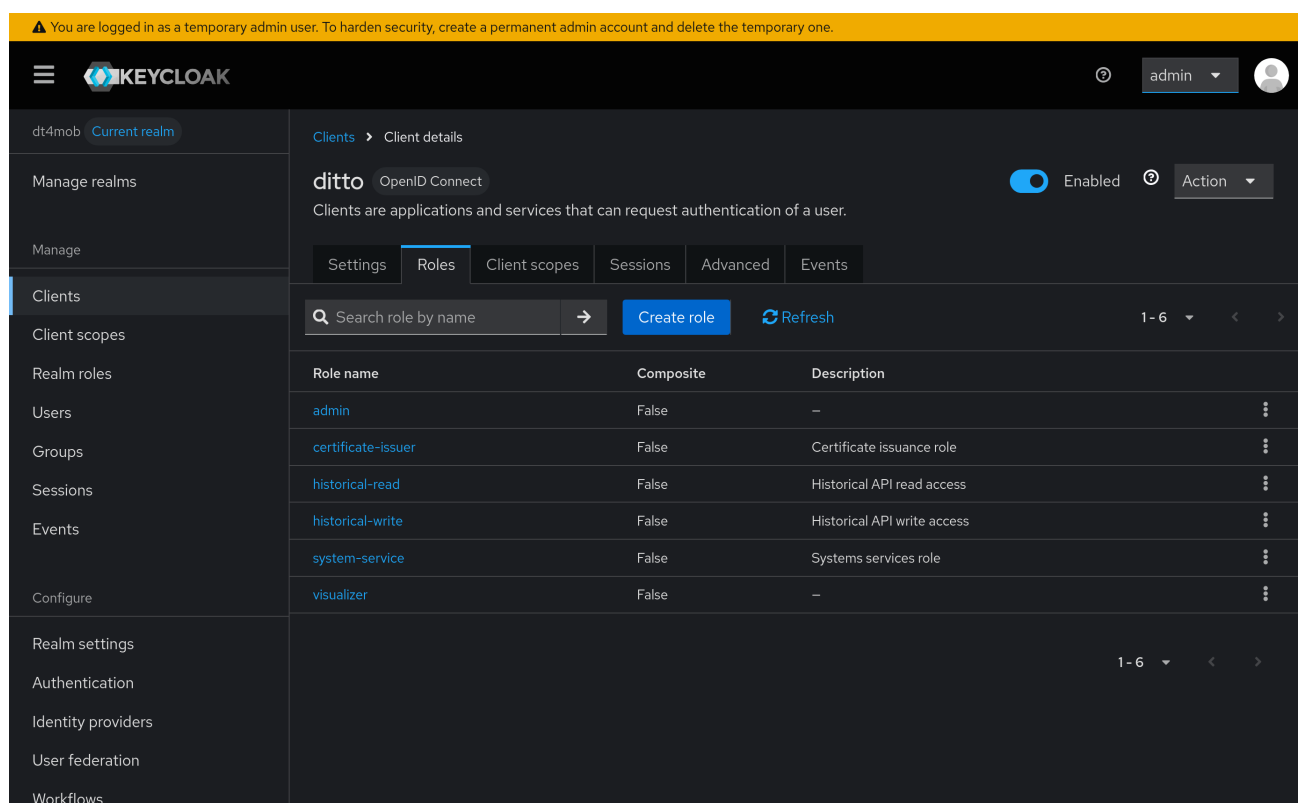


Figura 13 - Listagem de cargos do cliente

A partir daqui, pressione o botão “Create role”, que irá navegar para o formulário de criação de cargo. Insira o nome do novo cargo e opcionalmente uma descrição para o mesmo, e submeta o formulário para concluir a criação do cargo. Atribua o cargo a um utilizador conforme explicado na Secção 3.1.3 para começar a utilizá-lo.

17.3.6 Ditto

Como mencionado anteriormente, o Eclipse Ditto é o backend que permite o armazenamento e consulta de gémeos digitais na plataforma DT4Mob. As seguintes secções explicam como aceder ao Ditto, criar políticas e obter privilégios DevOps.

17.3.7 Aceder ao Ditto

O Ditto é exposto nos caminhos <https://<host>/ui> e <https://<host>/api>, onde <host> é o nome de domínio configurado na Secção 17.2, para a Interface Web e a API, respetivamente. Navegar para a Interface Web redirecionará automaticamente para o formulário de login do Keycloak, se o utilizador ainda não tiver uma sessão de autenticação válida. Não é necessário nenhum cargo para interagir com o Ditto. O acesso à API requer que o token de acesso seja obtido previamente e passado como um bearer token no cabeçalho Authorization.

17.3.8 Definição de políticas

As políticas controlam o acesso a si próprias e a things dentro do Ditto. Uma política é composta por um conjunto de entradas, cada uma definindo para um conjunto de subjects as operações de recurso permitidas. Uma thing tem exatamente uma política associada que controla o seu acesso. As políticas podem importar umas das outras para permitir reutilizar lógica para decisões de autorização comuns. Para mais informações, consulte a documentação relevante.

Uma política pode ser criada através da Interface Web no separador “policies” ou através da API. Para criar uma política através da Interface Web, certifique-se de que nenhuma política está selecionada e navegue para o separador “JSON” no painel direito (não na barra de navegação).

Figura 14 - Painel de políticas

A partir daqui, pressione o botão “create” no painel direito para começar a editar a nova política; não será possível editar nenhum dos campos antes de o fazer. Defina um ID de política único que deve seguir o formato <namespace>:<name> e escreva a definição da política na caixa de texto abaixo. Para detalhes sobre a sintaxe de definição de política, consulte a documentação do Ditto.

No painel esquerdo, na secção “Who am I”, encontra-se uma lista de todos os subjects que o utilizador atual representa. Estes são diretamente derivados do token de acesso obtido do Keycloak. A tabela abaixo lista os diferentes subjects extraídos do token que podem estar em definições de política:

Subject	Descrição
keycloak:sub:<uid>	O campo sub do token de acesso, identifica o utilizador de forma única. Não deve ser usado em definições de política, pois não é óbvio quem é o utilizador real a partir do UID.
keycloak:user:<uid>	O nome de utilizador extraído do token de acesso, identifica o utilizador de forma única neste momento, mas pode ser reutilizado para outro utilizador mais tarde se este for eliminado e um novo for criado. Preferível a keycloak:sub, mas na maioria dos casos devem ser usados cargos.
keycloak:role:<uid>	Cargos do utilizador extraídos do token de acesso. Geralmente a melhor opção ao definir políticas, pois permite definir acesso por cargo em vez de identidade do utilizador, sendo também mais fácil de gerir.

Tabela 2: Subjects obtidos a partir do token de acesso

17.3.9 Permitir mensagens do Hono

As políticas precisam de incluir um subject especial para permitir que mensagens sejam aceites da conexão Hono para things controladas pela política. Isto é feito adicionando a seguinte entrada à política:

```
"HONO": {
  "subjects": {
    "pre-authenticated:hono-connection-<tenant>": {
      "type": "Connection to Eclipse Hono"
    }
  },
  "resources": {
    "thing/": {
      "grant": [
        "READ",
        "WRITE"
      ],
      "revoke": []
    },
    "message/": {
      "grant": [
        "READ",
        "WRITE"
      ],
      "revoke": []
    },
    "policy/": {
      "grant": [
        "READ"
      ],
      "revoke": []
    }
  },
  "importable": "implicit"
}
```

Onde <tenant> é o nome do tenant que foi configurado na Secção 17.2.

17.3.9.1.1 Permitir exportação histórica

A conexão do exportador histórico também requer uma entrada especial para permitir que a evolução das things seja armazenada. Isto permite que as things sejam excluídas da base de dados histórica, de modo a que apenas as things estritamente necessárias sejam guardadas, ajudando com custos, desempenho e conformidade.

```
"KAFKA_EXPORT": {
  "subjects": {
    "pre-authenticated:kafka-export-connection-<tenant>": {
      "type": "Kafka Connection"
    }
  },
  "resources": {
    "thing/": {
      "grant": [
        "READ"
      ],

```

```

    "revoke": []
  },
  "policy:/{": {
    "grant": [
      "READ"
    ],
    "revoke": []
  }
},
"importable": "implicit"
}

```

Onde <tenant> é o nome do tenant que foi configurado na Secção 17.2.

17.3.10 Acesso DevOps

Algumas operações, como gerir conexões de entrada (Hono) e de saída (exporter histórico) ou executar operações de administração no Ditto, requerem a utilização de uma conta especial chamada DevOps. Esta conta não utiliza a gestão centralizada de utilizadores, sendo as suas credenciais geradas durante a instalação no cluster.

O nome de utilizador é devops, e a palavra-passe pode ser recuperada usando o seguinte comando:

```
$ kubectl get secret dt4mob-ditto-gateway-secret -o jsonpath="{.data.devops-
password}" | base64 -d
```

Estas credenciais podem então ser usadas através de autenticação básica nas APIs que requerem a utilização desta conta ou na Interface Web pressionando o botão “Authorize” no extremo direito da barra de navegação. Isto abrirá um popup com dois formulários para autenticação. O de cima é para autenticação regular usando Keycloak, enquanto que o de baixo é para autenticação DevOps.

Ao inserir as credenciais e submeter o formulário, o acesso às páginas “Connections” e “Operations” será possível sem receber erros de autenticação em falta.

Nota: Por vezes, o botão de rádio “Basic” junto ao formulário de login DevOps não está selecionado (isto deve-se a um bug no Ditto), e a autenticação não prosseguirá nesse caso. Para corrigir, basta selecionar o botão de rádio e submeter o formulário novamente.